GNX-py User Manual



GNX-py User Manual

GNSS data processing and modelling toolkit

Version 1.0.0

Hubert Pierzchała

October 18, 2026

Space Research Centre
Polish Academy of Sciences

GNX-py is released under the MIT License. The examples use public data and products of the International GNSS Service (IGS) and the EUREF Permanent Network (EPN); see THIRD_PARTY_NOTICES.

Contents

1 Introduction **1**
- 1.1 Introduction to GNX-py . . . 1
- 1.2 GNX-py library . . . 1
  - 1.2.1 Installation for this tutorial . . . 2
  - 1.2.2 Sample data used here . . . 3
  - 1.2.3 Downloading data for your own project . . . 3
  - 1.2.4 Current GNSS signal model . . . 4
- 1.3 Example of processing in GNX-py . . . 5
  - 1.3.1 Reading observations with `GNSSDataProcessor2` . . . 5
  - 1.3.2 Antenna metadata: satellite and receiver PCO . . . 7
  - 1.3.3 Reading broadcast navigation products . . . 7
  - 1.3.4 Broadcast coordinates interpolation algorithm . . . 8
  - 1.3.5 Signal emission time . . . 9
  - 1.3.6 Earth rotation / Sagnac correction . . . 11
  - 1.3.7 Elevation, azimuth, and a simple observation mask . . . 11
  - 1.3.8 Klobuchar ionosphere model . . . 12
  - 1.3.9 Troposphere models . . . 13
  - 1.3.10 Relativistic path range correction . . . 14
  - 1.3.11 Receiver PCO and antenna height as line-of-sight corrections . . . 14
  - 1.3.12 Manual SPP observation model . . . 16
- 1.4 SPP Pipeline in GNX-py . . . 17
  - 1.4.1 `SPPResult`: solution, residuals, and diagnostics . . . 19
  - 1.4.2 Interpreting position errors and RMS . . . 21
- 1.5 Summary . . . 22

2 Precise Point Positioning **23**
- 2.1 PPP in GNX-py . . . 23
  - 2.1.1 Parameters estimation in PPP . . . 23
- 2.2 Combined and uncombined PPP . . . 24
  - 2.2.1 PPP combined . . . 24
  - 2.2.2 PPP uncombined . . . 25
  - 2.2.3 Practical comparison . . . 25
  - 2.2.4 Overview of PPP module . . . 29
  - 2.2.5 Current GNX-py PPP capability map . . . 33
  - 2.2.6 Observation models: site displacement, satellite attitude and BeiDou clocks . . . 33
  - 2.2.7 Example 1 - Conventional combined PPP . . . 34
  - 2.2.8 Example 2 - Undifferenced & Uncombined PPP . . . 41
- 2.3 PPP-AR: ambiguity resolution . . . 45
  - 2.3.1 PPP-AR parameters in `PPPConfig` . . . 46
  - 2.3.2 Lightweight PPP-AR experiment . . . 46

3 Orbit and SIS Analysis **51**
- 3.1 GNX Orbit module . . . 51
  - 3.1.1 Algorithms and methods . . . 51
  - 3.1.2 Broadcast ephemeris selection . . . 52
  - 3.1.3 Satellite coordinates and PCO . . . 52

3.1.4 RAC frame and SISRE . . . 53
3.1.5 Satellite clocks, TGD and biases . . . 55
3.1.6 Satellite eclipse periods . . . 55
3.2 Notebook setup . . . 55
3.3 Practical configuration with SISConfig . . . 57
3.4 Example 1 - Broadcast and precise products comparison . . . 59
3.4.1 GPS: broadcast vs precise . . . 59
3.4.2 Satellite attitude during noon and midnight turns . . . 62
3.4.3 Galileo: broadcast vs precise . . . 63
3.4.4 Galileo broadcast record selection . . . 64
3.5 BeiDou in SIS analysis . . . 65
3.5.1 Legacy BDS B1IB3I . . . 65
3.5.2 Antenna reference point and satellite generations . . . 67
3.5.3 Clock datum, TGD, DCB, OSB and DSB . . . 68
3.5.4 Modern BDS and GEO . . . 70
3.5.5 PRN reassignment . . . 70
3.6 Example 2 - SP3 vs SP3 product comparison . . . 71
3.7 What to try next . . . 73

4 Ionosphere Introduction 75
4.1 Ionospheric delay in GNSS observations . . . 75
4.1.1 Definition of STEC . . . 75
4.1.2 Ionospheric delay in code and phase . . . 76
4.1.3 Observation equations . . . 76
4.1.4 Geometry-free combination . . . 76
4.1.5 Biases and STEC observables . . . 77
4.1.6 Phase-to-code leveling . . . 77
4.1.7 Mapping STEC to VTEC . . . 78
4.2 Current GNX-py ionosphere workflow . . . 78
4.3 Notebook setup . . . 78
4.4 Example of STEC processing in GNX-py . . . 80
4.4.1 TECConfig for this tutorial . . . 80
4.4.2 Running GPS STEC . . . 82
4.5 Inspecting STEC arcs . . . 83
4.6 Estimating VTEC above the station . . . 85
4.7 Comparing measured STEC with models . . . 87
4.7.1 BDGIM from a RINEX 4 navigation file . . . 88
4.8 Bias policy diagnostics . . . 89
4.9 BeiDou B1IB3I STEC smoke example . . . 89
4.10 What to try next . . . 91

5 Ionosphere Calibration 93
5.1 Receiver calibration . . . 93
5.2 Why receiver DCB matters . . . 93
5.3 Calibration as model alignment . . . 94
5.4 PPP-UDUC as an advanced DCB source . . . 94
5.5 Current GNX-py calibration workflow . . . 95
5.6 Notebook setup . . . 95
5.7 Weighted-average receiver calibration . . . 97
5.8 Comparing calibrated, GIM and user-defined DCB . . . 99
5.9 PPP-UDUC comparison when available . . . 100
5.10 How to pass your own receiver DCB . . . 101
5.11 What to try next . . . 101

6 Ionospheric Activity Indexes 103
6.1 Ionospheric activity indices . . . 103
6.1.1 SIDX (temporal VTEC change) . . . 105
6.1.2 ROT (Rate Of TEC) . . . 105
6.1.3 ROTI (Rate Of TEC Index) . . . 106

6.1.4 GIX (VTEC spatial gradient) . . . 106
6.1.5 Processing example . . . 107
6.1.6 Saving results . . . 118

7 Ionosphere Modelling with Kriging **119**
7.1 Leave-one-station-out check . . . 131

8 References **133**

Chapter 1

Introduction

1.1 Introduction to GNX-py

This notebook is a guided introduction to GNX-py. We will not start from a high-level black box. Instead, we will first walk through the ingredients of a GNSS positioning pipeline: observations, navigation products, signal transmission time, satellite coordinates, atmospheric corrections, antenna corrections, and the observation equation.

Only after that manual walk-through will we use `SPPConfig` and `SPPSession`, which automate the same chain in the actual library workflow.

1.2 GNX-py library

GNX-py is a GNSS processing library. The package is not only a set of readers; it tries to keep the full scientific workflow visible:

- `gnx_py.spp` contains Single Point Positioning tools. SPP is a good first place to learn the data flow because it uses broadcast products and a relatively small state vector.
- `gnx_py.ppp` contains Precise Point Positioning workflows, including combined and uncombined processing. Advanced modes such as PPP-AR should be treated as experimental unless the corresponding tutorial or validation note says otherwise.
- `gnx_py.ionosphere` contains STEC/VTEC tools, Global Ionosphere Map handling, the Klobuchar, NTCM and BDGIM broadcast models, kriging, activity indexes, and explicit bias policy for OSB/DCB/DSB products.
- `gnx_py.orbits` contains the signal-in-space comparison of broadcast and precise orbits and clocks (SIS/SISRE) with `SISConfig` and `SISSession`.
- `gnx_py.downloaders` downloads observations and products for a project with `DownloadConfig` and `DownloadSession`.
- The readers `gnx_py.io.rinex_obs` (RINEX 2/3/4 observations), `gnx_py.io.rinex_nav` (navigation), `gnx_py.io.sp3_reader` (SP3 orbits), `gnx_py.io.rinex_clk` (RINEX clocks), and `gnx_py.models.antex` (antenna calibrations) are part of the library.
- `gnx_py.core.gnss` and `gnx_py.models.biases` contain shared signal metadata and bias helpers used by higher-level modules; `gnx_py.models.tides`, `gnx_py.models.ocean_loading` and `gnx_py.models.attitude` hold the site-displacement and satellite-attitude models used by PPP.
- The file readers are collected in `gnx_py.io`, the correction models in `gnx_py.models`, and the settings, time and coordinate helpers and the preprocessing shared by all sessions in `gnx_py.core`. The module paths of versions before 1.0.0 (`gnx_py.rinex_obs`, `gnx_py.tools`, …) still import the same modules.
- `gnx_py.viz` contains the plotting and table-preview helpers used for the figures and tables in these tutorials.

A useful mental model is: configuration objects describe an experiment, session objects run it, and result objects store tables that can be inspected, plotted, or saved. The same pattern repeats across the

library: `SPPConfig`/`SPPSession`, `PPPConfig`/`PPPSession`, `TECConfig`/`TECSession`, `SISConfig`/`SISSession` and `DownloadConfig`/`DownloadSession`, each started with `run()`.

1.2.1 Installation for this tutorial

From a fresh clone, install the package into the environment used by Python, Jupyter, or your IDE:

```
python -m pip install -e ".[tutorial]"
```

The `tutorial` extra installs notebook and plotting packages used by the tutorial tree. Core GNX-py runtime dependencies remain in the base package. After installation, `import gnx_py` should work from any directory as long as the same virtual environment is active or selected as the IDE interpreter.

```
from __future__ import annotations

import contextlib
import io
import platform
import sys
from dataclasses import fields
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd


def find_repo_root(start: Path | None = None) -> Path:
    """Find the GNX repository root without editing Python import paths."""
    current = (start or Path.cwd()).resolve()
    for candidate in (current, *current.parents):
        if (candidate / "pyproject.toml").exists() and (candidate / "sample_data").exists():
            return candidate
    raise FileNotFoundError("Could not find the repository root with sample_data.")


REPO_ROOT = find_repo_root()
SAMPLE_DATA = REPO_ROOT / "sample_data"

import gnx_py as gnx
from gnx_py import viz

viz.apply_publication_style(font_scale=1.3, mathtext="cm")

GNX_PATH = Path(gnx.__file__).resolve()



def short_path(path: Path) -> str:
    """Show a path relative to the repository (or to ~), not as a machine-specific absolute path."""
    path = Path(path).absolute()  # no resolve(): keep the venv interpreter path, not its symlink target
    for base, prefix in ((REPO_ROOT, "<repo>"), (Path.home().resolve(), "~")):
        if base == path or base in path.parents:
            return f"{prefix}/{path.relative_to(base).as_posix()}"
    return str(path)


print(f"Python executable: {short_path(Path(sys.executable))}")
print(f"Python version: {platform.python_version()}")
print(f"gnx_py imported from: {short_path(GNX_PATH)}")

```

```
if REPO_ROOT not in GNX_PATH.parents:
    print("Warning: gnx_py was not imported from this repository checkout.")
    print(f"Reinstall this checkout with: python -m pip install -e {REPO_ROOT}")
```

```
Python executable: ~/venvs/gnx/bin/python
Python version: 3.12.6
gnx_py imported from: <repo>/gnx_py/__init__.py
```

1.2.2 Sample data used here

The tutorial uses compact example input files from `sample_data`. They are kept in the repository as teaching and testing inputs, not as package data installed into `site-packages`.

We will use station `BRUX` on day-of-year 035 of 2024. The code cells process one full day of 30 s data (2880 epochs); this still runs in a few seconds, and the surrounding explanation follows the full processing logic.

```
OBS_PATH = SAMPLE_DATA / "BRUX00BEL_R_20240350000_01D_30S_MO.crx.gz"
NAV_PATH = SAMPLE_DATA / "BRDC00IGS_R_20240350000_01D_MN.rnx"
ATX_PATH = SAMPLE_DATA / "igs20.atx"
DCB_PATH = SAMPLE_DATA / "CAS1OPSRAP_20240350000_01D_01D_DCB.BIA"
GIM_PATH = SAMPLE_DATA / "COD0OPSFIN_20240350000_01D_01H_GIM.INX"
SINEX_PATH = SAMPLE_DATA / "IGS0OPSSNX_20240350000_01D_01D_CRD.SNX"

TUTORIAL_WINDOW = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 5, 0, 0)]
NAV_WINDOW = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 5, 0, 0)]

path_table = pd.DataFrame(
    [
        ("observation", OBS_PATH),
        ("broadcast navigation", NAV_PATH),
        ("antenna calibration", ATX_PATH),
        ("code biases", DCB_PATH),
        ("ionosphere grid", GIM_PATH),
        ("station coordinates", SINEX_PATH),
    ],
    columns=["role", "path"],
)
path_table["relative path"] = path_table["path"].map(lambda p: str(p.relative_to(REPO_ROOT)))
path_table["exists"] = path_table["path"].map(lambda p: p.exists())
viz.df_head(path_table[["role", "relative path", "exists"]], truncate_str=None)
```

role	relative path	exists
observation	sample_data/BRUX00BEL_R_202403500…	True
broadcast navigation	sample_data/BRDC00IGS_R_202403500…	True
antenna calibration	sample_data/igs20.atx	True
code biases	sample_data/CAS1OPSRAP_2024035000…	True
ionosphere grid	sample_data/COD0OPSFIN_2024035000…	True
station coordinates	sample_data/IGS0OPSSNX_2024035000…	True

1.2.3 Downloading data for your own project

The files in `sample_data` are enough for every chapter of this manual. For your own stations and days, `gnx_py.downloaders` follows the same configuration + session pattern as the processing modules: `DownloadConfig` describes the project (dates, stations, product types, archives), `DownloadSession(config).run()` downloads what is missing and returns a `DownloadReport`. Orbits, clocks and biases are taken from one analysis-centre solution, SP3 files are also fetched for the day before and after (orbit interpolation needs them), and every file is checked before it is kept. The archives are BKG, CDDIS and CODE by default; `"euref"` adds the EUREF/EPN observation archive at BKG. CDDIS needs Earthdata credentials: a token in the environment variable `EARTHDATA_TOKEN`, or the `DownloadConfig` arguments `token`, `username` and `password`, or `credentials_file`.

The code below is shown, not run, because building this manual makes no network requests:

```python
from datetime import date

from gnx_py.downloaders import DownloadConfig, DownloadSession

config = DownloadConfig(
    project_name="brux_2024_035",
    output_root="data",                                  # files go to data/brux_2024_035/...
    dates=date(2024, 2, 4),
    stations="BRUX00BEL",
    product_types=("obs", "nav", "sp3", "clk", "bia", "ionex"),
    sources=("cddis", "bkg", "code", "euref"),           # "euref": EPN observations at BKG
)

report = DownloadSession(config).run()
print(report.summary())
print(report.product_sets.solution)                      # e.g. "CODOMGXFIN" for SP3, CLK and
    OSB
for outcome in report.outcomes():                        # one line per requested file
    print(outcome.data_type, outcome.day, outcome.station, outcome.status, outcome.path)
```

`DownloadConfig(..., dry_run=True, check_availability=False)` only lists the files and paths that would be downloaded. GNX-py reads the compressed files (`.gz`, `.Z`, `.crx.gz`) directly, so the paths in `report.files("obs")`, `report.files("sp3")` and so on can be passed to `SPPConfig` or `PPPConfig` without unpacking.

1.2.4 Current GNSS signal model

A GNSS observable is not just “GPS” or “Galileo”. It is tied to a constellation, signal, code observable, phase observable, antenna frequency, and bias convention. GNX-py keeps this mapping in `gnx_py.core.gnss`.

For BeiDou, the current conservative default is `B1IB3I`. Modern BDS combinations such as `B1CB2a` and `B1CB2b` exist in the metadata, but they need RINEX 4 navigation files and clock-consistent OSB/DSB products, and should be treated as requiring validation unless a specific workflow documents them as ready for production use.

```python
from gnx_py.core.gnss import (
    DEFAULT_MODE_BY_SYSTEM,
    SUPPORTED_MODES_BY_SYSTEM,
    SYSTEM_NAMES,
    mode_ionosphere_free_coefficients,
    mode_layout,
    mode_signals,
)

signal_rows = []
for system in ["G", "E", "C"]:
    default_mode = DEFAULT_MODE_BY_SYSTEM[system]
    for mode in SUPPORTED_MODES_BY_SYSTEM[system]:
        layout = mode_layout(mode)
        signal_rows.append(
            {
                "system": system,
                "name": SYSTEM_NAMES[system],
                "mode": mode,
                "default": mode == default_mode,
                "signals": ", ".join(mode_signals(mode)),
                "RINEX code/phase": ", ".join(f"{item['code']}/{item['phase']}" for item in
    layout),
                "ANTEX": ", ".join(item["antex"] for item in layout),
            }
        )
```

```
26
27 signal_table = pd.DataFrame(signal_rows)
28 viz.df_head(
29     signal_table[signal_table["default"] | signal_table["mode"].isin(["B1CB2a", "B1CB2b"])],
30     ncols=7,
31     reset_index=False,
32 )
```

system	name	mode	default	signals	RINEX code phase	ANTEX
G	GPS	L1L2	True	L1, L2	C1/L1, C2/L2	G01, G02
E	Galileo	E1E5a	True	E1, E5a	C1/L1, C5/L5	E01, E05
C	BeiDou	B1IB3I	True	B1I, B3I	C2/L2, C6/L6	C02, C06
C	BeiDou	B1CB2a	False	B1C, B2a	C1/L1, C5/L5	C01, C05
C	BeiDou	B1CB2b	False	B1C, B2b	C1/L1, C7/L7	C01, C07

1.3 Example of processing in GNX-py

We now go through the pipeline in the same spirit as a hand calculation. The purpose is educational: we want to see what the later `SPPSession` automates.

The path is:

1. Load observation data and metadata.
2. Load broadcast navigation data.
3. Interpolate satellite coordinates and clocks.
4. Move from reception time to signal emission time.
5. Correct satellite coordinates for Earth rotation during signal travel.
6. Compute elevation/azimuth and apply a simple elevation mask.
7. Inspect ionosphere, troposphere, relativistic, PCO, and antenna-height corrections.
8. Build a simplified SPP observation equation for one epoch.
9. Let `SPPSession` run the production pipeline on the same data window.

1.3.1 Reading observations with `GNSSDataProcessor2`

`GNSSDataProcessor2` takes paths to input files and information about the systems and signals we want to use. Here we choose GPS and the `L1L2` mode.

GNX-py reads every input format with its own readers: RINEX 2, 3 and 4 observation files (`gnx_py.io.rinex_obs`), RINEX navigation files (`gnx_py.io.rinex_nav`), SP3 orbits (`gnx_py.io.sp3_reader`), RINEX clock files (`gnx_py.io.rinex_clk`), and ANTEX antenna calibrations (`gnx_py.models.antex`). Compressed files (gzip, Unix `.Z`, bzip2, zip) are read in memory. Hatanaka-compressed observations, such as our `.crx.gz` file, are decompressed in memory by the RNXCMP `crx2rnx` program bundled with the `hatanaka` package; nothing is written next to the data. The time window `tlim` is applied while the file is read.

Earlier versions converted observation files with the external gfzrnx program or read them with `georinex`; neither is used any more. The old `use_gfz` argument is still accepted for compatibility, but it has no effect, so it is left out below.

```
1  processor = gnx.GNSSDataProcessor2(
2      obs_path=OBS_PATH,
3      nav_path=NAV_PATH,
4      atx_path=ATX_PATH,
5      sys={"G"},
6      mode="L1L2",
7  )
8
9  obs_data = processor.load_obs_data(tlim=TUTORIAL_WINDOW)
10 print(type(obs_data).__name__)
11
12 obs_fields = []
13 for field in fields(obs_data):
14     value = getattr(obs_data, field.name)
```

```
    obs_fields.append(
        {
            "field": field.name,
            "type": type(value).__name__,
            "shape/size": getattr(value, "shape", len(value) if hasattr(value, "__len__") and not isinstance(value, str) else ""),
        }
    )
viz.df_head(pd.DataFrame(obs_fields))
```

```
ObsData
```

field	type	shape/size
gps	DataFrame	(32451, 6)
gal	NoneType	
bds	NoneType	
sat_pco	dict	457
rec_pco	defaultdict	23
meta	tuple	12
interval	float	
header	ObsHeader	

We obtained an `ObsData` structure. The most important fields for this introduction are:

- `gps`, `gal`, `bds`: observation tables for active constellations. In this first example only `gps` is populated.
- `sat_pco`: satellite phase-center offsets read from ANTEX.
- `rec_pco`: receiver antenna phase-center offsets read from ANTEX.
- `meta`: station metadata read from the RINEX header, including station name, antenna, approximate coordinates, geodetic coordinates, observation interval, and available observation types.
- `header`: the parsed RINEX header itself.

This is one of the central design ideas of GNX-py: raw files are converted into structured objects before scientific processing starts.

```
gps_obs = obs_data.gps.copy()

print(f"GPS observation table shape: {gps_obs.shape}")
viz.df_head(gps_obs, nrows=6, ncols=8)

meta_names = [
    "station", "antenna", "radome", "antenna_delta_hen_m", "approx_xyz_m",
    "approx_flh_deg_m", "interval_s", "gps_obs_types", "gal_obs_types",
    "first_epoch", "last_epoch", "phase_shift",
]
meta_table = pd.DataFrame(
    {
        "field": meta_names,
        "value": [repr(value) for value in obs_data.meta],
    }
)
viz.df_head(meta_table, nrows=8, truncate_str=50)
```

```
GPS observation table shape: (32451, 6)
```

sv	time	C1W	L1C	S1W	C2W	L2W	S2W
G01	2024-02-04 00:00:00	20565617.438	108073094.917	56.031	20565618.186	84212824.036	56.031
G01	2024-02-04 00:00:30	20576284.447	108129151.083	55.875	20576285.277	84256504.152	55.875
G01	2024-02-04 00:01:00	20587036.947	108185654.478	55.750	20587037.593	84300532.758	55.750
G01	2024-02-04 00:01:30	20597873.998	108242604.174	55.750	20597874.812	84344909.125	55.750
G01	2024-02-04 00:02:00	20608795.959	108299999.309	55.625	20608796.736	84389632.588	55.625
G01	2024-02-04 00:02:30	20619802.483	108357838.959	55.625	20619803.269	84434702.433	55.625

field	value
station	'BRUX'
antenna	'JAVRINGANT_DM'
radome	'SCIS'
antenna_delta_hen_m	array([0.4689, 0. , 0.001])
approx_xyz_m	array([4027881.8468, 306998.261…
approx_flh_deg_m	array([50.79805957, 4.35855934…
interval_s	30.0
gps_obs_types	['C1C', 'C1L', 'C1W', 'C2L', 'C2W…

1.3.2 Antenna metadata: satellite and receiver PCO

Another important part of `ObsData` is antenna metadata. Phase-center offsets (PCO) describe the vector from the antenna reference point to the effective phase center for a given signal/frequency.

GNX-py stores satellite PCO in `sat_pco`, usually keyed by satellite and ANTEX frequency code, and receiver PCO in `rec_pco`, keyed by antenna, radome, and signal frequency. These values later enter the observation model through line-of-sight projections.

```
sat_pco_sample = pd.DataFrame(
    [
        {"key": str(key), "pco_mm": value}
        for key, value in list(obs_data.sat_pco.items())[:6]
    ]
)
rec_pco_sample = pd.DataFrame(
    [
        {"key": str(key), "pco_mm": value}
        for key, value in list(obs_data.rec_pco.items())[:6]
    ]
)

print("Satellite PCO sample")
viz.df_head(sat_pco_sample, truncate_str=None)
print("Receiver PCO sample")
viz.df_head(rec_pco_sample, truncate_str=None)
```

```
Satellite PCO sample
```

key	pco_mm
('E01', 'E05')	[122.14, -9.38, 792.04]
('E01', 'E07')	[122.84, -8.98, 831.06]
('E01', 'E06')	[122.04, -9.21, 845.56]
('E01', 'E01')	[121.33, -8.01, 879.87]
('E01', 'E08')	[122.48, -9.17, 811.55]
('E02', 'E05')	[122.69, -9.25, 772.88]

```
Receiver PCO sample
```

key	pco_mm
('JAVRINGANT_DM', 'SCIS', 'G01')	[0.29, 1.5, 85.93]
('JAVRINGANT_DM', 'SCIS', 'E01')	[0.29, 1.5, 85.93]
('JAVRINGANT_DM', 'SCIS', 'J01')	[0.29, 1.5, 85.93]
('JAVRINGANT_DM', 'SCIS', 'S01')	[0.29, 1.5, 85.93]
('JAVRINGANT_DM', 'SCIS', 'C01')	[0.29, 1.5, 85.93]
('JAVRINGANT_DM', 'SCIS', 'G02')	[0.16, 0.15, 116.64]

1.3.3 Reading broadcast navigation products

Now we load broadcast navigation data with `load_broadcast_orbit()`. The returned `OrbitData` object may contain GPS, Galileo, and BeiDou navigation tables as well as broadcast ionosphere coefficients.

For GPS, the navigation rows contain Keplerian orbital elements and satellite clock parameters. They are not yet satellite coordinates at observation epochs. We still need to select the proper navigation message and interpolate/propagate the satellite state.

```
nav_data = processor.load_broadcast_orbit(tlim=NAV_WINDOW)
print(type(nav_data).__name__)

orbit_fields = []
for field in fields(nav_data):
    value = getattr(nav_data, field.name)
    orbit_fields.append(
        {
            "field": field.name,
            "type": type(value).__name__,
            "shape/size": getattr(value, "shape", len(value) if hasattr(value, "__len__") and not isinstance(value, str) else ""),
        }
    )
viz.df_head(pd.DataFrame(orbit_fields))

gps_orb = nav_data.gps_orb.copy()
print(f"GPS broadcast navigation table shape: {gps_orb.shape}")
viz.df_head(gps_orb, nrows=6, ncols=8, floatfmt=".6g")
```

```
OrbitData
```

field	type	shape/size
gps_orb	DataFrame	(451, 31)
gal_orb	NoneType	
bds_orb	NoneType	
gpsa	list	4
gpsb	list	4
gala	list	3
bdsa	NoneType	
bdsb	NoneType	
iono	DataFrame	(44, 10)
nav_info	NavInfo	

```
GPS broadcast navigation table shape: (451, 31)
```

sv	time	SVclockBias	SVclockDrift	SVclockDriftRate	IODE	Crs	DeltaN
G08	2024-02-03 23:59:44	-0.000165336	-1.81899e-12	0	15	73.2188	4.70448e-09
G29	2024-02-03 23:59:44	-0.000607626	6.82121e-13	0	23	52.25	4.39875e-09
G31	2024-02-03 23:59:44	-0.00022764	-1.13687e-13	0	0	4.875	4.75698e-09
G01	2024-02-04 00:00:00	0.000168205	1.36424e-12	0	84	-40.5625	3.99945e-09
G02	2024-02-04 00:00:00	-0.000490906	5.79803e-12	0	58	-32.8438	4.34447e-09
G03	2024-02-04 00:00:00	0.000170307	2.48974e-11	0	65	-65.5312	3.98195e-09

1.3.4 Broadcast coordinates interpolation algorithm

The GPS broadcast message contains parameters of the satellite orbit. To obtain satellite ECEF coordinates at epoch `t`, the algorithm follows the classical broadcast ephemeris sequence:

$$
\begin{aligned}
t_k &= t - t_{oe} \quad \text{(apply GPS week crossover)} \\
M_k &= M_0 + \left(\sqrt{\mu/a^3} + \Delta n\right) t_k \\
\text{solve } M_k &= E_k - e \sin E_k \\
v_k &= 2 \tan^{-1}\left(\sqrt{\frac{1+e}{1-e}} \tan \frac{E_k}{2}\right) \\
u_k &= \omega + v_k + c_{uc} \cos 2(\omega+v_k) + c_{us} \sin 2(\omega+v_k) \\
r_k &= a(1 - e \cos E_k) + c_{rc} \cos 2(\omega+v_k) + c_{rs} \sin 2(\omega+v_k) \\
i_k &= i_0 + \dot{i}\, t_k + c_{ic} \cos 2(\omega+v_k) + c_{is} \sin 2(\omega+v_k) \\
\lambda_k &= \Omega_0 + (\dot{\Omega} - \omega_E) t_k - \omega_E t_{oe} \\
\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} &= \mathbf{R}_3(-\lambda_k)\, \mathbf{R}_1(-i_k)\, \mathbf{R}_3(-u_k) \begin{bmatrix} r_k \\ 0 \\ 0 \end{bmatrix}
\end{aligned}
$$

The satellite clock is modeled with broadcast clock coefficients and the relativistic satellite-clock term:

$$
\begin{aligned}
dt^{sat} &= a_0 + a_1(t - t_0) + a_2(t - t_0)^2 + \Delta t_{rel} \\
\Delta t_{rel} &= -2 \cdot \frac{r_{sat} \cdot v_{sat}}{c^2}
\end{aligned}
$$

Where:

$$
\begin{aligned}
t_0 &: \text{satellite clock reference epoch} \\
t_{oe} &: \text{ephemeris reference epoch} \\
t_k = t - t_{oe} &\text{ with week crossover handling} \\
M_0 &: \text{mean anomaly at } t_{oe} \\
\mu &: \text{Earth gravitational constant} \\
E_k &: \text{eccentric anomaly solving } M_k = E_k - e \sin E_k \\
v_k &: \text{true anomaly} \\
c_{uc}, c_{us} &: \text{corrections to argument of latitude} \\
c_{rc}, c_{rs} &: \text{corrections to orbital radius} \\
c_{ic}, c_{is} &: \text{corrections to inclination} \\
\Omega_0 &: \text{longitude of ascending node at } t_{oe} \\
\omega_E &: \text{Earth rotation rate} \\
r_{sat} &: \text{satellite position vector} \\
v_{sat} &: \text{satellite velocity vector}
\end{aligned}
$$

GNX-py wraps this broadcast propagation in `BroadcastInterp`. The class selects a valid navigation message for each satellite and observation epoch (the record whose t_{oe} is nearest to the epoch, within 2 h for GPS and BeiDou and 3 h for Galileo; for Galileo the records already transmitted at the epoch come first; a GPS epoch with no record within 2 h takes the nearest record, while Galileo and BeiDou epochs without a valid record are left out), propagates the state, and attaches satellite coordinates and clocks to the observation table. A satellite that the navigation message marks as unhealthy is left out, and one warning names it: in this file that is G01, so the interpolation below prints a `RuntimeWarning`.

1.3.5 Signal emission time

A receiver timestamp is the signal reception time. Satellite coordinates, however, should refer to the signal emission time. A first approximation uses the measured pseudorange:

$$R = c(t_{\mathrm{rcv}}[\mathrm{reception}] - t^{sat}[\mathrm{emission}])$$
$$t^{sat}[\mathrm{emission}] = t_{\mathrm{rcv}}[\mathrm{reception}] - R/c$$
$$T[\mathrm{emission}] = t_{\mathrm{rcv}}[\mathrm{reception}] - R/c - dt^{sat}$$

Here `R` is the measured pseudorange, `dt^{sat}` is the satellite clock offset, and `c` is the speed of light. In practice, GNX-py first propagates the satellite to reception time, estimates emission time, and then propagates again at emission time.

```
## Build an ionosphere-free GPS code combination for the educational manual path.
manual_obs = gps_obs.copy()
gnx.make_ionofree(manual_obs, mode="L1L2", sys="G")

interp = gnx.BroadcastInterp(
    obs=manual_obs,
    nav=gps_orb,
    sys="G",
    mode="L1L2",
    emission_time=True,
)

with contextlib.redirect_stdout(io.StringIO()) as interp_log:
    obs_with_sat = interp.interpolate()

for line in interp_log.getvalue().splitlines()[:3]:
    print(line)

signal_travel_time = obs_with_sat["P3"] / 299_792_458.0 + obs_with_sat["clk"]
obs_with_sat["emission_epoch_est"] = (
    obs_with_sat.index.get_level_values("time") - pd.to_timedelta(signal_travel_time, unit="s")
)

print(f"Observation rows with selected broadcast coordinates: {obs_with_sat.shape[0]}")
viz.df_head(
    obs_with_sat[["P3", "x", "y", "z", "clk", "emission_epoch_est"]],
    nrows=6,
    ncols=8,
    floatfmt=".6f",
)
```

```
<repo>/gnx_py/core/coordinates.py:2666: RuntimeWarning: GPS satellites without a usable
    broadcast ephemeris (left out): G01 (unhealthy, health=63).
  result = self.get_coordinates()


Observation rows with selected broadcast coordinates: 31593
```

sv	time	P3	x	y	z	clk	emission epoch_est
G02	2024-02-04 00:00:00	22182503.154408	21576411.893139	12769385.577729	9640932.047138	-0.000491	2024-02-03 23:59:59.926498013
G02	2024-02-04 00:00:30	22196048.874523	21595794.677088	12798190.428792	9555187.896853	-0.000491	2024-02-04 00:00:29.926452829
G02	2024-02-04 00:01:00	22209652.379536	21615006.481446	12826735.170148	9469266.827763	-0.000491	2024-02-04 00:00:59.926407453
G02	2024-02-04 00:01:30	22223315.717831	21634045.845389	12855020.127025	9383170.397830	-0.000491	2024-02-04 00:01:29.926361876
G02	2024-02-04 00:02:00	22237036.435233	21652911.308939	12883045.630446	9296900.180248	-0.000491	2024-02-04 00:01:59.926316109
G02	2024-02-04 00:02:30	22250818.240630	21671601.417433	12910812.023759	9210457.743606	-0.000491	2024-02-04 00:02:29.926270137

The `obs_with_sat` frame now contains observation columns (`C1W`, `C2W`, `P3`, …), selected broadcast message parameters, satellite coordinates (`x`, `y`, `z`), satellite clock `clk`, and the estimated emission epoch; the preview above shows only a subset of these columns.

The coordinates are still ECEF coordinates tied to the Earth orientation at the relevant epoch. Because Earth rotates while the signal travels, the next step is the Sagnac/Earth-rotation correction.

1.3.6 Earth rotation / Sagnac correction

During signal travel the Earth rotates. If this is ignored, the line-of-sight geometry is slightly inconsistent: the satellite coordinate is tied to Earth orientation at emission time, while the receiver is evaluated at reception time.

GNX-py applies the correction as a rotation around the Earth rotation axis:

$$\tilde{\mathbf{r}}^{\,sat} \equiv \text{satellite ECEF at emission time}$$

$$\Delta t = \frac{\|\tilde{\mathbf{r}}^{\,sat} - \mathbf{r}_{rcv}\|}{c}$$

$$\mathbf{r}^{sat} = \mathbf{R}_3(\omega_E\,\Delta t)\,\tilde{\mathbf{r}}^{\,sat}, \quad \mathbf{R}_3(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The same wrapper also computes azimuth and elevation from receiver coordinates. Because $\mathbf{R}_3$ rotates about the Z axis, the corrected coordinate `ze` equals `z`, so the preview below shows only the `x`/`y` components before and after the correction.

```
obs_geom = gnx.CustomWrapper(
    obs=obs_with_sat,
    flh=obs_data.meta[5],
    xyz_a=obs_data.meta[4],
    mode="L1L2",
    epochs=None,
).run()

sagnac_delta = (
    obs_geom[["xe", "ye", "ze"]].to_numpy()
    - obs_geom[["xe_ur", "ye_ur", "ze_ur"]].to_numpy()
)
sagnac_norm = np.linalg.norm(sagnac_delta, axis=1)

print(f"Median coordinate rotation magnitude: {np.median(sagnac_norm):.3f} m")
viz.df_head(obs_geom[["x", "y", "xe", "ye", "ev", "az"]], nrows=6, ncols=8)
```

```
Median coordinate rotation magnitude: 111.491 m
```

sv	time	x	y	xe	ye	ev	az
G02	2024-02-04 00:00:00	21576411.893	12769385.578	21576480.335	12769269.931	44.036	135.554
G02	2024-02-04 00:00:30	21595794.677	12798190.429	21595863.315	12798074.608	43.812	135.663
G02	2024-02-04 00:01:00	21615006.481	12826735.170	21615075.315	12826619.174	43.587	135.771
G02	2024-02-04 00:01:30	21634045.845	12855020.127	21634114.874	12854903.957	43.362	135.879
G02	2024-02-04 00:02:00	21652911.309	12883045.630	21652980.531	12882929.287	43.138	135.986
G02	2024-02-04 00:02:30	21671601.417	12910812.024	21671670.832	12910695.507	42.913	136.093

1.3.7 Elevation, azimuth, and a simple observation mask

Elevation and azimuth give the local viewing geometry. Low-elevation observations usually have larger atmospheric errors, multipath, and modeling uncertainty. A simple SPP workflow commonly applies an elevation mask before solving the receiver position.

```
manual_pipeline = obs_geom[obs_geom["ev"] > 10].copy()

print(f"Rows before elevation mask: {len(obs_geom)}")
print(f"Rows after 10 degree mask: {len(manual_pipeline)}")

elevation_summary = manual_pipeline.reset_index().groupby("sv")["ev"].agg(["min", "median", "
    max"])
viz.df_head(elevation_summary, nrows=10)
```

```
Rows before elevation mask: 31593
Rows after 10 degree mask: 25376
```

sv	min	median	max
G02	10.100	48.393	86.924
G03	10.055	48.318	85.262
G04	10.004	37.871	76.203
G05	10.015	24.413	62.036
G06	10.008	24.112	63.661
G07	10.048	32.507	69.194
G08	10.035	46.314	80.350
G09	10.098	47.079	88.440
G10	10.033	24.042	61.563
G11	10.027	28.408	51.730

1.3.8 Klobuchar ionosphere model

For single-frequency code observations the ionosphere is one of the dominant error sources. The GPS broadcast message carries Klobuchar coefficients. The model estimates a slant group delay using the receiver position, satellite azimuth/elevation, time of week, and broadcast coefficients.

Inputs in semicircles unless noted; `E` is elevation and `A` is azimuth:

$$\psi = \frac{0.0137}{E + 0.11} - 0.022 \qquad \text{Earth-centered angle}$$

$$\phi_I = \phi_u + \psi \cos A, \quad \phi_I \in [-0.416,\, 0.416] \qquad \text{IPP latitude}$$

$$\lambda_I = \lambda_u + \frac{\psi \sin A}{\cos \phi_I} \qquad \text{IPP longitude}$$

$$\phi_m = \phi_I + 0.064 \cos(\lambda_I - 1.617) \qquad \text{geomagnetic latitude at IPP}$$

$$t' = (43200\, \lambda_I + t_{\mathrm{GPS}}) \bmod 86400 \qquad \text{local time}$$

$$A_I = \sum_{n=0}^{3} \alpha_n \phi_m^n, \quad A_I \geq 0 \qquad \text{amplitude}$$

$$P_I = \sum_{n=0}^{3} \beta_n \phi_m^n, \quad P_I \geq 72000 \qquad \text{period}$$

$$X_I = \frac{2\pi\,(t' - 50400)}{P_I} \qquad \text{phase}$$

$$F = 1 + 16(0.53 - E)^3 \qquad \text{slant factor}$$

$$I_{L1} = \begin{cases} [5{\cdot}10^{-9} + A_I(1 - \frac{X_I^2}{2} + \frac{X_I^4}{24})]F, & |X_I| \leq 1.57, \\ 5{\cdot}10^{-9} F, & |X_I| > 1.57, \end{cases}$$

$$\Delta R = c\, I_{L1}, \qquad I_f = \left(\frac{f_{L1}}{f}\right)^2 I_{L1}$$

In this notebook the final manual solution uses the ionosphere-free `L1L2` combination, so the Klobuchar values are shown mainly to explain what the model contributes in single-frequency SPP. The same holds for the `SPPSession` run below: the ionosphere-free modes (`L1L2`, `E1E5a`, `B1IB3I`, …) do not use the ionospheric correction, so `ionosphere_model` changes the result only in single-frequency modes such as `L1`, `E1` or `B1I`.

Klobuchar is one of four ionosphere models selected with `ionosphere_model` in `SPPConfig` and `PPPConfig` : `"klobuchar"` (GPS broadcast coefficients), `"ntcm"` (the Galileo NTCM-G broadcast model), `"bdgim"` (BDGIM, the BeiDou-3 global broadcast model of BDS-SIS-ICD-B2b-1.0, read from the `> ION` records of a RINEX 4 navigation file such as `BRD400DLR_S_20240350000_01D_MN.rnx` in `sample_data`), and `"gim"` (a global ionosphere map in IONEX format). When a RINEX 4 file carries several broadcast coefficient sets during the day, Klobuchar and NTCM use the set in force at each epoch. A GIM gives the delay in the datum of the satellite DCBs printed in its own IONEX header; when the code biases come from a bias product in `dcb_path`, single-frequency SPP and PPP refer the GIM correction to that product's

code-bias datum (GPS L1/L2 and Galileo E1/E5a, the signals of the IONEX DCBs), so the ionospheric correction and the code biases come from one datum.

```
from gnx_py.ionosphere.models import klobuchar

manual_reset = manual_pipeline.reset_index()
receiver_lat, receiver_lon, receiver_h = obs_data.meta[5]

manual_pipeline["ion_l1_m"] = [
    klobuchar(
        azimuth=row.az,
        elev=row.ev,
        fi=receiver_lat,
        lambda_=receiver_lon,
        tow=row.time.hour * 3600 + row.time.minute * 60 + row.time.second,
        beta=nav_data.gpsb,
        alfa=nav_data.gpsa,
    )
    for row in manual_reset.itertuples(index=False)
]

viz.df_head(manual_pipeline[["ev", "az", "ion_l1_m"]], nrows=6)
```

sv	time	ev	az	ion_l1_m
G02	2024-02-04 00:00:00	44.036	135.554	2.056
G02	2024-02-04 00:00:30	43.812	135.663	2.064
G02	2024-02-04 00:01:00	43.587	135.771	2.071
G02	2024-02-04 00:01:30	43.362	135.879	2.078
G02	2024-02-04 00:02:00	43.138	135.986	2.086
G02	2024-02-04 00:02:30	42.913	136.093	2.094

1.3.9 Troposphere models

Tropospheric delay is non-dispersive at GNSS frequencies, so an ionosphere-free code combination does not remove it. GNX-py offers three troposphere models in its configuration objects (`troposphere_model`): `"saastamoinen"`, `"collins"` (the UNB3 model) and `"niell"`. SPP accepts all three; PPP requires `"niell"`, whose wet mapping function also maps the estimated zenith delay. In this introduction we use Saastamoinen.

For Saastamoinen, the tutorial uses the standard-atmosphere version implemented in the library:

$$
\begin{aligned}
\epsilon &= |\mathrm{ev}| \cdot \frac{\pi}{180} \\
P(h) &= Pr\,(1 - 2.26 \times 10^{-5} h)^{5.225} \\
T(h) &= Tr - 0.0065\,h \\
H(h) &= Hr\,\exp(-6.396 \times 10^{-4} h) \\
e(h) &= 0.01\,H(h)\,\exp(-37.2465 + 0.213166\,T(h) - 2.56908{\times}10^{-4} T(h)^2)
\end{aligned}
$$

$$
d_{\mathrm{trop}}(\epsilon, h) = \frac{0.002277}{\sin\epsilon}\left[P(h) - \frac{B(h)}{\tan^2\epsilon}\right] + \frac{0.002277}{\sin\epsilon}\left(\frac{1255}{T(h)} + 0.05\right) e(h)
$$

where $B(h)$ is interpolated in height from the standard table of the model (1.156 at sea level).

```
from gnx_py.models.troposphere import saastamoinen

manual_pipeline["tro"] = saastamoinen(manual_pipeline["ev"].to_numpy(), receiver_h)

viz.df_head(manual_pipeline[["ev", "tro"]], nrows=6)
print(f"Troposphere delay range in this window: {manual_pipeline['tro'].min():.3f} to {
    manual_pipeline['tro'].max():.3f} m")
```

sv	time	ev	tro
G02	2024-02-04 00:00:00	44.036	3.380
G02	2024-02-04 00:00:30	43.812	3.394
G02	2024-02-04 00:01:00	43.587	3.408
G02	2024-02-04 00:01:30	43.362	3.422
G02	2024-02-04 00:02:00	43.138	3.436
G02	2024-02-04 00:02:30	42.913	3.451

```
Troposphere delay range in this window: 2.353 to 13.066 m
```

1.3.10 Relativistic path range correction

There are two relativistic effects in this part of the pipeline. The broadcast satellite clock already includes a relativistic satellite-clock term. A separate path-range correction can also be applied to the geometric range:

$$\rho_{\text{geom}} = \|\mathbf{r}^{sat} - \mathbf{r}_{rcv}\|,$$
$$\Delta\rho_{\text{rel}} = \frac{2\,\mu}{c^2}\,\ln\left(\frac{r^{sat} + r_{rcv} + \rho_{\text{geom}}}{r^{sat} + r_{rcv} - \rho_{\text{geom}}}\right),$$
$$\rho_{\text{corr}} = \rho_{\text{geom}} + \Delta\rho_{\text{rel}}.$$

Here `r^{sat}` and `r_{rcv}` are vector norms, `c` is the speed of light, and `mu` is Earth's gravitational constant. The magnitude is small in SPP, but it is useful to see it because PPP workflows keep such terms explicit.

```
MU_EARTH = 3.986004418e14
CLIGHT = 299_792_458.0

receiver_xyz = obs_data.meta[4].astype(float)
sat_xyz = manual_pipeline[["xe", "ye", "ze"]].to_numpy(dtype=float)
geom_range = np.linalg.norm(sat_xyz - receiver_xyz, axis=1)
sat_norm = np.linalg.norm(sat_xyz, axis=1)
receiver_norm = np.linalg.norm(receiver_xyz)

manual_pipeline["dprel"] = (2 * MU_EARTH / CLIGHT**2) * np.log(
    (sat_norm + receiver_norm + geom_range) / (sat_norm + receiver_norm - geom_range)
)

viz.df_head(manual_pipeline[["ev", "dprel"]], nrows=6, floatfmt=".4f")
print(f"Relativistic path correction range: {manual_pipeline['dprel'].min():.4f} to {
    manual_pipeline['dprel'].max():.4f} m")
```

sv	time	ev	dprel
G02	2024-02-04 00:00:00	44.0361	0.0142
G02	2024-02-04 00:00:30	43.8115	0.0142
G02	2024-02-04 00:01:00	43.5869	0.0142
G02	2024-02-04 00:01:30	43.3624	0.0142
G02	2024-02-04 00:02:00	43.1379	0.0142
G02	2024-02-04 00:02:30	42.9135	0.0142

```
Relativistic path correction range: 0.0126 to 0.0174 m
```

1.3.11 Receiver PCO and antenna height as line-of-sight corrections

The receiver PCO in ANTEX is given in a local topocentric frame. To use it in the observation model, GNX-py converts the local vector to ECEF and projects it onto the line of sight.

For the unit line-of-sight vector `e` pointing from the satellite to the receiver, a simplified receiver PCO projection is:

$$\Delta^{rcv}_{PCO,f} = \mathbf{e}^T \mathbf{p}^{ECEF}_f$$

The phase centre lies at the antenna reference point plus the PCO, so the measured range is $\rho(\text{ARP}) + \Delta^{rcv}_{PCO,f}$ and the correction is subtracted from the observation (`gnx_py.core.tools` computes the same `pco_los` columns, and SPP, PPP and TEC all subtract them).

For an ionosphere-free combination, the frequency-dependent receiver PCO is combined with the same coefficients as the code observations:

$$\Delta^{rcv}_{PCO,IF} = k_1 \Delta^{rcv}_{PCO,1} - k_2 \Delta^{rcv}_{PCO,2}$$

The antenna height from the RINEX header can be treated similarly: it is a local displacement of the antenna reference point and can be projected onto the line of sight. The PPP filters and SPP apply it (`antenna_h=True`, the default; the `ah_los` column): the measured range is $\rho(\text{marker}) + \Delta_{ah}$ with $\Delta_{ah} = \mathbf{e}^T \Delta\mathbf{r}^{ECEF}_{ARP}$, the header's ANTENNA: DELTA H/E/N rotated to ECEF, and Δ_{ah} is subtracted from the observation, so the coordinates refer to the marker, the point of the SINEX reference coordinates. With `antenna_h=False` it is not subtracted and the coordinates refer to the antenna reference point. The manual solution below subtracts it too.

Satellite PCO follows the same geometric idea, but the offset is given in the satellite body frame, so the processing needs the satellite attitude. GNX-py takes it from the published yaw-steering law of each satellite block (`gnx_py.models.attitude`) or from an ORBEX attitude file of the orbit's analysis centre (`attitude_path`); the same attitude frame serves the satellite PCV and the phase wind-up in PPP. This matters for precise orbits, which refer to the satellite centre of mass. Broadcast orbits and clocks, as in this chapter, already refer to the antenna phase centre of the signal the message's clock refers to, and the broadcast group delays map it to the user's signal, so GNX-py applies no satellite PCO with broadcast orbits (the `sat_pco` setting of the session below then changes nothing).

```python
def receiver_pco_m(signal_code: str) -> np.ndarray:
    antenna_name, radome = obs_data.meta[1], obs_data.meta[2]
    for key, value in obs_data.rec_pco.items():
        if key == (antenna_name, radome, signal_code):
            return np.array(value, dtype=float) / 1000.0
    return np.zeros(3)

los_to_receiver = -(manual_pipeline[["xe", "ye", "ze"]].to_numpy(dtype=float) - receiver_xyz)
los_unit = los_to_receiver / np.linalg.norm(los_to_receiver, axis=1)[:, None]

pco_l1_ecef = gnx.enu_to_ecef(dENU=receiver_pco_m("G01"), flh=obs_data.meta[5])
pco_l2_ecef = gnx.enu_to_ecef(dENU=receiver_pco_m("G02"), flh=obs_data.meta[5])

manual_pipeline["pco_los_l1"] = los_unit @ pco_l1_ecef
manual_pipeline["pco_los_l2"] = los_unit @ pco_l2_ecef
k1, k2 = mode_ionosphere_free_coefficients("L1L2")
manual_pipeline["pco_los_l3"] = k1 * manual_pipeline["pco_los_l1"] - k2 * manual_pipeline["
    pco_los_l2"]

## RINEX stores antenna delta as H/E/N. Convert to an E/N/U displacement before ECEF projection
    .
ant_h, ant_e, ant_n = obs_data.meta[3]
ant_height_ecef = gnx.enu_to_ecef(dENU=np.array([ant_e, ant_n, ant_h]), flh=obs_data.meta[5])
manual_pipeline["antenna_h_los"] = los_unit @ ant_height_ecef

viz.df_head(
    manual_pipeline[["pco_los_l1", "pco_los_l2", "pco_los_l3", "antenna_h_los"]],
    nrows=6,
    floatfmt=".4f",
)
```

sv	time	pco_los_l1	pco_los_l2	pco_los_l3	antenna_h_los
G02	2024-02-04 00:00:00	-0.0591	-0.0811	-0.0251	-0.3254
G02	2024-02-04 00:00:30	-0.0589	-0.0808	-0.0250	-0.3241
G02	2024-02-04 00:01:00	-0.0586	-0.0804	-0.0249	-0.3228
G02	2024-02-04 00:01:30	-0.0584	-0.0801	-0.0248	-0.3214
G02	2024-02-04 00:02:00	-0.0581	-0.0798	-0.0247	-0.3201
G02	2024-02-04 00:02:30	-0.0579	-0.0794	-0.0245	-0.3187

1.3.12 Manual SPP observation model

At this point we have enough ingredients to write a simplified SPP code observation equation. For an ionosphere-free GPS code combination in this short educational example:

$$P_{IF}^{corr} = P_{IF} + c\,dt^{sat} - T - \Delta\rho_{rel} - \Delta_{PCO,IF}^{rcv} - \Delta_{ah}$$
$$P_{IF}^{corr} \approx \rho(\mathbf{r}_{marker}, \mathbf{r}^{sat}) + b_{rcv}$$

The unknowns are the receiver position correction and receiver clock bias. In matrix form for one epoch:

$$\mathbf{v} = \mathbf{A}\,\Delta\mathbf{x}$$

where each row of `A` contains the line-of-sight partial derivatives and a clock column. The next cell solves one epoch from the prepared observation table. This is not meant to replace `SPPSession`; it is a transparent miniature of what the automated pipeline later does for every epoch.

```
first_epoch = manual_pipeline.index.get_level_values("time").min()
epoch_obs = manual_pipeline.xs(first_epoch, level="time").copy()

x_est = receiver_xyz.copy()
clock_est_m = 0.0

P_corr = (
    epoch_obs["P3"].to_numpy(dtype=float)
    + CLIGHT * epoch_obs["clk"].to_numpy(dtype=float)
    - epoch_obs["tro"].to_numpy(dtype=float)
    - epoch_obs["dprel"].to_numpy(dtype=float)
    - epoch_obs["pco_los_l3"].to_numpy(dtype=float)  # range model rho(ARP) + pco_los, as in gnx_py.spp
    - epoch_obs["antenna_h_los"].to_numpy(dtype=float)  # rho(ARP) = rho(marker) + antenna_h_los (antenna_h=True)
)
sat_epoch_xyz = epoch_obs[["xe", "ye", "ze"]].to_numpy(dtype=float)

iteration_rows = []
for iteration in range(6):
    diff = sat_epoch_xyz - x_est
    rho = np.linalg.norm(diff, axis=1)
    design = np.column_stack((-diff / rho[:, None], np.ones(len(rho))))
    residual = P_corr - (rho + clock_est_m)
    step = np.linalg.lstsq(design, residual, rcond=None)[0]
    x_est = x_est + step[:3]
    clock_est_m = clock_est_m + step[3]
    iteration_rows.append(
        {
            "iteration": iteration + 1,
            "dX_m": step[0],
            "dY_m": step[1],
            "dZ_m": step[2],
            "dClock_m": step[3],
            "position_step_norm_m": np.linalg.norm(step[:3]),
        }
    )
```

```
36
37 manual_enu = gnx.ecef_to_enu(dXYZ=x_est - receiver_xyz, flh=obs_data.meta[5])
38 manual_solution = pd.Series(
39     {
40         "epoch": first_epoch,
41         "satellites": len(epoch_obs),
42         "east_offset_from_header_m": manual_enu[0],
43         "north_offset_from_header_m": manual_enu[1],
44         "up_offset_from_header_m": manual_enu[2],
45         "receiver_clock_m": clock_est_m,
46     }
47 )
48
49 viz.df_head(pd.DataFrame(iteration_rows), floatfmt=".3e")
50 ## The Series mixes a timestamp, a count and floats, so the floats are rounded here.
51 viz.df_head(
52     manual_solution.map(lambda v: round(v, 3) if isinstance(v, float) else v)
53     .rename_axis("quantity")
54     .to_frame("value"),
55     truncate_str=None,
56 )
```

iteration	dX_m	dY_m	dZ_m	dClock_m	position step_norm_m
1	-3.331e-01	4.828e-01	1.122e+00	6.090e+01	1.266e+00
2	2.084e-08	-7.740e-09	1.054e-08	-1.475e-08	2.460e-08
3	-9.213e-10	-6.067e-10	2.059e-09	1.381e-09	2.336e-09
4	-9.093e-10	1.891e-09	-2.941e-09	-5.359e-09	3.613e-09
5	2.075e-09	-8.686e-10	2.891e-10	3.912e-09	2.268e-09
6	-6.072e-10	2.729e-11	4.707e-09	2.852e-09	4.746e-09

quantity	value
epoch	2024-02-04 00:00:00
satellites	10
east_offset_from_header_m	0.507
north_offset_from_header_m	0.938
up_offset_from_header_m	0.683
receiver_clock_m	60.905

The offsets `east/north/up_offset_from_header_m` are the estimate minus the approximate position of the RINEX header (`receiver_xyz`, the a-priori position of the loop), rotated to east/north/up. The estimate is the marker (the antenna height is applied, as in the session below), and the header position is only an approximation, so these offsets are not errors; the session below gives errors against the SINEX coordinates, with the opposite sign (reference minus estimate).

The manual solution shows the idea, not a production-quality positioning engine. It uses one epoch, a compact correction set, and a very simple least-squares loop. The important point is the structure: after satellite coordinates and corrections are prepared, SPP reduces to geometry plus receiver clock estimation.

Now we can switch to the GNX-py session API. The session performs the same chain repeatedly, handles configuration choices, stores residuals, and returns a structured result object.

1.4 SPP Pipeline in GNX-py

The manual preparation above is useful for learning, but it is not how we want users to run routine processing. In GNX-py, the pipeline is represented by:

- `SPPConfig`: input files, systems, frequency modes, correction models, masks, and solver options,
- `SPPSession`: the object that reads the data, applies preprocessing and corrections, runs the solver, and returns `SPPResult`.

This separation is deliberate. Configuration makes scientific assumptions visible; the session keeps the workflow reproducible.

```
spp_config = gnx.SPPConfig(
    obs_path=OBS_PATH,
    nav_path=NAV_PATH,
    atx_path=ATX_PATH,
    dcb_path=DCB_PATH,
    gim_path=GIM_PATH,
    sinex_path=SINEX_PATH,
    sys={"G"},
    gps_freq="L1L2",
    gal_freq="E1E5a",
    bds_freq="B1IB3I",
    orbit_type="broadcast",
    ionosphere_model="klobuchar",
    troposphere_model="saastamoinen",
    time_limit=TUTORIAL_WINDOW,
    day_of_year=35,
    station_name="BRUX",
    ev_mask=10,
    sat_pco="los",
    rec_pco=True,
    rel_path=True,
    antenna_h=True,  # the default: the antenna height of the RINEX header is applied
    screen=False,
)

config_summary = pd.Series(
    {
        "systems": ", ".join(sorted(spp_config.sys)),
        "GPS frequency mode": spp_config.gps_freq,
        "Galileo frequency mode": spp_config.gal_freq,
        "BeiDou frequency mode": spp_config.bds_freq,
        "orbit source": spp_config.orbit_type,
        "ionosphere model (single-frequency modes only)": spp_config.ionosphere_model,
        "troposphere model": spp_config.troposphere_model,
        "satellite PCO": spp_config.sat_pco,
        "receiver PCO": spp_config.rec_pco,
        "relativistic path": spp_config.rel_path,
        "antenna height (ANTENNA: DELTA H/E/N)": spp_config.antenna_h,
        "station": spp_config.station_name,
        "window start": f"{TUTORIAL_WINDOW[0]:%Y-%m-%d %H:%M}",
        "window end": f"{TUTORIAL_WINDOW[1]:%Y-%m-%d %H:%M}",
    }
)
viz.df_head(config_summary.rename_axis("setting").to_frame("value"), nrows=14, truncate_str=
    None)
```

setting	value
systems	G
GPS frequency mode	L1L2
Galileo frequency mode	E1E5a
BeiDou frequency mode	B1IB3I
orbit source	broadcast
ionosphere model (single-frequenc…	klobuchar
troposphere model	saastamoinen
satellite PCO	los
receiver PCO	True
relativistic path	True
antenna height (ANTENNA: DELTA H/…	True
station	BRUX
window start	2024-02-04 00:00
window end	2024-02-05 00:00

```
with contextlib.redirect_stdout(io.StringIO()) as spp_log:
    spp_result = gnx.SPPSession(spp_config).run()

```

```
for line in spp_log.getvalue().splitlines()[:5]:
    print(line)

solution = spp_result.solution.copy()
print(f"Solved epochs: {len(solution)}")
summary_cols = ["time", "de", "dn", "du", "n_gps", "rms_v", "iters"]
viz.df_head(solution[summary_cols], nrows=5, ncols=7)
viz.df_tail(solution[summary_cols], nrows=5, ncols=7)
```

```
<repo>/gnx_py/core/coordinates.py:2666: RuntimeWarning: GPS satellites without a usable
    broadcast ephemeris (left out): G01 (unhealthy, health=63).
  result = self.get_coordinates()


Solved epochs: 2880
```

time	de	dn	du	n_gps	rms_v	iters
2024-02-04 00:00:00	-0.238	-0.102	-2.008	10	0.593	2
2024-02-04 00:00:30	0.202	-0.406	0.354	10	0.437	2
2024-02-04 00:01:00	0.541	0.424	-1.531	10	0.442	2
2024-02-04 00:01:30	0.053	-0.297	-2.591	10	0.994	2
2024-02-04 00:02:00	-0.501	0.062	-2.834	10	0.607	2

time	de	dn	du	n_gps	rms_v	iters
2024-02-04 23:57:30	0.453	-0.268	0.533	10	0.727	2
2024-02-04 23:58:00	0.254	0.095	0.707	10	0.670	2
2024-02-04 23:58:30	0.564	0.074	1.197	10	1.085	2
2024-02-04 23:59:00	0.017	-0.460	0.899	10	0.718	2
2024-02-04 23:59:30	0.366	0.170	-0.100	10	1.026	2

1.4.1 `SPPResult`: solution, residuals, and diagnostics

The result is a dataclass. The most important field is `solution`, which stores estimated receiver coordinates, receiver clock terms, quality diagnostics, and ENU errors when a reference coordinate is available.

The columns `de`, `dn`, `du` are the reference minus the estimate, rotated to east/north/up at the reference. The reference is the station's SOLUTION/ESTIMATE position in the SINEX file of `sinex_path`, the solution valid at the first observation epoch, propagated to that epoch with its velocities when the file gives them (the daily IGS solution in `sample_data` has none). A station that is not in the SINEX file gives NaN columns and one warning; without `sinex_path` the columns are absent. The SINEX coordinates refer to the marker, and so does the SPP estimate: with `antenna_h=True` (the default, as in PPP) the antenna height of the RINEX header (0.4689 m for BRUX) is removed along the line of sight, see the receiver PCO section. With `antenna_h=False` the estimate is the antenna reference point and `du` contains the antenna height with a negative sign (about -0.47 m for BRUX).

The residual tables are split by constellation:

- `residuals_gps`
- `residuals_gal`
- `residuals_bds`

Each table keeps the preprocessed observations of its system, indexed by satellite and epoch, with the columns the adjustment writes:

- `V`: the post-fit pseudorange residual of an observation used in the epoch's solution (computed minus observed). `V` is finite exactly for the observations used.
- `V_rejected`: the residual of an observation that the outlier loop removed, against the final solution of its epoch; NaN for every other row. `viz.plot_spp_residuals` draws these values as black crosses.
- `spp_rejected` and `spp_reject_reason`: why an observation was not used, either for itself (`postfit_residual`, `nonfinite_correction`) or because its whole epoch was rejected (for example `gdop_above_limit` with the default `spp_max_gdop=30`, or `insufficient_observations`).

In this GPS-only example only the GPS residual table is populated. Later tutorials will use this structure to compare systems and inspect processing diagnostics.

```
result_fields = []
for field in fields(spp_result):
    value = getattr(spp_result, field.name)
    result_fields.append(
        {
            "field": field.name,
            "type": type(value).__name__,
            "shape": getattr(value, "shape", None),
        }
    )

viz.df_head(pd.DataFrame(result_fields))

print(type(spp_result.residuals_gps).__name__, spp_result.residuals_gps.shape)
print("Galileo residuals:", spp_result.residuals_gal)
print("BeiDou residuals:", spp_result.residuals_bds)
viz.df_head(
    spp_result.residuals_gps[["P3", "ev", "tro", "V", "V_rejected", "spp_rejected"]],
    nrows=6,
    ncols=8,
)

rejected = spp_result.residuals_gps["spp_rejected"]
print(f"Observations not used: {int(rejected.sum())} of {len(rejected)}")
if rejected.any():
    viz.df_head(spp_result.residuals_gps.loc[rejected, "spp_reject_reason"].value_counts().
    to_frame("rows"))
```

field	type	shape
solution	DataFrame	(2880, 24)
residuals_gps	DataFrame	(25376, 87)
residuals_gal	NoneType	None
residuals_bds	NoneType	None

```
DataFrame (25376, 87)
Galileo residuals: None
BeiDou residuals: None
```

sv	time	P3	ev	tro	V	V_rejected	spp_rejected
G02	2024-02-04 00:00:00	22182503.154	44.036	3.380	-0.247	nan	False
G02	2024-02-04 00:00:30	22196048.875	43.812	3.394	-0.906	nan	False
G02	2024-02-04 00:01:00	22209652.380	43.587	3.408	-0.134	nan	False
G02	2024-02-04 00:01:30	22223315.718	43.362	3.422	-0.035	nan	False
G02	2024-02-04 00:02:00	22237036.435	43.138	3.436	0.230	nan	False
G02	2024-02-04 00:02:30	22250818.241	42.913	3.451	-0.431	nan	False

```
Observations not used: 0 of 25376
```

In this run every observation that passed the elevation mask entered the solution, so `V_rejected` is NaN everywhere; on data with outliers the same columns tell which observations were removed and why.

1.4.1.1 Warnings

The session above printed one warning: G01 is unhealthy on this day and is left out. It printed none about `ionosphere_model`, which has no effect in this ionosphere-free run (see the Klobuchar section). GNX-py reports conditions that change the processing or the meaning of the results (an unhealthy or excluded satellite, a missing correction, an input file it does not use, such as `clk_path` with broadcast orbits, an antenna model that differs from the one of the orbit product) as Python warnings, as a rule once per run, also when several processing steps meet the same condition. Not every switch that does nothing on a route warns: `ionosphere_model` in an ionosphere-free mode passes silently. A configuration reused for another run warns again (Python's default warning filter may still show an identical message only once per Python process). The tutorials leave these warnings visible instead of silencing them with `warnings.filterwarnings("ignore")`: they are part of the result.

1.4.2 Interpreting position errors and RMS

The columns `de`, `dn`, and `du` are local East, North, and Up errors with respect to the reference station coordinates. A compact way to summarize the run is to compute component RMS values and a 3D RMS:

$$\mathrm{RMS}_e = \sqrt{\frac{1}{N}\sum_i e_i^2}, \quad \mathrm{RMS}_n = \sqrt{\frac{1}{N}\sum_i n_i^2}, \quad \mathrm{RMS}_u = \sqrt{\frac{1}{N}\sum_i u_i^2}$$

$$\mathrm{RMS}_{3D} = \sqrt{\mathrm{RMS}_e^2 + \mathrm{RMS}_n^2 + \mathrm{RMS}_u^2}$$

For this notebook the numbers are not a benchmark. They are a quick check that the session produced a physically interpretable positioning time series.

```
enu_cols = ["de", "dn", "du"]
rms_components = np.sqrt((solution[enu_cols] ** 2).mean())
rms_table = rms_components.rename({"de": "east", "dn": "north", "du": "up"}).to_frame("RMS_m")
rms_table.loc["3D", "RMS_m"] = float(np.sqrt((rms_components ** 2).sum()))
viz.df_head(rms_table.rename_axis("component"))

fig = viz.plot_spp_positioning(
    solution,
    columns=enu_cols,
    show_satellites=False,
    title="SPP session: BRUX, GPS L1/L2, broadcast navigation",
    figsize=(10, 8),
)
```

component	RMS_m
east	0.621
north	0.920
up	2.038
3D	2.321

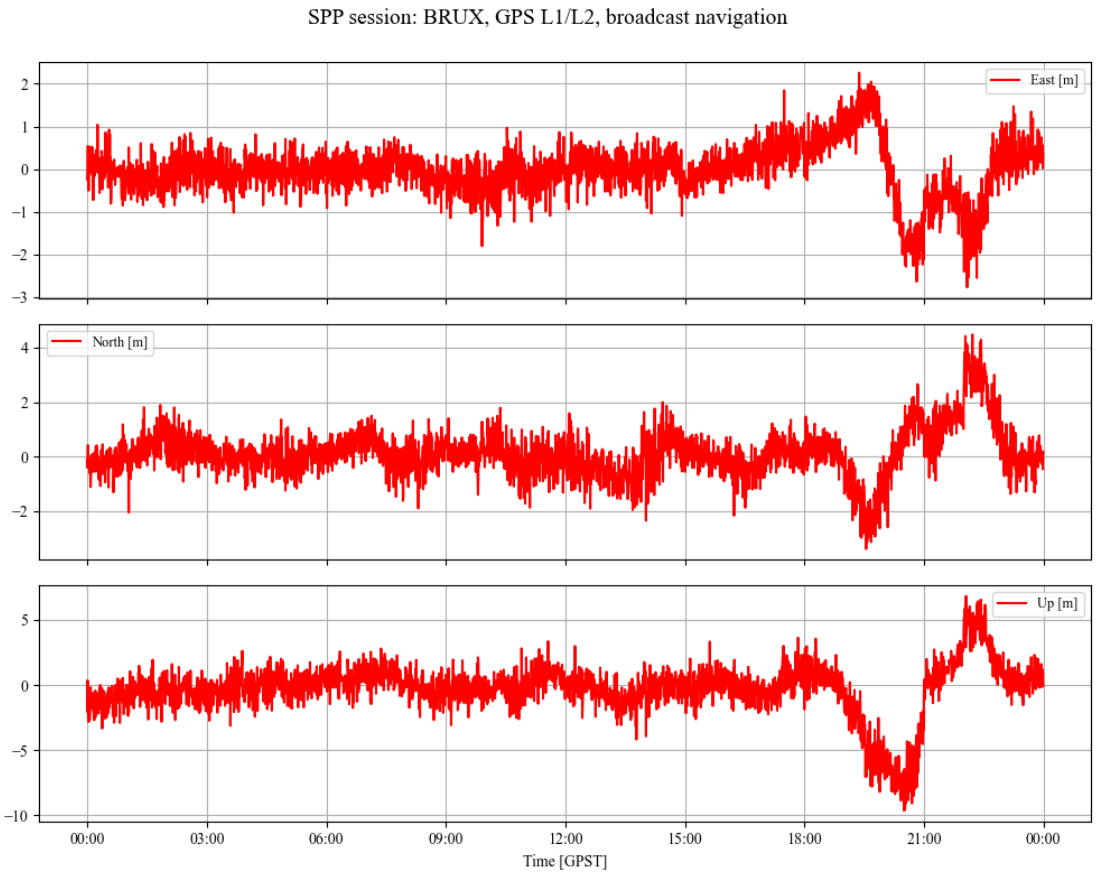


Figure 1.1: SPP position-error time series for BRUX using GPS broadcast navigation.

1.5 Summary

In this chapter you saw the processing chain from raw input files to a structured SPP result:

1. `GNSSDataProcessor2` loaded observations, antenna metadata, station metadata, and broadcast navigation products with the library's own RINEX and ANTEX readers.
2. `BroadcastInterp` selected broadcast messages, left out the unhealthy satellite, and propagated satellite coordinates and clocks.
3. We moved from reception time to emission time and corrected satellite coordinates for Earth rotation.
4. We inspected elevation/azimuth, Klobuchar ionosphere, Saastamoinen troposphere, relativistic path range, receiver/satellite PCO ideas, and antenna height projection.
5. We solved a small manual one-epoch SPP example to see the geometry of the problem.
6. We then used `SPPConfig` and `SPPSession` to run the automated GNX-py pipeline and read its residual columns (`V`, `V_rejected`, `spp_reject_reason`) and warnings.

For your own stations and days, `DownloadConfig` and `DownloadSession` fetch the observations and a consistent set of products. The next tutorial builds on this foundation and moves from SPP to PPP, where precise products, combined/uncombined observables, ionospheric constraints, and bias policy become more central.

Chapter 2

Precise Point Positioning

2.1 PPP in GNX-py

In this tutorial, I will discuss the PPP module of the GNX-py library. Since the core of the PPP technique is the use of the Kalman filter (Kouba and Héroux 2001), I will first discuss the basic information about this algorithm and its application in PPP. For detailed information about the Kalman filter, its implementation methods, and its variants, I encourage you to refer to the guide (Labbe 2020). GNX-py builds on the FilterPy library discussed there: every PPP filter uses `GNXExtendedKalmanFilter`, a subclass of FilterPy's `ExtendedKalmanFilter` that computes the Kalman gain with a Cholesky solve of the symmetrised innovation covariance, leaves rejected observations out of the update, and re-initialises the receiver clock from the code observations (details below).

I will also discuss the corrections used in PPP: precise satellite orbits and clocks, troposphere, ionosphere, satellite and receiver phase-center offsets and variations, phase wind-up, relativistic path correction, solid Earth, ocean and pole tides, satellite attitude, and bias products. This helps you understand what happens during parameter estimation instead of treating PPP as a black box.

Current GNX-py supports several PPP routes: combined ionosphere-free PPP, uncombined PPP, single-system and mixed-GNSS processing, GPS/Galileo/BeiDou signal metadata, ionospheric constraints, trace diagnostics, and optional PPP ambiguity resolution (PPP-AR). We will keep the old tutorial style - intuition, equations, code, and interpretation - while updating the examples to the current API and `sample_data`.

2.1.1 Parameters estimation in PPP

Observation model:

For each satellite s and frequency i code and phase observations at a single frequency can be written as (Teunissen and Montenbruck 2017):

$$
\begin{aligned}
C_i &= \rho_r^s + (\Delta_{PCO,i}^{rcv} - \Delta_{PCO,i}^{s}) - c(dt^s - dt_r) + \gamma_i I \\
&\quad + T + me_{wet} ZTD + \Delta_{ah} + \rho_{rel} + e_r^s \, \delta_{tides} + OSB_{C_i} + \varepsilon_{C_i} \\
L_i &= \rho_r^s + (\Delta_{PCO,i}^{rcv} - \Delta_{PCO,i}^{s}) - c(dt^s - dt_r) - \gamma_i I \\
&\quad + T + me_{wet} ZTD + \Delta_{ah} + \rho_{rel} + e_r^s \, \delta_{tides} + OSB_{L_i} + \lambda_i N_i + \lambda_i phw + \varepsilon_{L_i}
\end{aligned}
$$

where:

C_i — code pseudorange observation

L_i — carrier phase observation

ρ_i — geometric distance between satellite s and receiver r

$\Delta_{PCO,i}^{r}$ - receiver phase center offset

$\Delta^s_{PCO,i}$ - satellite phase center offset

c — speed of light

$d_{ts,i}$ — satellite clock offset

dt_r - receiver clock offset

T_i — slant tropospheric delay

me_{wet} - mapping function

ZTD - zenith tropospheric delay

Δ_{ah} - antenna height

ρ_{rel} - relativistic path range correction

e^s_r - line-of-sight unit vector

δ_{tides} - site displacement: solid Earth tide (IERS 2010, steps 1 and 2), ocean tide loading (`ocean_loading_path`) and pole tide (`erp_path`)

OSB_{C_i} / OSB_{L_i} - observable specific satellite hardware bias

λ_i — carrier phase wavelength at frequency i

N_i — ambiguity for satellite i

phw - carrier phase wind-up correction

$\varepsilon^C_i, \varepsilon^L_i$ — measurement noise

Ionosphere-free observations, that are free from the influence of the ionosphere are recorded as (Teunissen and Montenbruck 2017) :

$$
\begin{aligned}
C_{IF} &= \frac{f_1^2}{(f_1^2 - f_2^2)} C1 - \frac{f_2^2}{(f_1^2 - f_2^2)} C2 \\
&= \rho^s_r + (\Delta^{rcv}_{PCO,IF} - \Delta^s_{PCO,IF}) - c(dt^s - dt_r) + T + me_{wet}\, ZTD \\
&+ \Delta_{ah} + \rho_{rel} + e^s_r\, \delta_{tides} + \varepsilon_{C_{IF}}
\end{aligned}
$$

$$
\begin{aligned}
L_{IF} &= \frac{f_1^2}{(f_1^2 - f_2^2)} L1 - \frac{f_2^2}{(f_1^2 - f_2^2)} L2 \\
&= \rho^s_r + (\Delta^{rcv}_{PCO,IF} - \Delta^s_{PCO,IF}) - c(dt^s - dt_r) + T + me_{wet}\, ZTD \\
&+ \Delta_{ah} + \rho_{rel} + e^s_r\, \delta_{tides} + \varepsilon_{L_{IF}} + \lambda_{IF} N_{IF}
\end{aligned}
$$

Current version note. The equations above remain the conceptual foundation for both PPP routes in GNX-py. In the `combined` route, the code and phase observations are first transformed into ionosphere-free combinations. In the `uncombined` route, each frequency is modeled more directly and the ionosphere can become an explicit state or a constrained pseudo-observation. The practical API switch is `PPPConfig(positioning_mode="combined")` or `PPPConfig(positioning_mode="uncombined")`.

2.2 Combined and uncombined PPP

The most important modeling choice in this tutorial is the difference between `combined` and `uncombined` PPP.

2.2.1 PPP combined

In combined PPP we form ionosphere-free combinations of two frequencies before the filter update. For code:

$$P_{IF} = k_1 P_1 - k_2 P_2, \qquad k_1 = \frac{f_1^2}{f_1^2 - f_2^2}, \quad k_2 = \frac{f_2^2}{f_1^2 - f_2^2}$$

and analogously for phase. The first-order ionospheric delay changes sign between code and phase, but it scales with $1/f^2$, so the combination removes it to first order.

The benefit is simplicity: the filter does not need one slant-ionosphere state per satellite. The cost is that measurement noise increases, because two noisy measurements are linearly combined. The ambiguity is also no longer a clean integer ambiguity on one carrier. It becomes an ionosphere-free ambiguity combination, which is an integer only together with the wide lane; the combined filters of GNX-py do not separate the wide lane, so they fix no ambiguity and keep the float solution.

Current combined PPP routes include single-system and mixed-system filters. The main public entry point remains `PPPConfig(positioning_mode="combined")` and `PPPSession(config).run()`.

2.2.2 PPP uncombined

In uncombined PPP, the filter keeps frequency-specific code and phase observations. The ionosphere can be modeled explicitly:

$$\begin{aligned}
P_1 &\approx \rho + c\,dt_r - c\,dt^s + T + I_1 + b_{P_1} + \varepsilon_{P_1} \\
P_2 &\approx \rho + c\,dt_r - c\,dt^s + T + I_2 + b_{P_2} + \varepsilon_{P_2} \\
L_1 &\approx \rho + c\,dt_r - c\,dt^s + T - I_1 + \lambda_1 N_1 + b_{L_1} + \varepsilon_{L_1} \\
L_2 &\approx \rho + c\,dt_r - c\,dt^s + T - I_2 + \lambda_2 N_2 + b_{L_2} + \varepsilon_{L_2}
\end{aligned}$$

The uncombined layout is more flexible: it can use ionospheric constraints, keep separate ambiguities, and handle signal-specific biases more directly. It is also harder numerically because the state vector is larger and some states are strongly correlated. Bias products matter a lot here. OSB, DCB, and DSB products encode different datum assumptions.

2.2.3 Practical comparison

Topic	Combined PPP	Uncombined PPP
Main idea	Build ionosphere-free observables before filtering	Model code/phase observations per frequency
Ionosphere	First-order ionosphere removed algebraically	Estimated explicitly or constrained with GIM/STEC-like products
Noise	IF combination amplifies observation noise	Raw observations keep native noise but add more states
Ambiguities	IF ambiguity is not a simple original carrier integer	Frequency-specific ambiguities are more explicit
Biases	Bias handling often hidden inside IF products/combinations	Bias datum is central; OSB/DCB/DSB consistency matters
AR	Combined IF filters fix no ambiguity (float result)	More natural AR target, still validation-sensitive
Simplicity	Easier baseline route	More flexible but more complex
GNX-py recommendation	Use for baseline PPP validation	Use for ionosphere constraints, advanced diagnostics, and AR experiments

The PPP technique enables the positioning of a single receiver using phase and code observations with sub-centimeter accuracy (Kouba and Héroux 2001; Glaner 2022; Schönemann 2014). The accuracy of the final position depends on the quality of the orbit models, clocks, and corrections used. PPP positioning involves determining a number of unknown parameters. For the purposes of this introduction, we will consider the case of a conventional model, in which code and phase observations are combined into

ionosphere-free observations in which first-order ionospheric delay is eliminated. In this situation, our state vector contains the following parameters:

$$\mathbf{x} = \begin{bmatrix} X & Y & Z & d_{tr} & isb & ztd & N_1 & N_2 & ... & N_n \end{bmatrix}^{\mathrm{T}}$$

where

X, Y, Z — receiver coordinates (ECEF) dt_r — receiver clock offset ztd — zenith tropospheric delay isb - Inter system bias
N_i — carrier phase ambiguity for satellite i

X, Y, Z, and dt_r are the receiver coordinates and offset of the receiver clock, isb is the inter-system bias—in the case of processing data from two GNSS constellations at once, it reflects the difference in the time reference system and receiver delays between the processed signals from different GNSS. N_i are phase ambiguities for a $i-th$ satellite. This is the total number of phase cycles on the satellite-receiver path. During estimation, it is determined as a floating point number.

Current version note. This state vector describes the conventional combined ionosphere-free model. In the uncombined model, the state is larger because the filter may carry frequency-specific ambiguities, ionospheric states, and receiver/signal bias terms. A simplified uncombined state can be written as:

$$\mathbf{x}_{UDUC} = \begin{bmatrix} X & Y & Z & d_{tr} & isb & ztd & I_1 & ... & I_n & N_{1,1} & N_{1,2} & ... & b_{C,1} & b_{C,2} & ... \end{bmatrix}^{\mathrm{T}}$$

where I_i is a slant ionospheric delay state for satellite i, $N_{i,j}$ is the phase ambiguity for satellite i and frequency j, and b_C terms represent receiver/signal code-bias states or datum choices used by the selected uncombined branch. This is why uncombined PPP is more flexible, but also more sensitive to bias products and numerical constraints.

EKF prediction:

Let's discuss the process of parameter estimation using the Extended Kalman Filter. We assume that we have access to a time series of GNSS observations from a given receiver. We call what we measure a state: our state is the estimated parameters. The Kalman filter, in very simple terms, combines the prediction of the state of our system with the state obtained from measurements. Filtration is performed sequentially: in each epoch, we process the measurements available in that epoch and perform a prediction and update step. Let's first discuss the prediction step and look at the equations:

$$\hat{\mathbf{x}}_k = \mathbf{F}\hat{\mathbf{x}}_{k-1}$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_{k-1}\mathbf{P}_{k-1|k-1}\mathbf{F}_{k-1}^{\mathrm{T}} + \mathbf{Q}_{k-1}$$

$\hat{\mathbf{x}}_k$ and $\hat{\mathbf{x}}_{k-1}$ are our state vectors in epochs k and $k-1$, respectively. This vector contains the parameters discussed at the beginning of this section, i.e., the receiver and satellite parameters.

Matrix F is the so-called state transition matrix. The $F\,x$ operation is intended to map the predicted evolution of the state between adjacent epochs. Matrix F is an element of the process model, i.e., it is a mathematical representation of our knowledge or assumptions about the behavior of the system. Let's illustrate this with an example. Assuming the state vector discussed above and a stationary GNSS receiver, we can make the following assumptions about our system:

- the position of the receiver does not change
- we do not know the characteristics of the clock - we assume white noise with high variance
- we assume that isb is constant over time
- we assume that ZTD can be described as a random walk
- satellite uncertainties are constant, assuming that cycle slips have already been eliminated
- In this case, the matrix F will look like this:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 1 \end{bmatrix}$$

Each column of the matrix corresponds to a parameter in vector X. Try multiplying an example vector, even a 3 or 4 element vector, by a similar matrix on a piece of paper—you will see that after multiplication you will get the same vector (except for the clock, but we will get to that in a moment). This is the task of the matrix F - it reflects our predicted behavior of the system during the passage of one epoch.

The P matrix is an a-priori error matrix of the state vector elements. In the prediction step, the variance is propagated to the next epoch. The Q matrix, or process noise matrix, reflects our uncertainty about the process model. In other words, the Q matrix reflects our confidence in the accuracy of the description of the system's behavior by the F matrix. Assuming that we are certain that the receiver is stationary and modeling the clock as white noise, the first three diagonal elements of the Q matrix will be equal to zero (for coordinates), while for the clock we set the variance to a very large value (the combined preset of GNX-py uses `q_dt` = 9e9 m², the uncombined one 1e9 m²). We assume that the process model accurately describes the ambiguities and ISB as constant over time, so the corresponding elements of the Q matrix are also equal to zero. We model ZTD as a random walk: GNX-py adds $q_{tro}\,\Delta t$ per epoch, with `q_tro` = 1e-8 m²/s (about 6 mm per square root of an hour) and Δt the time since the previous epoch in seconds.

$$\mathbf{Q} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 9 \cdot 10^9 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 0 & 0 & & q_{tro}\,\Delta t & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \end{bmatrix}$$

A white-noise clock with such a variance is practically free at every epoch. GNX-py therefore gives it a prior from the code: at every epoch where the predicted clock variance exceeds `clock_prior_sigma_m`^{2} ((100 m)² by default), the receiver clock is re-initialised from that epoch's code observations with this variance before the update.

EKF update:

In the update step, we apply our measurements into the model and combine information about the measurements and their accuracy with information about the predicted state of the system and the accuracy of this prediction.

$$\begin{aligned} \mathbf{y}_k &= \mathbf{z}_k - \mathbf{h}(\hat{\mathbf{x}}_{k|k-1}) \\ \mathbf{S}_k &= \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^{\mathrm{T}} + \mathbf{R}_k \\ \mathbf{K}_k &= \mathbf{P}_{k|k-1} \mathbf{H}_k^{\mathrm{T}} \mathbf{S}_k^{-1} \\ \hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k \\ \mathbf{P}_{k|k} &= (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^{\mathrm{T}} + \mathbf{K}_k \mathbf{R}_k \mathbf{K}_k^{\mathrm{T}} \end{aligned}$$

The last line is the Joseph form of the covariance update used by FilterPy and GNX-py; it equals $(\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)\mathbf{P}_{k|k-1}$ for the optimal gain, but keeps $\mathbf{P}$ symmetric and positive definite in floating-point arithmetic.

where
$\mathbf{y_k}$ - prefit residuals

$\mathbf{z}_k$ — vector of observed measurements at epoch k
$\mathbf{h}(\cdot)$ — nonlinear measurement function
$\mathbf{H}_k$ — Jacobian of $\mathbf{h}$ with respect to $\mathbf{x}$
$\mathbf{R}_k$ — measurement noise covariance
$\mathbf{S_k}$ - prefit residuals covariance matrix
$\mathbf{K_k}$ - Kalman gain
$\mathbf{Q}_{k-1}$ — process noise covariance
$\mathbf{\hat{x}_k}$ - updated state vector
$\mathbf{P_{k|k}}$ - updated a-priori covariance matrix

Prefit residuals are the difference between measurements and predicted measurements. In our case, this is the difference between GNSS observations, $\mathbf{z_k}$, corrected for all available corrections, and calculated (predicted) observations. The function $\mathbf{h}(\cdot)$ is a function that maps the elements of the state vector to observations. In our case, predicted pseudorange and phase observation, denoted by $\hat{z_{C_i}}$ and $\hat{z_{L_i}}$ are computed as:

$$\hat{z_{C_i}} = \rho + (dt_r + isb) + me_{wet}\, ZTD$$
$$\hat{z_{L_i}} = \rho + (dt_r + isb) + me_{wet}\, ZTD + \lambda_i\, N_i$$

Where dt_r, isb, ZTD and N_i are predicted elements of state vector; the receiver clock and the ISB are expressed in metres, and the ionosphere-free filters estimate the float ambiguity term $\lambda_i N_i$ directly in metres.

The Jacobian matrix is a design matrix—as in standard LSQ, each column contains the partial derivatives of the observation equation with respect to a given element of the state vector.

$$\mathbf{H}_k = \begin{bmatrix} \frac{\partial h_1}{\partial X} & \frac{\partial h_1}{\partial Y} & \frac{\partial h_1}{\partial Z} & \frac{\partial h_1}{\partial d_{tr}} & \frac{\partial h_1}{\partial \mathrm{ISB}} & \frac{\partial h_1}{\partial ztd} & \frac{\partial h_1}{\partial N_1} & 0 & \cdots & 0 \\ \frac{\partial h_2}{\partial X} & \frac{\partial h_2}{\partial Y} & \frac{\partial h_2}{\partial Z} & \frac{\partial h_2}{\partial d_{tr}} & \frac{\partial h_2}{\partial \mathrm{ISB}} & \frac{\partial h_2}{\partial ztd} & 0 & \frac{\partial h_2}{\partial N_2} & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial h_m}{\partial X} & \frac{\partial h_m}{\partial Y} & \frac{\partial h_m}{\partial Z} & \frac{\partial h_m}{\partial d_{tr}} & \frac{\partial h_m}{\partial \mathrm{ISB}} & \frac{\partial h_m}{\partial ztd} & 0 & 0 & \cdots & \frac{\partial h_m}{\partial N_m} \end{bmatrix}$$

Assuming that we are processing code and phase observations from two GNSS systems, each row of the $\mathbf{H_k}$ matrix corresponds to a given observation from the $\mathbf{z}$ vector. The partial derivatives with respect to ambiguities are equal to zero for code observations, and similarly, derivative with respect to ISB is equal to zero for the reference constellation for the receiver clock (GPS by default). Assuming that for each satellite we process one IF code observation and one IF phase observation, the vector $\mathbf{z}$ and the matrix $\mathbf{h_k}$ will have dimensions of $N \cdot 2$ and ($N \cdot 2$, 6+N), respectively, where N is the number of satellites visible at a given epoch and 6 stands for the 6 default parameters: the three station coordinates, receiver clock offset, ISB and ZTD.

The R matrix is a diagonal matrix of measurement variances. In GNX-py you can customize code variances and phases, although I can only vouch for the default values I used for positioning classes. The variances depend on the elevation e and on the filter class that the session selects. The ionosphere-free (combined) filters use

$$\sigma^2 = a + \frac{b}{\sin^2 e}$$

and the uncombined filters scale a zenith variance c with $1/\sin e$:

$$\sigma^2 = \frac{c}{\sin e}$$

Filter class (route)	Code variance [m²]	Phase variance [m²]
`PPPDualFreqMultiGNSS` (combined, two or more systems), GPS and Galileo	$1 + 0.0025/\sin^2 e$	$10^{-4} + 0.0003/\sin^2 e$

Filter class (route)	Code variance [m²]	Phase variance [m²]
`PPPDualFreqMultiGNSS`, BeiDou; `PPPDualFreqSingleGNSS` (combined, one system)	$9 + 0.0025/\sin^2 e$	$9 \cdot 10^{-4} + 0.0003/\sin^2 e$
uncombined without ionospheric constraints (`PPPUdSingleGNSS`, `PPPUdGenericMixedGNSS`)	$c/\sin e$, $c =$ `uncombined_code_var` $= 1.0$	$c/\sin e$, $c =$ `uncombined_phase_var` $= 10^{-4}$
uncombined with ionospheric constraints (`PPPUdSingleGNSS`, `PPPUducSFSingleGNSS`, `PPPFilterMultiGNSSIonConstGEC`, `PPPFilterMultiGNSSIonConst`)	$c_{sys}/\sin e$, `uncombined_iono_code_var_by_system` : G 0.3, E 0.3, C 1.0	$c/\sin e$, $c =$ `uncombined_phase_var` $= 10^{-4}$
`PPPUdMixedGNSS` (G+E with constraints, at least one single-frequency system)	$1/\sin e$ (fixed in the code)	$10^{-3}/\sin e$ (fixed in the code)
`PPPSingleFreqSingleGNSS` (single frequency without constraints)	one code/phase average (GRAPHIC) per satellite: $0.25\,(c_{code} + c_{phase})/\sin e$, `uncombined_code_var` $= 1.0$, `uncombined_phase_var` $= 10^{-4}$	(in the average)

The two examples below use the first and the fourth row; the uncombined constrained row covers both dual- and single-frequency routes. BeiDou-3 code observations that cannot be moved to the datum of the precise clocks get at least (3 m)² more. Observations rejected by the code and phase screening are left out of the Kalman update altogether (they are not merely down-weighted), so they cannot pull the receiver clock.

$\mathbf{S_k}$, as shown above, is the covariance matrix of observation residuals. The $\mathbf{K_k}$ - Kalman gain we can intuitively represent as a measure of our "confidence" in the state vector estimate relative to the prediction. A small gain means little "confidence" in the measurements. In the equation for the updated state vector $\hat{x_k}$, in the action $\mathbf{K}_k\mathbf{y}_k$ we weight the difference of measurements and predicted measurements by a measure of our confidence in measurements/uncertainty about predictions.

In GNX-py the approach to matrix composition is as follows: it stacks matrices in GPS, Galileo and BeiDou order, and within systems in code, phase order. This isn't particularly significant, but I mentioned it here for clarity. Notice the matrix H; I've listed the derivatives with respect to all components of the estimated state vector. Naturally, the code observations have a 0 in the derivative over the ambiguity, just as Galileo phase observations have zeros in the column designated for GPS phase ambiguities, and vice versa.

2.2.4 Overview of PPP module

We have already discussed the basics of the Kalman filter. Now we will demonstrate how GNX-py performs precise point positioning. The multi-step preprocessing and the positioning itself are hidden in the `PPPSession` class. The only required argument of this class is an instance of `PPPConfig`.

Current PPP routing in GNX-py includes:

- GPS (`G`), Galileo (`E`), and BeiDou (`C`) metadata and preprocessing.
- `B1IB3I` as the safest current BeiDou dual-frequency default.
- `positioning_mode="combined"` for ionosphere-free PPP.
- `positioning_mode="uncombined"` for frequency-specific code/phase modeling.
- Single-system and mixed-GNSS routes.
- Optional ionospheric constraints for uncombined PPP from a GIM or from the Klobuchar, NTCM or BDGIM broadcast models.
- Bias products: OSB, DCB, and DSB handling where supported by the selected branch.
- Site displacement (solid Earth tide, ocean tide loading, pole tide) and satellite attitude (yaw-steering laws, ORBEX files, a turn guard).
- Trace diagnostics via `trace_filter=True`.
- PPP-AR controls via `pppar_*` fields.

`examples/run_ppp_example.py` is a runnable script that sets up the same kind of session on the `sample_data` files (uncombined GPS+Galileo+BeiDou PPP with the GIM ionospheric constraint and PPP-AR, on the GRG orbits, clocks and OSB biases plus the CAS DSB, which differ from the CODE products of this notebook; one hour by default, `--full` for the whole day). Inside the package, `PPPSession` is defined in `gnx_py.ppp.config` and exported as `gnx_py.PPPSession`; notebooks and scripts use it the same way.

```
import contextlib
import importlib.util
import io
from datetime import datetime
from pathlib import Path

import gnx_py as gnx
import numpy as np
import pandas as pd
from gnx_py import viz

viz.apply_publication_style(font_scale=1.6, mathtext="cm")
```

Below we define paths to the current `sample_data` files. We process one full day of 30 s data (2024-02-04, 2880 epochs). To experiment faster, shorten `PPP_TIME_LIMIT` and `UDUC_TIME_LIMIT`; for a representative PPP benchmark, extend the same configuration to more days and stations.

Figures and table previews come from `gnx_py.viz`, the plotting package of GNX-py (`from gnx_py import viz`); the setup cell above applied its publication style once for the whole notebook. We still load one small statistical helper, `measure_convergence_time()`, from `tutorial/ppp/stat_tools.py`. It is kept outside the notebook so the tutorial compiles to Markdown/PDF without a long utility cell.

We do not filter warnings in this notebook. The PPP runs below print a few of them (for example satellites dropped near attitude turns, or an antenna model that differs from the one of the orbit product); each condition is reported once per run, and the text explains the warnings where they appear.

```
def find_repo_root(start: Path | None = None) -> Path:
    current = (start or Path.cwd()).resolve()
    for candidate in (current, *current.parents):
        if (candidate / "pyproject.toml").exists() and (candidate / "sample_data").exists():
            return candidate
    raise FileNotFoundError("Could not find repository root with sample_data.")


def load_tutorial_module(relative_path: str, module_name: str):
    module_path = REPO_ROOT / relative_path
    spec = importlib.util.spec_from_file_location(module_name, module_path)
    module = importlib.util.module_from_spec(spec)
    assert spec.loader is not None
    spec.loader.exec_module(module)
    return module


REPO_ROOT = find_repo_root()
SAMPLE_DATA = REPO_ROOT / "sample_data"

stat_tools = load_tutorial_module("tutorial/ppp/stat_tools.py", "ppp_stat_tools")
measure_convergence_time = stat_tools.measure_convergence_time

STATION = "BRUX"
NAME = STATION
DOY = 35
SYS = {"G", "E"}

OBS_PATH = SAMPLE_DATA / "BRUX00BEL_R_20240350000_01D_30S_MO.crx.gz"
NAV_PATH = SAMPLE_DATA / "BRDC00IGS_R_20240350000_01D_MN.rnx"
SP3_LIST = [
    SAMPLE_DATA / "COD0MGXFIN_20240340000_01D_05M_ORB.SP3",
```

```
    SAMPLE_DATA / "COD0MGXFIN_20240350000_01D_05M_ORB.SP3",
    SAMPLE_DATA / "COD0MGXFIN_20240360000_01D_05M_ORB.SP3",
]
ATX_PATH = SAMPLE_DATA / "igs20.atx"
BIAS_PATH = SAMPLE_DATA / "COD0OPSFIN_20240350000_01D_01D_OSB.BIA"
GIM_PATH = SAMPLE_DATA / "COD0OPSFIN_20240350000_01D_01H_GIM.INX"
SINEX_PATH = SAMPLE_DATA / "IGS0OPSSNX_20240350000_01D_01D_CRD.SNX"

PPP_TIME_LIMIT = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 5, 0, 0)]
UDUC_TIME_LIMIT = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 5, 0, 0)]

RUN_PPP_EXAMPLES = True

path_table = pd.DataFrame(
    [("OBS", OBS_PATH), ("NAV", NAV_PATH), ("ATX", ATX_PATH), ("BIAS/OSB", BIAS_PATH), ("GIM",
    GIM_PATH), ("SINEX", SINEX_PATH), *[("SP3", p) for p in SP3_LIST]],
    columns=["role", "path"],
)
path_table["relative_path"] = path_table["path"].map(lambda p: str(p.relative_to(REPO_ROOT)))
path_table["exists"] = path_table["path"].map(lambda p: p.exists())
viz.df_head(path_table[["role", "relative_path", "exists"]], nrows=len(path_table), ncols=3,
    truncate_str=None)


def make_ppp_config(
    *,
    positioning_mode: str,
    time_limit,
    systems=None,
    ionosphere_model=None,
    use_iono_constr: bool | None = None,
    use_iono_rms: bool | None = None,
    pppar_enabled: bool = False,
    pppar_combined_if_ar_enabled: bool = False,
    **overrides,
):
    systems = SYS if systems is None else systems
    if ionosphere_model is None:
        ionosphere_model = "gim" if positioning_mode == "uncombined" else False
    if use_iono_constr is None:
        use_iono_constr = positioning_mode == "uncombined"

    kwargs = dict(
        obs_path=OBS_PATH,
        nav_path=NAV_PATH,
        sp3_path=SP3_LIST,
        atx_path=ATX_PATH,
        dcb_path=BIAS_PATH,
        sinex_path=SINEX_PATH,
        gim_path=GIM_PATH,
        sys=systems,
        gps_freq="L1L2",
        gal_freq="E1E5a",
        bds_freq="B1IB3I",
        screen=False,
        orbit_type="precise",
        ionosphere_model=ionosphere_model,
        use_iono_constr=use_iono_constr,
        troposphere_model="niell",
        sat_pco="los",
        rec_pco=True,
        windup=True,
        rel_path=True,
```

```
        solid_tides=True,
        antenna_h=True,
        ev_mask=10,
        cycle_slips_detection=True,
        min_arc_len=10,
        station_name=NAME,
        day_of_year=DOY,
        time_limit=time_limit,
        positioning_mode=positioning_mode,
        trace_filter=False,
        reset_every=0,
        pppar_enabled=pppar_enabled,
        pppar_combined_if_ar_enabled=pppar_combined_if_ar_enabled,
    )
    if use_iono_rms is not None:
        # read only by the uncombined routes with ionospheric constraints; set explicitly elsewhere it gives a warning
        kwargs["use_iono_rms"] = use_iono_rms
    kwargs.update(overrides)
    return gnx.PPPConfig(**kwargs)
```

role	relative path	exists
OBS	sample_data/BRUX00BEL_R_202403500…	True
NAV	sample_data/BRDC00IGS_R_202403500…	True
ATX	sample_data/igs20.atx	True
BIAS/OSB	sample_data/COD0OPSFIN_2024035000…	True
GIM	sample_data/COD0OPSFIN_2024035000…	True
SINEX	sample_data/IGS0OPSSNX_2024035000…	True
SP3	sample_data/COD0MGXFIN_2024034000…	True
SP3	sample_data/COD0MGXFIN_2024035000…	True
SP3	sample_data/COD0MGXFIN_2024036000…	True

As I mentioned, the class "controlling" the processing is `PPPConfig`. It defines both the input products and the scientific strategy.

Important current fields include:

- `obs_path`, `nav_path`, `sp3_path`, `atx_path`, `dcb_path`, `gim_path`, `sinex_path` - paths to observations and products; `clk_path` adds 30 s RINEX clock files of the same analysis centre as the SP3 files.
- `sys` - a set of constellation codes, e.g. `{"G"}`, `{"G", "E"}`, `{"C"}`, or `{"G", "E", "C"}`.
- `gps_freq`, `gal_freq`, `bds_freq` - signal modes. For BeiDou, `B1IB3I` is the safest current dual-frequency default.
- `positioning_mode` - `"combined"` for ionosphere-free combinations, `"uncombined"` for frequency-specific modeling. The old `"single"` route is legacy; prefer `"uncombined"` with a single-frequency mode.
- `orbit_type` - usually `"precise"` for PPP.
- `ionosphere_model` - `"gim"`, `"klobuchar"`, `"ntcm"`, `"bdgim"`, or `False`, depending on the route.
- `use_iono_constr` - adds ionospheric pseudo-observations in uncombined PPP.
- `use_iono_rms`, `sigma_iono_0`, `sigma_iono_end`, `t_end` - control how strongly ionospheric constraints influence the early filter.
- `troposphere_model` - PPP requires `"niell"`.
- `sat_pco`, `rec_pco`, `windup`, `rel_path`, `solid_tides`, `antenna_h` - correction switches; `solid_tide_step2`, `ocean_loading_path`, `erp_path`, `satellite_attitude`, `attitude_path` and `attitude_turn_guard` refine the models (next section).
- `bds_precise_clock_datum` - the ionosphere-free datum of the precise BeiDou clocks (`"auto"` by default).
- `trace_filter` - enables compact filter diagnostics.
- `reset_every` - optional filter reset interval for validation experiments.
- `pppar_*` - PPP ambiguity-resolution controls discussed later in this notebook.

You can control the estimation process by modifying the diagonal values of the P and Q matrices. For example, `p_amb`, `p_crd`, `p_isb`, `p_dt`, etc. are initial state variances, while parameters with prefix `q_` describe process noise: `q_crd` and `q_dt` are variances per epoch, while `q_tro`, `q_iono`, `q_isb`, `q_dcb` and

`q_amb` are random-walk rates in m^2/s, multiplied by the time between epochs. Settings that the selected filter does not read are reported with a warning instead of being ignored silently.

2.2.5 Current GNX-py PPP capability map

The current PPP module is broader than the original tutorial version. The table below is a practical map, not a promise that every combination is production-ready for every data set.

Capability	Current status in GNX-py
GPS	Mature across combined and uncombined workflows
Galileo	Mature across GPS/Galileo combined and uncombined workflows
BeiDou	Supported; `B1IB3I` is the safest current dual-frequency default
Mixed GNSS	Available for combined and uncombined routes; validate products carefully
Combined single-system	Supported
Combined mixed-system	Supported
Uncombined single-system	Supported
Uncombined mixed-system	Supported, product-sensitive
Ionospheric constraints	Available in uncombined PPP: GIM, Klobuchar, NTCM, BDGIM
Bias products	OSB/DCB/DSB handling exists, but datum consistency is essential
PPP-AR	Uncombined routes through `pppar_*` controls (single frequency only with ionospheric constraints); combined filters stay float; validate carefully
Site displacement	Solid Earth tide (IERS 2010 steps 1 and 2), ocean tide loading (BLQ file), pole tide (ERP file)
Satellite attitude	Yaw-steering laws per satellite block, ORBEX attitude files, turn guard
Trace diagnostics	`trace_filter=True` prints compact filter events
Driver script	`examples/run_ppp_example.py` shows a runnable setup

The session routes every configuration to one filter class: `PPPDualFreqSingleGNSS` and `PPPDualFreqMultiGNSS` (combined, one or more systems); `PPPUdSingleGNSS`, `PPPUducSFSingleGNSS` and `PPPSingleFreqSingleGNSS` (uncombined, one system, dual or single frequency); `PPPUdGenericMixedGNSS`, `PPPFilterMultiGNSSIonConstGEC` and `PPPFilterMultiGNSSIonConst` (uncombined, two or more systems) and `PPPUdMixedGNSS` (uncombined G+E with ionospheric constraints and at least one single-frequency system). In normal user code, prefer `PPPConfig` and `PPPSession`; the session routes to the concrete filter class.

2.2.6 Observation models: site displacement, satellite attitude and BeiDou clocks

Three groups of models matter at the centimetre level of PPP. The sample data contain no BLQ, ERP or ORBEX file, so the examples below use the defaults; the settings are shown for your own data (`DownloadSession` downloads ERP and ORBEX files with the product types `"erp"` and `"att"`).

Site displacement. With `solid_tides=True` the station moves with the solid Earth tide of the IERS 2010 Conventions, steps 1 and 2 (`solid_tide_step2=True`, up to about 12 mm radial for K1). `ocean_loading_path` adds ocean tide loading from a BLQ file of the Onsala ocean loading service (the station is found by its four-character site code; a station missing from the file gives one warning and no loading). `erp_path` with an IGS ERP file adds the pole tide. The displacement enters the observation model as $e_r^s \, \delta_{tides}$ above.

Satellite attitude. The satellite PCO, the azimuth of the satellite PCV and the phase wind-up all need the orientation of the satellite body. GNX-py computes one attitude frame per satellite and epoch: from an ORBEX attitude file of the orbit's analysis centre (`attitude_path`) where it covers a satellite's arc, otherwise from the published yaw-steering law of the satellite block (GPS IIR, IIF and III, Galileo

IOV/FOC, BeiDou-2 and BeiDou-3). Near noon and midnight turns at a small Sun elevation above the orbit plane ($|\beta|$), analysis centres sometimes fly the turn the other way round from the published laws, and their clocks then hold one wind-up cycle (about 10.7 cm of the ionosphere-free phase). The turn guard `attitude_turn_guard="auto"` (default) therefore drops a satellite's observations within 35 min of such a turn when the product's centre is not known to follow the laws for that satellite class; the satellite restarts its ambiguity after the turn, and a warning says how many observations were dropped. `"all"` guards every class and `"off"` none.

BeiDou precise clocks. Precise BeiDou clocks refer to an ionosphere-free pair, and products differ: the 2024 MGEX products of `sample_data` refer to IF(B1I,B3I), newer products may refer their BeiDou-3 clocks to IF(B1C,B2a). `bds_precise_clock_datum="auto"` resolves the datum once per session (from the bias product of the clocks' analysis centre, else from a comparison with the broadcast clocks, else IF(B1I,B3I) with a warning) and moves the code observations to it; the result reports it in `solution.attrs["precise_clock_datum"]`.

```
config = make_ppp_config(
    positioning_mode="combined",
    time_limit=PPP_TIME_LIMIT,
    ocean_loading_path="data/brux.blq",            # BLQ file from the Onsala ocean loading service
    erp_path="data/COD0MGXFIN_20240350000_01D_12H_ERP.ERP.gz",
    attitude_path="data/COD0MGXFIN_20240350000_01D_30S_ATT.OBX.gz",  # ORBEX of the orbit's centre
)
```

2.2.7 Example 1 - Conventional combined PPP

```
config = make_ppp_config(
    positioning_mode="combined",
    time_limit=PPP_TIME_LIMIT,
    ionosphere_model=False,
    use_iono_constr=False,
    pppar_enabled=False,
)

config_summary = pd.Series(
    {
        "systems": ", ".join(sorted(config.sys)),
        "positioning_mode": config.positioning_mode,
        "gps_freq": config.gps_freq,
        "gal_freq": config.gal_freq,
        "bds_freq": config.bds_freq,
        "orbit_type": config.orbit_type,
        "ionosphere_model": config.ionosphere_model,
        "use_iono_constr": config.use_iono_constr,
        "time_limit": " to ".join(f"{t:%Y-%m-%d %H:%M}" for t in config.time_limit),
        "pppar_enabled": config.pppar_enabled,
    }
)
viz.df_head(config_summary.rename_axis("field").to_frame("value"), nrows=len(config_summary), ncols=2, truncate_str=None)
```

field	value
systems	E, G
positioning_mode	combined
gps_freq	L1L2
gal_freq	E1E5a
bds_freq	B1IB3I
orbit_type	precise
ionosphere_model	False
use_iono_constr	False
time_limit	2024-02-04 00:00 to 2024-02-05 00…
pppar_enabled	False

The class in which all processing and positioning takes place is `PPPSession`. The only thing we pass to it is our configuration object. In the code cell below we capture the text logs so the notebook remains readable.

```
controller = gnx.PPPSession(config=config)
```

To start processing, call `.run()`. In this tutorial the run covers the full day 2024-02-04 (2880 epochs at 30 s). Multi-day or multi-station validation belongs in a script such as `examples/run_ppp_example.py` rather than in a tutorial notebook.

```
if RUN_PPP_EXAMPLES:
    with contextlib.redirect_stdout(io.StringIO()) as combined_log:
        results = controller.run()
    for line in combined_log.getvalue().splitlines()[:6]:
        print(line)
else:
    results = None
    print("Set RUN_PPP_EXAMPLES=True to execute the combined PPP example.")
```

```
<repo>/gnx_py/ppp/config.py:1135: RuntimeWarning: SP3 files disagree on the position at an
    epoch both carry (more than 1 m): C11 2024-02-04 00:00:00 (14.61 m); that position node is
    not used.
  _, positions = lagrange_reception_interp(obs=obs, sp3_df=sp3_df, degree=degree, owner=self.
    config)


<repo>/gnx_py/ppp/config.py:338: UserWarning: attitude turn guard ('auto'): 388 Galileo
    observations of 3 satellites (E03, E05, E09) near noon/midnight turns at small |beta| are
    dropped and their ambiguities restart after the turn, because the clocks of analysis
    centres can differ from the attitude laws by a wind-up cycle there. attitude_turn_guard='
    off' keeps them with the attitude laws
  crd_by_sys[sys] = self._preprocess(


<repo>/gnx_py/ppp/config.py:338: UserWarning: attitude turn guard ('auto'): 84 GPS observations
     of 1 satellite (G04) near noon/midnight turns at small |beta| are dropped and their
    ambiguities restart after the turn, because the clocks of analysis centres can differ from
    the attitude laws by a wind-up cycle there. attitude_turn_guard='off' keeps them with the
    attitude laws
  crd_by_sys[sys] = self._preprocess(
```

The run prints its warnings. The attitude turn guard drops the observations of a few GPS and Galileo satellites near noon/midnight turns at small $|\beta|$; the satellites restart their ambiguities after the turn. The SP3 check reports that two consecutive daily SP3 files give the position of the BeiDou satellite C11 differently at the day boundary (14.6 m), so that position node is not used for interpolation; the check covers every satellite of the SP3 files, also of systems this run does not process, and reports the node once per run.

```
type(results).__annotations__ if results is not None else "combined example not executed"
```

```
{'solution': "Union['pd.DataFrame', None]",
 'residuals_gps': "Union['pd.DataFrame', None]",
 'residuals_gal': "Union['pd.DataFrame', None]",
 'residuals_bds': "Union['pd.DataFrame', None]",
 'convergence': 'Union[float, int, None]'}
```

Our results are a `PPPResult` class instance. It contains:

- `solution` - a dataframe with estimated receiver coordinates, clock, ZTD, optional ISB, diagnostics, and AR columns. Every route, also an empty solution, has the columns `x`, `y`, `z` (ECEF, metres) and `ct_min`, the filter's convergence time in **minutes** (NaN when the position did not settle). `de`, `dn`, `du` are the reference minus the estimate in east/north/up, from the station's SINEX solution

valid at the first epoch; they are NaN when the run has no `sinex_path` or the station is not in the SINEX file (with one warning).

- `residuals_gps`, `residuals_gal`, `residuals_bds` - residual-enriched observation tables, routed by system: each holds the residuals of its own system, and a system that was not processed stays `None`.
- `convergence` - the same convergence time in hours (`None` when the filter did not declare convergence). It is an internal stability indicator: the filter takes the time after which the epoch-to-epoch change of its position estimate stays below a small threshold (5 mm on most routes). In the tutorial we also use `measure_convergence_time()` from `stat_tools.py` to measure convergence based on position accuracy.

`solution.attrs["precise_clock_datum"]` records the datum of the precise BeiDou clocks used for the code biases (it is not set when BeiDou is not processed, as here). Current version note: the result may also include `ar_fixed`, `ar_ratio`, `ar_ok`, and more detailed `ar_reason_*` columns. These are useful when PPP-AR is enabled or explicitly disabled by a safety gate.

```
if results is not None:
    sol = results.solution.copy()
    print(sol.columns.tolist())
    print("Columns of every route:", gnx.PPPResult.SOLUTION_COLUMNS)
    print(f"ct_min: {sol['ct_min'].iloc[-1]:.1f} min, convergence: {results.convergence} h")
    print("precise_clock_datum:", sol.attrs.get("precise_clock_datum"))
else:
    sol = pd.DataFrame()
    print("No solution available because the example was not executed.")
```

```
['de', 'dn', 'du', 'dtr', 'ztd', 'x', 'y', 'z', 'reference_system', 'systems', 'n_sats_total',
    'n_states', 'n_code_rejected_total', 'n_phase_reset_total', 'ar_fixed', 'ar_ratio', 'ar_ok
    ', 'ar_disabled_reason', 'ar_attempted_groups', 'ar_accepted_groups', 'ar_rejected_groups',
     'ar_last_reason', 'ar_partial_groups', 'ar_full_groups', 'ar_partial_fixed_ambiguities', '
    ar_excluded_no_phase_bias', 'ar_excluded_mixed_products', 'ar_excluded_wide_lane', '
    ar_excluded_partial', 'ar_excluded_half_cycle', 'ar_phase_variance_factor', '
    ar_reason_missing_n1', 'ar_reason_below_min_ambiguities', '
    ar_reason_elevation_size_mismatch', 'ar_reason_n2_size_mismatch', 'ar_reason_n1_index_oob',
     'ar_reason_n2_index_oob', 'ar_reason_lock_age_size_mismatch', 'ar_reason_young_arc', '
    ar_reason_candidate_selection_gate', 'ar_reason_no_reference_satellite', '
    ar_reason_wide_lane_gate', 'ar_reason_ill_conditioned', 'ar_reason_lambda_error', '
    ar_reason_ratio_covariance_invalid', 'ar_reason_ratio_reject', 'ar_reason_no_phase_bias', '
    ar_reason_mixed_products', 'ar_reason_half_cycle', 'ar_reason_eligible_size_mismatch', '
    ar_reason_success_rate', 'ar_reason_narrow_lane_gate', 'ar_reason_fix_validation', '
    ar_reason_position_check', 'ar_reason_if_not_integer', 'ar_reason_accepted', 'isb', 'isb_E
    ', 'n_sats_G', 'n_code_rejected_G', 'n_phase_reset_G', 'ar_G_min_age', 'code_res_rms_G', '
    phase_res_rms_G', 'code_res_scatter_rms_G', 'phase_res_scatter_rms_G', 'n_sats_E', '
    n_code_rejected_E', 'n_phase_reset_E', 'ar_E_min_age', 'code_res_rms_E', 'phase_res_rms_E',
     'code_res_scatter_rms_E', 'phase_res_scatter_rms_E', 'ct_min']
Columns of every route: ('x', 'y', 'z', 'ct_min')
ct_min: 37.0 min, convergence: 0.6166666666666667 h
precise_clock_datum: None
```

As you can see, in the `solution` dataframe we have access to the receiver position, clock offset, ZTD, ISB, and diagnostics. Because we use the SINEX file, we also have topocentric errors with respect to the reference position. In current GNX-py, constellation counters such as `n_sats_G`, `n_sats_E`, and `n_sats_C` may appear depending on the route.

```
if not sol.empty:
    viz.df_tail(sol, nrows=5, ncols=12)
```

time	de	dn	du
2024-02-04 23:57:30	-0.002	-0.004	0.001
2024-02-04 23:58:00	-0.002	-0.004	0.001
2024-02-04 23:58:30	-0.002	-0.004	0.001
2024-02-04 23:59:00	-0.002	-0.004	0.001
2024-02-04 23:59:30	-0.002	-0.004	0.001

PDF preview: 8 column(s) omitted from this wide table.

Let's select GPS and Galileo residuals and visualize them with `viz.plot_residuals`. The function accepts either time or elevation on the x-axis. We use small trimming values here; you can increase `cut_init_time` (epochs dropped at the start of the session) and `skip_obs` (epochs dropped at the start of every satellite arc) to remove more of the initialization phase from the plot.

The combined filter stores two versions of the phase residuals. In `postfit_phase_res`, all satellites of one constellation can share a common offset at each epoch, which belongs to the receiver clock and ISB terms and says nothing about individual satellites (in this run it is small: a median of 2-3 mm and about 1 cm at the 95th percentile of the epochs). `postfit_phase_res_demeaned` removes that per-epoch, per-constellation median, leaving the satellite-specific scatter. We plot the demeaned column with `value_col=` whenever the route provides it.

```
if results is not None:
    res = results.residuals_gps
    res_gal = results.residuals_gal
else:
    res = None
    res_gal = None
```

```
residual_frames = [frame for frame in (res, res_gal) if frame is not None and not frame.empty]
if residual_frames:
    print("Post-fit residual columns:", [c for c in residual_frames[0].columns if c.startswith(
    "postfit_")])

## Demeaned phase residuals when the route provides them; None lets viz pick the default column
    .
PHASE_RES_COL = (
    "postfit_phase_res_demeaned"
    if residual_frames and all("postfit_phase_res_demeaned" in frame.columns for frame in
    residual_frames)
    else None
)
print("Phase residual column used below:", PHASE_RES_COL or "viz default")
```

```
Post-fit residual columns: ['postfit_code_res', 'postfit_phase_res', 'postfit_code_res_demeaned
    ', 'postfit_phase_res_demeaned']
Phase residual column used below: postfit_phase_res_demeaned
```

```
if res is not None and not res.empty:
    fig = viz.plot_residuals(res, obs_type="phase", x_axis="elev", cut_init_time=3, skip_obs=2,
     value_col=PHASE_RES_COL)
```

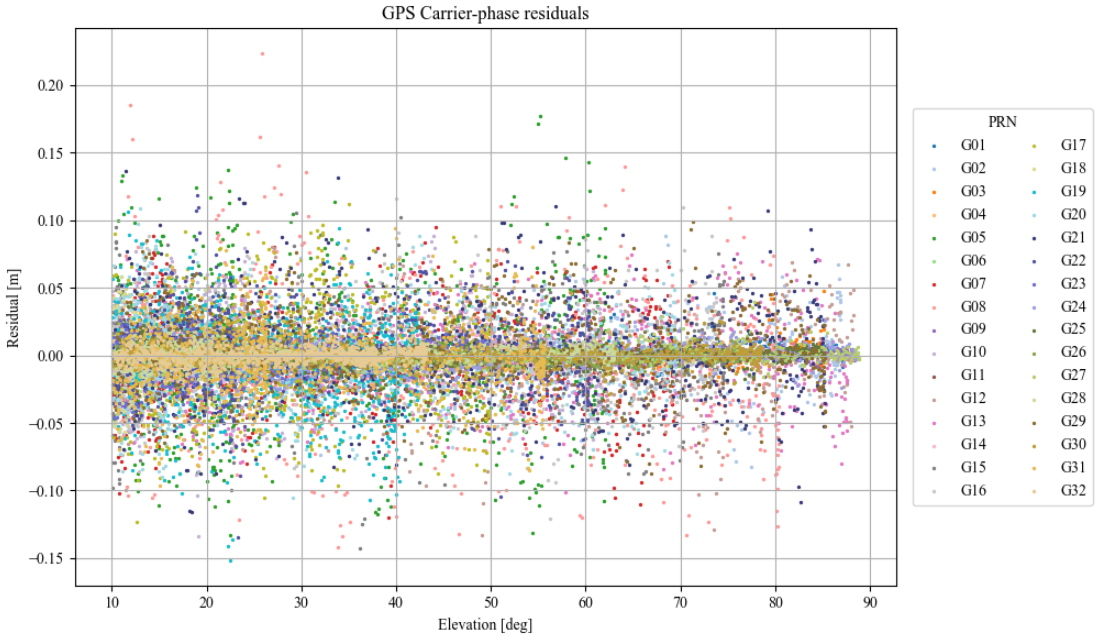


Figure 2.1: GPS phase post-fit residuals plotted against satellite elevation.

```
if res_gal is not None and not res_gal.empty:
    fig = viz.plot_residuals(res_gal, obs_type="phase", x_axis="elev", cut_init_time=3,
    skip_obs=2, value_col=PHASE_RES_COL)
```

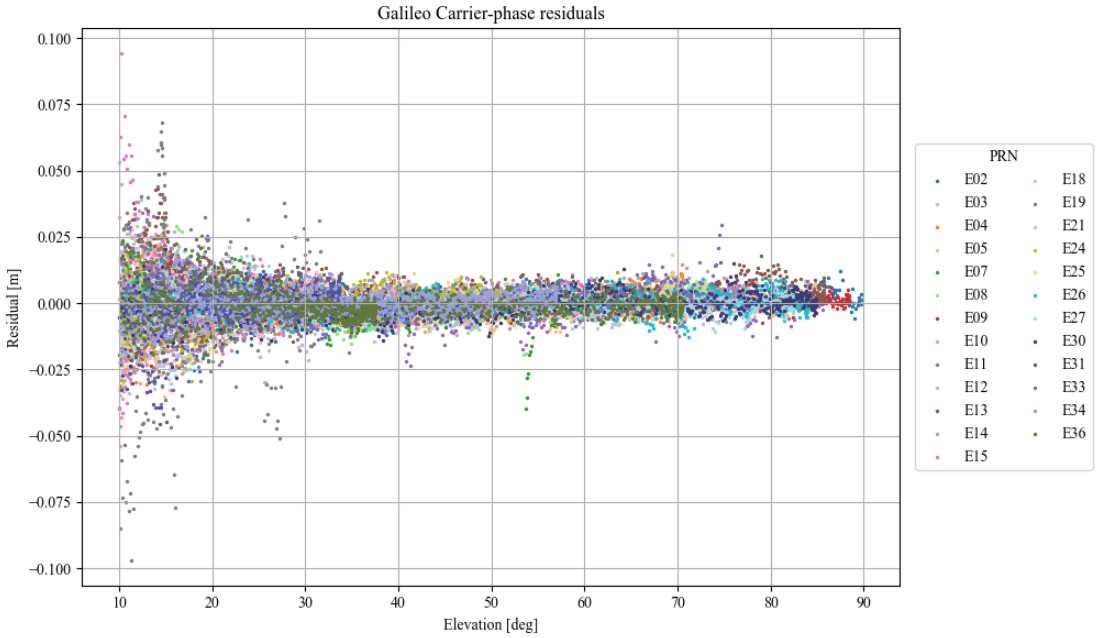


Figure 2.2: Galileo phase post-fit residuals plotted against satellite elevation.

Residual statistics can be processed as ordinary pandas dataframes. The cell below applies the same trimming as the plots (the first 3 epochs of the session and the first 2 epochs of every satellite arc) and computes the standard deviation of the phase residuals per satellite (PRN). The first table compares the two constellations and the second lists the Galileo satellites. In this full-day run, the typical Galileo satellite is about three times less noisy than the typical GPS satellite (median per-PRN standard deviation 0.006 m vs 0.018 m); pooled over all observations the Galileo residuals are 0.007 m against 0.024

m for GPS. The largest Galileo value in the table belongs to E19 (0.026 m), twice the next one (E12, 0.012 m). Each PRN has its own colour in the plot, so the legend identifies the satellites: in this run E19 gives all ten Galileo residuals below -0.06 m in the plot above and about half of those beyond ±3 cm. This is a data-dependent observation for this station and product set rather than a universal rule. Always compare residuals after the same initialization cut, with the same observation type and the same residual column.

```
def residuals_after_cut(frame: pd.DataFrame, columns, cut_init_epochs: int = 3, skip_obs: int =
     2):
    """Residual column(s) trimmed like viz.plot_residuals(cut_init_time=..., skip_obs=...).

    Drops the first ``cut_init_epochs`` epochs of the session, then exactly the
    first ``skip_obs`` epochs of every satellite arc (index level ``sv``);
    indexed by (sv, time).
    """
    times = frame.index.get_level_values("time")
    kept_times = np.sort(times.unique())[cut_init_epochs:]
    values = frame.loc[times.isin(kept_times), columns].sort_index()
    index = values.index.to_frame(index=False)
    arc_epoch = index.groupby("sv")["time"].rank(method="dense").to_numpy()  # 1 = first epoch
    of the arc
    return values[arc_epoch > skip_obs]


prn_std = {}
pooled_std = {}
for system, frame in (("GPS", res), ("Galileo", res_gal)):
    if frame is None or frame.empty:
        continue
    phase_col = PHASE_RES_COL or next((c for c in ("v", "postfit_phase_res", "vl1") if c in
    frame.columns), None)
    if phase_col is None:
        continue
    values = residuals_after_cut(frame, phase_col, cut_init_epochs=3, skip_obs=2)
    prn = values.index.get_level_values("sv").str[:3]
    prn_std[system] = values.groupby(prn).std().rename("phase_residual_std_m").rename_axis("prn
    ").sort_index()
    pooled_std[system] = values.std()

if prn_std:
    overview = pd.DataFrame(
        {
            "system": list(prn_std),
            "n_prn": [len(s) for s in prn_std.values()],
            "all_obs_std_m": [pooled_std[system] for system in prn_std],
            "median_prn_std_m": [s.median() for s in prn_std.values()],
            "max_prn_std_m": [s.max() for s in prn_std.values()],
        }
    )
    viz.df_head(overview, nrows=len(overview), ncols=5, floatfmt=".4f")

if "Galileo" in prn_std:
    viz.df_head(prn_std["Galileo"], nrows=len(prn_std["Galileo"]), ncols=2, floatfmt=".4f")
```

system	n_prn	all_obs_std m	median_prn std_m	max_prn_std m
GPS	32	0.0241	0.0178	0.0505
Galileo	25	0.0073	0.0058	0.0255

prn	phase residual_std m
E02	0.0054
E03	0.0075
E04	0.0060
E05	0.0060
E07	0.0057
E08	0.0058
E09	0.0040
E10	0.0100
E11	0.0092
E12	0.0122
E13	0.0076
E14	0.0049
E15	0.0088
E18	0.0059
E19	0.0255
E21	0.0039
E24	0.0063
E25	0.0055
E26	0.0042
E27	0.0038
E30	0.0044
E31	0.0074
E33	0.0057
E34	0.0047
E36	0.0055

I mentioned the convergence indicator earlier. The filter's own `ct_min` measures when the position estimate stops changing; it needs no reference coordinates, but a position can be stable and still biased. In `stat_tools.py` you will find a simple function that measures convergence for a time series based on accuracy instead: the first time the 2D or 3D error with respect to the reference coordinates has stayed below a threshold for a minimum number of consecutive epochs. The function takes a convergence threshold, mode (`2D` or `3D`), and a minimum stability duration.

```
print(measure_convergence_time.__name__)
```

```
measure_convergence_time
```

```
ct = measure_convergence_time(df=sol, threshold=0.05, min_stable_duration=5, mode="2D") if not
    sol.empty else np.nan
```

```
print(f"Convergence time (2D error below 5 cm for 5 epochs): {ct} min")
if not sol.empty:
    print(f"Filter convergence time ct_min: {sol['ct_min'].iloc[-1]:.1f} min")
```

```
Convergence time (2D error below 5 cm for 5 epochs): 18.5 min
Filter convergence time ct_min: 37.0 min
```

With `viz.plot_positioning_series`, you can create a default plot of the positioning results and estimated parameters: one panel per available column (ENU errors, receiver clock, ISB, ZTD and the AR diagnostics). Additionally, you can mark the convergence time on the ENU panels with `convergence_time=`, given in minutes, as returned by `measure_convergence_time()`.

```
if not sol.empty:
    fig = viz.plot_positioning_series(
        sol,
        title=f"Combined PPP positioning results for {NAME}",
        color="black",
        convergence_time=None if np.isnan(ct) else ct,
    )
```

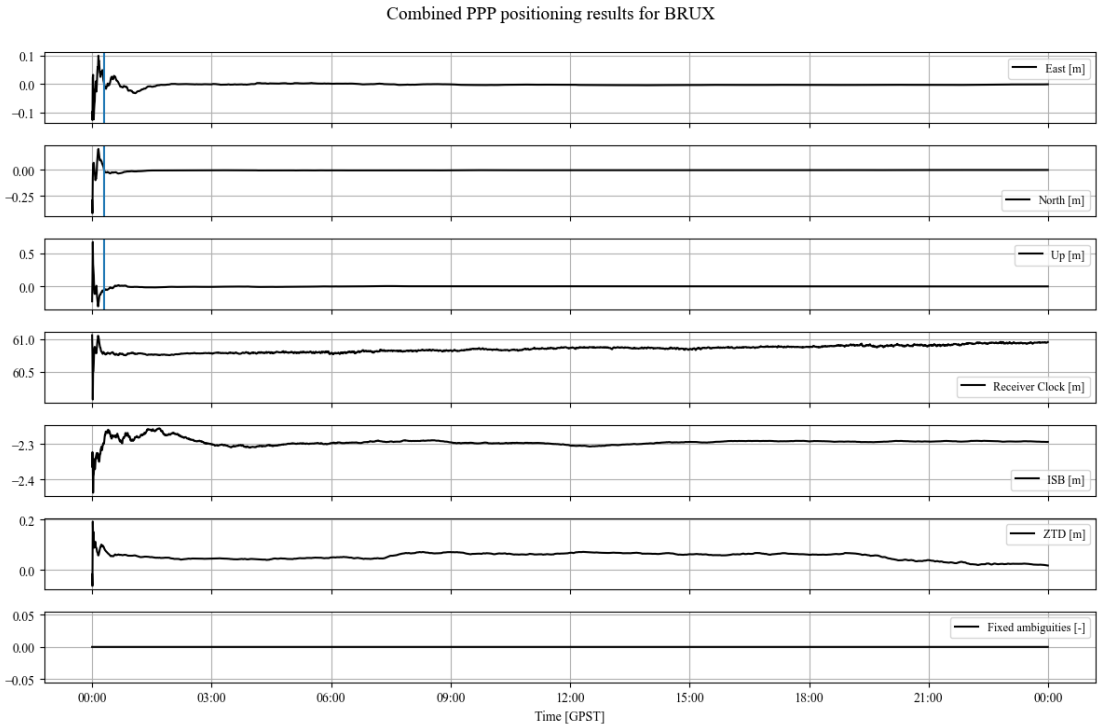


Figure 2.3: Combined PPP positioning errors and estimated parameters for the full-day tutorial run.

Even over a full day, read the plot as a single-station example rather than a final accuracy statement. For a real PPP assessment, process several days and stations and compare convergence, ENU errors, residuals, and the stability of ZTD/clock estimates. The *Fixed ambiguities* panel stays at zero because ambiguity resolution is disabled in this example (`pppar_enabled=False`); PPP-AR is discussed at the end of the notebook.

2.2.8 Example 2 - Undifferenced & Uncombined PPP

Let's look at another example. This time, we use the undifferenced and uncombined approach. In this approach, we do not eliminate the ionosphere by forming IF combinations. Instead, code and phase observations on multiple frequencies enter the filter separately, and the ionosphere can be estimated explicitly for each satellite (Glaner 2022; Schönemann 2014).

GNX-py can enrich this system with ionospheric constraints. These constraints are pseudo-observations derived from an ionosphere model such as GIM. They help the filter in the early phase by telling it what slant ionospheric delay is plausible, while still allowing the measurements to dominate later.

In current `PPPConfig`, this is controlled mainly by:

- `positioning_mode="uncombined"`
- `ionosphere_model="gim"` or another supported model
- `use_iono_constr=True`
- `use_iono_rms`, `sigma_iono_0`, `sigma_iono_end`, `t_end`

Two details of the constrained routes matter for the results. First, a GIM gives the slant ionosphere in the datum of the satellite DCBs printed in its IONEX header, while the slant-ionosphere states of the filter absorb the code-bias datum of the bias product; GNX-py therefore refers the GIM constraint to the product's datum (with the IONEX DCBs of the same two signals, else a DSB product in `dcb_path`). Second, with the slant ionosphere pinned by the constraint every code observation acts on the geometry, so the constrained routes weight the code with $c_{sys}/\sin e$ per system (see the weight table above).

In the UDUC approach our system has more parameters. For each satellite we may carry frequency-specific phase ambiguities. We may also carry ionosphere states and receiver/signal bias parameters. The

bias handling depends on the selected products: OSB, DCB, and DSB products are not interchangeable bookkeeping labels; they define the datum of the observation model.

Current GNX-py supports uncombined single-system and mixed-system routes. Mixed no-constraint uncombined PPP is powerful, but more sensitive to bias/product consistency. For teaching, we run the same full-day GPS+Galileo sample and compare it with the combined solution.

```
config = make_ppp_config(
    positioning_mode="uncombined",
    time_limit=UDUC_TIME_LIMIT,
    ionosphere_model="gim",
    use_iono_constr=True,
    use_iono_rms=False,
    pppar_enabled=False,
)
```

```
if RUN_PPP_EXAMPLES:
    controller = gnx.PPPSession(config=config)
    with contextlib.redirect_stdout(io.StringIO()) as uncombined_log:
        uncombined_results = controller.run()
    for line in uncombined_log.getvalue().splitlines()[:6]:
        print(line)
else:
    uncombined_results = None
    print("Set RUN_PPP_EXAMPLES=True to execute the uncombined PPP example.")
```

```
<repo>/gnx_py/ppp/config.py:1135: RuntimeWarning: SP3 files disagree on the position at an
    epoch both carry (more than 1 m): C11 2024-02-04 00:00:00 (14.61 m); that position node is
    not used.
  _, positions = lagrange_reception_interp(obs=obs, sp3_df=sp3_df, degree=degree, owner=self.
    config)


<repo>/gnx_py/ppp/config.py:338: UserWarning: attitude turn guard ('auto'): 388 Galileo
    observations of 3 satellites (E03, E05, E09) near noon/midnight turns at small |beta| are
    dropped and their ambiguities restart after the turn, because the clocks of analysis
    centres can differ from the attitude laws by a wind-up cycle there. attitude_turn_guard='
    off' keeps them with the attitude laws
  crd_by_sys[sys] = self._preprocess(


<repo>/gnx_py/ppp/config.py:338: UserWarning: attitude turn guard ('auto'): 84 GPS observations
     of 1 satellite (G04) near noon/midnight turns at small |beta| are dropped and their
    ambiguities restart after the turn, because the clocks of analysis centres can differ from
    the attitude laws by a wind-up cycle there. attitude_turn_guard='off' keeps them with the
    attitude laws
  crd_by_sys[sys] = self._preprocess(
```

```
if uncombined_results is not None:
    unc_sol = uncombined_results.solution.copy()
else:
    unc_sol = pd.DataFrame()
```

```
print("Notebook keeps uncombined results in memory. Use unc_sol.to_csv(...) in your own
    analysis script if you want to save them.")
```

```
Notebook keeps uncombined results in memory. Use unc_sol.to_csv(...) in your own analysis
    script if you want to save them.
```

```
if not unc_sol.empty:
    viz.df_tail(unc_sol, nrows=5, ncols=12)
```

time	de	dn	du
2024-02-04 23:57:30	-0.000	-0.005	-0.001
2024-02-04 23:58:00	-0.000	-0.005	-0.001
2024-02-04 23:58:30	-0.000	-0.005	-0.001
2024-02-04 23:59:00	-0.000	-0.005	-0.001
2024-02-04 23:59:30	-0.000	-0.005	-0.001

PDF preview: 8 column(s) omitted from this wide table.

The uncombined result exposes states that do not appear in the combined IF solution. In particular, columns related to receiver/signal code biases may appear, often with names such as `dcb_*`. In the current implementation, the filter uses a datum convention: one code datum may be held or strongly constrained, and other signal biases are estimated relative to it. Phase hardware delays are mostly absorbed by float ambiguities.

Do not compare these bias columns as if they were universal physical constants. Their interpretation depends on the selected observations, products, and datum policy.

```
unc_ct = measure_convergence_time(df=unc_sol, threshold=0.05, min_stable_duration=5, mode="2D")
     if not unc_sol.empty else np.nan
print(f"Convergence time: {unc_ct} min")
```

```
Convergence time: 16.0 min
```

```
if not unc_sol.empty:
    fig = viz.plot_positioning_series(
        unc_sol,
        title=f"Uncombined PPP positioning results for {NAME}",
        color="black",
        convergence_time=None if np.isnan(unc_ct) else unc_ct,
    )
```

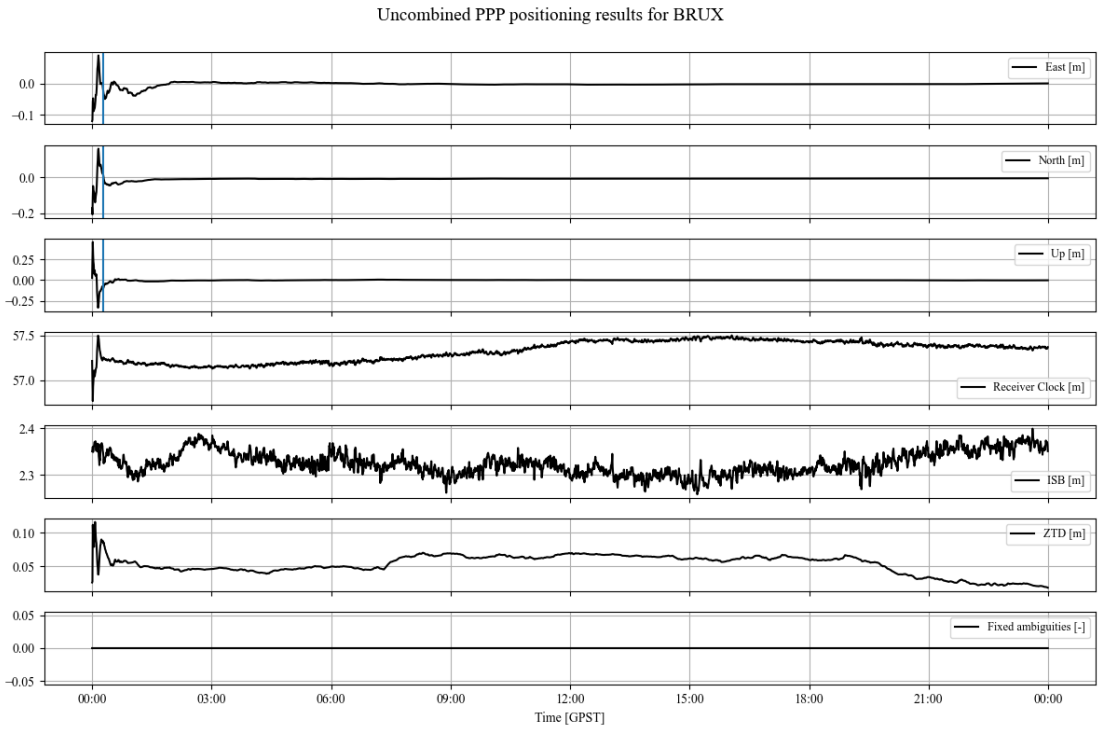


Figure 2.4: Uncombined PPP positioning errors and estimated parameters for the full-day tutorial run.

Let's take a look at the residuals. Unlike the IF approach, the uncombined route has separate residual columns for each code and phase signal. The route of this example (`PPPFilterMultiGNSSIonConst`:

GPS+Galileo, dual frequency, GIM constraints) stores in `vc1`, `vl1`, `vc2`, `vl2` the prefit residuals (innovations) $\mathbf{y}_k$ of the predicted state, taken after the receiver-clock prior, not post-fit residuals; the other uncombined routes store the post-fit residuals in `postfit_code_res_<band>` and `postfit_phase_res_<band>` (see `gnx_py/ppp/README.md`). Innovations also carry the prediction error of the state, so they are larger than the post-fit phase residuals of the combined example: in this run the phase innovations have a standard deviation of 0.056 m (L1) and 0.068 m (L2), and part of that is common to all satellites of an epoch (removing the mean of each epoch leaves 0.029 m and 0.044 m). Compare residuals of different routes only when they are of the same kind. Below we preview these columns, summarise their spread per signal after the same trimming as in the plot (the first epochs of the session and of every satellite arc carry the convergence transient), and plot the phase residuals with `viz.plot_uduc_residuals`, which understands this two-frequency layout and draws one panel per band. For a single band, `viz.plot_residuals` also works on this frame: it falls back to `vl1` (phase) or `vc1` (code).

```
residuals = uncombined_results.residuals_gps.copy() if uncombined_results is not None and
    uncombined_results.residuals_gps is not None else pd.DataFrame()
```

```
if not residuals.empty:
    residual_cols = [c for c in residuals.columns if c.startswith(("vc", "vl"))]
    viz.df_tail(residuals[residual_cols], nrows=5, ncols=8)
```

sv	time	vc1	vl1	vc2	vl2
G17_2_1	2024-02-04 23:59:30	0.131	-0.035	-0.013	-0.030
G19_2_1	2024-02-04 23:59:30	-0.086	-0.050	-0.130	-0.043
G21_1_1	2024-02-04 23:59:30	0.280	-0.064	0.811	-0.105
G28_2_1	2024-02-04 23:59:30	0.016	-0.038	0.033	-0.023
G31_2_1	2024-02-04 23:59:30	-0.035	-0.024	-0.261	0.010

```
if not residuals.empty:
    # Same trimming as the plot below: cut_init_time=3, skip_obs=2.
    band_values = residuals_after_cut(residuals, residual_cols, cut_init_epochs=3, skip_obs=2)
    band_summary = pd.DataFrame(
        {
            "std_m": band_values.std(),
            "rms_m": np.sqrt((band_values**2).mean()),
            "n_obs": band_values.count(),
        }
    ).rename_axis("column")
    viz.df_head(band_summary, nrows=len(band_summary), ncols=4, floatfmt=".4f")
```

column	std_m	rms_m	n_obs
vc1	0.2566	0.2566	25853
vl1	0.0560	0.0563	25853
vc2	0.2742	0.2742	25853
vl2	0.0682	0.0684	25853

```
if not residuals.empty:
    fig = viz.plot_uduc_residuals(residuals, obs_type="phase", x_axis="time", skip_obs=2,
    cut_init_time=3, mode="L1L2")
```

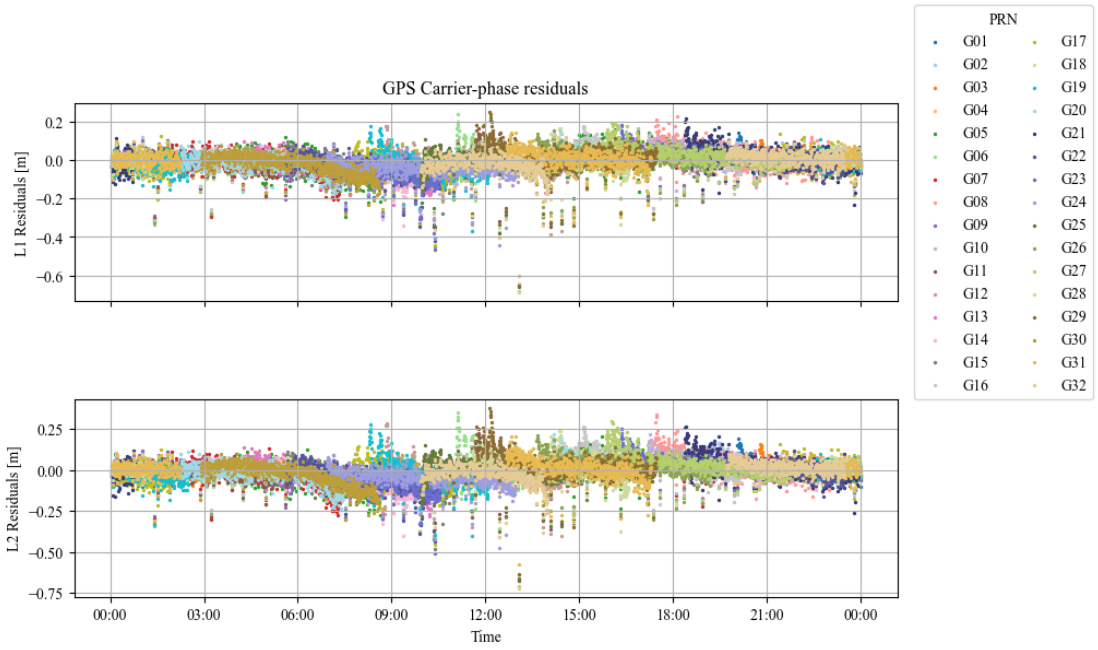


Figure 2.5: Uncombined PPP phase innovations (prefit residuals) of the GPS observations, one panel per band.

2.3 PPP-AR: ambiguity resolution

PPP float treats carrier-phase ambiguities as real-valued states. This is robust, but convergence can be slow because the filter must estimate ambiguities together with coordinates, clock, troposphere, and sometimes ionosphere/bias states.

PPP ambiguity resolution tries to use the integer nature of carrier-phase ambiguities. In a simplified view, the filter has a float ambiguity vector $\hat{\mathbf{a}}$ and covariance $\mathbf{Q}_a$. Integer least-squares searches for the integer vector $\mathbf{n}$ that best explains the float solution:

$$\hat{\mathbf{n}} = \arg\min_{\mathbf{n}\in\mathbb{Z}^m} (\hat{\mathbf{a}} - \mathbf{n})^T \mathbf{Q}_a^{-1} (\hat{\mathbf{a}} - \mathbf{n})$$

LAMBDA is a common method for making this integer search efficient. The ratio test compares the best and second-best integer candidates. A high ratio means the best candidate is much better separated from the next candidate. A low ratio threshold gives more fixes, but increases the risk of false fixes. A high threshold gives fewer fixes, but usually safer ones.

Important PPP-AR ideas:

- **Float vs fixed**: float ambiguities are real-valued; fixed ambiguities are constrained toward integer values.
- **Ratio test**: protects against ambiguous integer candidates.
- **Lock time / arc age**: avoids fixing ambiguities too early after a satellite rises or after a cycle slip.
- **Partial fixing**: allows fixing a mature subset when the full ambiguity set is not yet safe.
- **Hold constraint**: after a fix is accepted, a soft constraint can keep the ambiguity close to the fixed integer.
- **False fix risk**: an incorrect integer constraint can improve formal precision while degrading the actual position.
- **Combined IF caution**: ionosphere-free ambiguities are combinations of original ambiguities and biases; they are integers only together with the wide lane, which the combined filters do not separate, so the combined filters of GNX-py fix no ambiguity: with `pppar_enabled=True` they warn and return the float solution.

- **What is fixed**: the uncombined routes fix the ambiguities. With two frequencies they fix the wide lane by rounding and then the narrow lane with LAMBDA; with one frequency (only with ionospheric constraints, `use_iono_constr=True`) the ambiguities are searched with LAMBDA directly. Single-frequency PPP without constraints, whose code/phase average has no integer ambiguity, stays float and warns, like the combined filters. Candidates need phase biases (phase OSB) from the same solution as the orbits and clocks; a phase flagged as a possible half-cycle ambiguity (RINEX LLI bit 1) is not fixed while the flag is set, and the arc age starts at 0 on the first epoch of every arc.

2.3.1 PPP-AR parameters in `PPPConfig`

Parameter	Meaning	Intuition and safe use
`pppar_enabled`	Enables AR attempts where the selected filter supports them	Keep `False` for baseline validation; turn on only after float PPP is stable
`pppar_warmup_epochs`	Number of epochs before AR attempts start	Larger values reduce early false-fix risk; default `60` is conservative
`pppar_min_ambiguities`	Minimum number of ambiguity candidates for a full fix	Too low can over-trust weak geometry; typical safe value starts around `4`
`pppar_ratio_threshold`	LAMBDA ratio-test threshold	Lower gives more fixes but more false-fix risk; higher is safer but may fix rarely
`pppar_constraint_sigma_cycles`	Soft hold sigma for accepted fixes, in cycles	Smaller means stronger hold; too small can make a false fix more damaging
`pppar_constraint_sigma_floor_cycles`	Lower bound for hold sigma	Prevents the hold from becoming unrealistically strong; `None` disables the floor
`pppar_min_lock_epochs`	Minimum arc age before a candidate can be fixed	Useful when cycle slips or short arcs are common
`pppar_partial_fixing_enabled`	Allows mature subsets to be fixed (default `True`)	Can increase useful fixes, but requires careful diagnostics
`pppar_partial_min_ambiguities`	Minimum subset size for partial fixing	Keep at least several ambiguities; `None` follows full-fix policy
`pppar_combined_if_ar_enabled`	Gate for combined ionosphere-free AR	The combined filters fix no ambiguity even with this gate; keep `False`

Current version note: GNX-py also has supporting AR fields such as candidate elevation and wide-lane gates. The table above focuses on the parameters most useful in a tutorial experiment.

2.3.2 Lightweight PPP-AR experiment

The next cell runs the full-day uncombined PPP (2880 epochs) three times:

1. float PPP, with AR disabled,
2. AR with a permissive ratio threshold,
3. AR with a stricter ratio threshold and partial fixing enabled.

This is not a benchmark. It is a small diagnostic experiment showing how AR settings change the number of accepted fixes and the resulting ENU RMS for one station and one day. For a scientific conclusion, repeat this over more days and independent stations/products. PPP-AR needs the phase OSBs from the same solution as the orbits and clocks: with mixed products GNX-py keeps the float solution and warns (override: `pppar_allow_mixed_products=True`). The experiment therefore uses the matched GRG orbits, clocks and OSB from `sample_data` instead of the CODE products used above.

```
ar_experiment_configs = [
    (
        "float",
```

```
        dict(pppar_enabled=False),
    ),
    (
        "AR permissive ratio",
        dict(
            pppar_enabled=True,
            pppar_warmup_epochs=2,
            pppar_min_ambiguities=4,
            pppar_ratio_threshold=1.2,
            pppar_min_lock_epochs=1,
            pppar_partial_fixing_enabled=False,
        ),
    ),
    (
        "AR stricter ratio + partial",
        dict(
            pppar_enabled=True,
            pppar_warmup_epochs=20,
            pppar_min_ambiguities=2,
            pppar_ratio_threshold=3.0,
            pppar_min_lock_epochs=10,
            pppar_partial_fixing_enabled=True,
            pppar_partial_min_ambiguities=2,
        ),
    ),
]

## PPP-AR: phase OSBs from the same solution as the orbits and clocks (GRG)
AR_SP3_LIST = [SAMPLE_DATA / f"GRG0MGXFIN_202403{day}0000_01D_05M_ORB.SP3" for day in (4, 5, 6)
    ]
AR_BIAS_PATH = SAMPLE_DATA / "GRG0MGXFIN_20240350000_01D_01D_OSB.BIA"

ar_rows = []
ar_solutions = {}

if RUN_PPP_EXAMPLES:
    for label, ar_kwargs in ar_experiment_configs:
        ar_config = make_ppp_config(
            positioning_mode="uncombined",
            time_limit=UDUC_TIME_LIMIT,
            ionosphere_model="gim",
            use_iono_constr=True,
            use_iono_rms=False,
            sp3_path=AR_SP3_LIST,
            dcb_path=AR_BIAS_PATH,
            **ar_kwargs,
        )
        with contextlib.redirect_stdout(io.StringIO()):
            ar_result = gnx.PPPSession(ar_config).run()
        ar_sol = ar_result.solution.copy()
        ar_solutions[label] = ar_sol
        late = ar_sol[ar_sol.index >= ar_sol.index[0] + pd.Timedelta(hours=1)]  # after the
    first hour
        ar_rows.append(
            {
                "case": label,
                "ratio_threshold": ar_kwargs.get("pppar_ratio_threshold", np.nan),
                "partial_fixing": ar_kwargs.get("pppar_partial_fixing_enabled", False),
                "epochs": len(ar_sol),
                "fixed_epochs": int((ar_sol.get("ar_fixed", 0) > 0).sum()) if "ar_fixed" in
    ar_sol else 0,
                "fixed_ambiguities_total": int(ar_sol["ar_fixed"].sum()) if "ar_fixed" in
    ar_sol else 0,
```

```
                "max_ratio": float(ar_sol["ar_ratio"].max()) if "ar_ratio" in ar_sol and ar_sol["ar_ratio"].notna().any() else np.nan,
                "rms_2d_m": float(np.sqrt((ar_sol["de"] ** 2 + ar_sol["dn"] ** 2).mean())),
                "rms_3d_m": float(np.sqrt((ar_sol["de"] ** 2 + ar_sol["dn"] ** 2 + ar_sol["du"] ** 2).mean())),
                "rms_2d_after_1h_m": float(np.sqrt((late["de"] ** 2 + late["dn"] ** 2).mean())),
                "rms_3d_after_1h_m": float(np.sqrt((late["de"] ** 2 + late["dn"] ** 2 + late["du"] ** 2).mean())),
            }
        )

    ar_summary = pd.DataFrame(ar_rows)
    # "epochs" is the same for every case (see the text above); drop it so the preview fits 8 columns.
    after_1h = ["rms_2d_after_1h_m", "rms_3d_after_1h_m"]
    viz.df_head(ar_summary.drop(columns=["epochs", *after_1h]), nrows=len(ar_summary), ncols=8, floatfmt=".4g")
    viz.df_head(ar_summary[["case", *after_1h]], nrows=len(ar_summary), ncols=3, floatfmt=".4f")
else:
    ar_summary = pd.DataFrame()
    print("Set RUN_PPP_EXAMPLES=True to run the lightweight PPP-AR experiment.")
```

```
<repo>/gnx_py/models/antex.py:852: AntexVersionWarning: ANTEX IGS20_2437 (igs20.atx) differs from the antenna model of the orbit product (PCV:IGS20_2290): calibrations changed after week 2290 are not those the product was computed with
  vnote = check_version(atx, label)


<repo>/gnx_py/models/antex.py:852: AntexVersionWarning: ANTEX IGS20_2437 (igs20.atx) differs from the antenna model of the orbit product (PCV:IGS20_2290): calibrations changed after week 2290 are not those the product was computed with
  vnote = check_version(atx, label)


<repo>/gnx_py/models/antex.py:852: AntexVersionWarning: ANTEX IGS20_2437 (igs20.atx) differs from the antenna model of the orbit product (PCV:IGS20_2290): calibrations changed after week 2290 are not those the product was computed with
  vnote = check_version(atx, label)
```

case	ratio threshold	partial fixing	fixed_epochs	fixed ambiguities total	max_ratio	rms_2d_m	rms_3d_m
float	nan	False	0	0	nan	0.0167	0.03124
AR permissive ratio	1.2	False	2623	23228	416.4	0.01604	0.03096
AR stricter ratio + partial	3	True	2640	23503	7224	0.0161	0.03101

case	rms_2d_after 1h_m	rms_3d_after 1h_m
float	0.0075	0.0081
AR permissive ratio	0.0060	0.0071
AR stricter ratio + partial	0.0060	0.0071

The expected pattern is not always “more fixing means better position”. A permissive ratio threshold usually accepts more fixes. That may improve horizontal RMS, but it can also increase vertical error or hide a false-fix problem. A stricter ratio threshold can reject most or all fixes, which looks conservative but may be appropriate if the ambiguity separation is weak. In this full-day run the permissive threshold fixes 2623 of 2880 epochs and the stricter threshold with partial fixing 2640: partial fixing and the smaller minimum number of ambiguities make up for the higher ratio threshold. The day RMS hardly changes (2D 0.017, 0.016 and 0.016 m; 3D 0.031 m in all three cases) because it is dominated by the first minutes before the float solution converges. After the first hour the fixed solutions are better by one to two millimetres (3D RMS 0.0071 m against 0.0081 m float, 2D 0.0060 m against 0.0075 m): with matched

products the fixed and float positions agree to a few millimetres, and the fix sharpens the horizontal position. The three runs also print a warning that the ANTEX file of `sample_data` (IGS20 week 2437) is newer than the antenna model the GRG products were computed with (IGS20_2290); calibrations changed after that week are not those of the product.

This is why PPP-AR should always be interpreted together with diagnostics: `ar_fixed`, `ar_ratio`, `ar_ok`, `ar_last_reason`, `ar_partial_groups`, and the `ar_reason_*` counters. If the AR configuration improves only the formal number of fixes but worsens ENU errors or residuals, treat it as a warning sign.

As I mentioned in the Introduction notebook, working in GNX-py mostly involves creating a configuration class and passing it to the session class. PPP follows the same pattern, but the scientific choices behind the configuration are more consequential: combined vs uncombined routing, ionospheric constraints, bias products, trace diagnostics, and AR settings all change what the filter estimates. I encourage you to try other positioning modes and frequencies after you understand these assumptions.

Chapter 3

Orbit and SIS Analysis

3.1 GNX Orbit module

The `orbits` module of the GNX-py library allows us to compare different GNSS orbit and clock products. In this tutorial, I will first discuss the ideas behind the comparison and then show how the current GNX-py API performs the same workflow with `SISConfig` and `SISSession`. (`SISSession` was called `SISController` in earlier versions; the old name still works, but it emits a `DeprecationWarning`.)

The core task is simple to state: we generate or interpolate satellite coordinates and clocks from two sources, align them on common satellite/epoch pairs, and inspect their differences. One product can be treated as the reference, for example a precise SP3 orbit, while the other can be a broadcast navigation message. The output is not only a Cartesian difference. GNX-py also expresses the orbit difference in the local satellite frame: radial, along-track and cross-track. This is much more informative than raw `dx`, `dy`, `dz`, because different components have different physical meaning and different impact on a user range.

The same comparison also produces Signal-in-Space Error / Signal-in-Space Range Error style metrics. In Galileo documentation we usually meet the term SISE/SISRE (European Union Agency for the Space Programme 2021); in GPS documentation the closely related term is URE (National Executive Committee for Space-Based Positioning, Navigation, and Timing 2023). In both cases, the idea is to combine orbit and clock differences into one range-like quantity, with appropriate weights for the geometry.

Current version note. The current GNX-py implementation supports GPS, Galileo and BeiDou in this SIS/SISRE workflow (GLONASS and QZSS only in SP3-vs-SP3 comparisons, without antenna offsets and group delays). The validated BeiDou path is legacy BDS `B1IB3I` for MEO/IGSO satellites; BeiDou GEO satellites are skipped. Modern BDS modes such as `B1C`, `B2a` and `B2b` are guarded as validation/experimental paths because they require RINEX 4 CNAV/ISC information and clock-consistent OSB/DSB products. We will return to this later, because for BeiDou the clock datum is often the difference between a useful result and a misleading one.

This notebook keeps the style of the previous tutorials: first the intuition and equations, then a small executable example, and only after that interpretation of the result. The examples use `sample_data/` from the repository and avoid local absolute paths.

3.1.1 Algorithms and methods

When implementing methods and algorithms, I relied on information described in (Montenbruck et al. 2018, 2015). The processing in `SISSession` can be summarized in the following steps:

- read the broadcast navigation file (RINEX 3 or 4; RINEX 2 for GPS only) or the SP3 files with the GNX-py readers (`gnx_py.io.rinex_nav`, `gnx_py.io.sp3_reader`),
- select the broadcast record valid at each epoch and evaluate the broadcast coordinates/clocks, or interpolate the SP3 product (Chebyshev interpolation of the positions, extended with the adjacent-day files, and linear interpolation of the clocks between two valid clock nodes),

- optionally apply satellite antenna phase-center offsets to the precise orbit, in the body frame given by the satellite attitude,
- align the two products on common satellites and epochs, and leave out the rows of a satellite whose PRN was reassigned during the run,
- compute Cartesian differences between the products,
- create a local satellite frame from the orbit of the first product (`orb_path_0`),
- convert Cartesian differences into radial / along-track / cross-track components,
- remove the common clock datum of every epoch (for BeiDou separately for each satellite generation),
- combine orbit, clock and group-delay terms into SIS/SISRE quantities,
- flag eclipse periods and, for BeiDou, annotate the clock convention of the comparison,
- return a dataframe that can be inspected, plotted and summarized.

I will now discuss the selection of the broadcast record, the coordinate part, and then the clock and bias part, because clock conventions and biases deserve separate attention.

3.1.2 Broadcast ephemeris selection

A navigation file holds several records for every satellite, each with its own reference time of ephemeris (Toe). For every epoch GNX-py uses the usual post-processing rule: the record with the nearest Toe within the validity interval of the system.

system	validity around Toe	additional rule
GPS	2 h	without a record within 2 h, the nearest record of any age
Galileo	3 h	a record already transmitted at the epoch is preferred
BeiDou	2 h	-

The Galileo rule needs a short comment. A Galileo record is transmitted some 11-12 minutes after its Toe (`TransTime` in the navigation file), so the record with the nearest Toe is often one that no receiver had at that epoch yet. Within the 3 h window GNX-py therefore takes the nearest Toe among the records whose transmission time is not later than the epoch (a record without a known transmission time counts as transmitted from its Toe), and falls back to the nearest Toe only when no transmitted record exists. GPS records are transmitted well before their Toe, so for GPS the nearest Toe is already a transmitted record. We will check both statements on the data in Example 1.

GPS and BeiDou records with a non-zero health value are not used, whatever the signal mode. For Galileo the health bits of the signals of the selected mode decide: epochs at which the satellite, or the record selected for the epoch, is flagged unhealthy for those signals are left out. A satellite left without a usable record is named in a warning.

3.1.3 Satellite coordinates and PCO

Let us consider a comparison between broadcast and precise orbits. Suppose we want to investigate how accurate the broadcast orbit and clock are in relation to a precise final product. The coordinates obtained from a navigation message are referenced to a satellite antenna phase center (APC), while SP3 coordinates are commonly referenced to the satellite center of mass (COM). Directly subtracting APC and COM coordinates would mix geometry with antenna offsets and can easily introduce metre-level differences.

Which phase center does a broadcast orbit refer to? For GPS and BeiDou, the broadcast orbit and clock refer to the phase center of the signal (or signal combination) of the message clock, and the broadcast group delays map that reference to the other signals (National Executive Committee for Space-Based Positioning, Navigation, and Timing 2023). Galileo defines one apparent phase center for all frequencies (European Union Agency for the Space Programme 2021). When a precise product is compared with a navigation message, GNX-py moves the precise orbit to the following reference APC, whatever signal mode is selected:

- GPS: the ionosphere-free L1/L2 combination, IF(L1,L2);

- BeiDou: B3I, the signal to which the BeiDou broadcast clock and every group delay of the message refer;
- Galileo: the APC of the selected mode (for example IF(E1,E5a) for `E1E5a`); only `E5aE5b`, which has no clock of its own in the navigation message, is compared at IF(E1,E5b), the clock pair of I/NAV.

When the reference differs from the selected mode (for example BeiDou `B1IB3I`, whose reference is B3I), the radial offset of the APC of the mode from the reference APC is reported in the column `dR_apc_signal` . This column is information only: it enters neither `dR` nor SISRE, because a user corrects this offset through the group delays.

This is why the ANTEX (`.atx`) file matters. It contains satellite antenna phase-center offsets (PCO). If we want to compare broadcast APC coordinates with precise SP3 coordinates, we apply the satellite PCO to the precise COM coordinates and express both products in a comparable frame. With `apply_satellite_pco=False` GNX-py runs without an ANTEX file (with the default `apply_satellite_pco =True`, `run()` asks for `atx_path` as soon as one product is an SP3 file), but for broadcast-vs-SP3 SIS analysis that result should be treated only as a rough diagnostic.

In `.ATX` files, the PCO is given in a satellite-fixed (body) system. For a satellite in nominal yaw-steering attitude this system can be described through three unit vectors (Teunissen and Montenbruck 2017):

$$\begin{aligned}
\mathbf{e}_z &= -\frac{\mathbf{r}_{sat}}{\|\mathbf{r}_{sat}\|},\\
\hat{\mathbf{s}} &= \frac{\mathbf{r}_{sun}-\mathbf{r}_{sat}}{\|\mathbf{r}_{sun}-\mathbf{r}_{sat}\|},\\
\mathbf{e}_y &= \frac{\mathbf{e}_z\times\hat{\mathbf{s}}}{\|\mathbf{e}_z\times\hat{\mathbf{s}}\|},\\
\mathbf{e}_x &= \mathbf{e}_y\times\mathbf{e}_z.
\end{aligned}$$

Here $\mathbf{r}_{sat}$ is the satellite position vector and $\mathbf{r}_{sun}$ is the Sun position vector. $\mathbf{e}_z$ points to the Earth center (the antenna boresight), $\mathbf{e}_y$ is the solar-panel axis, normal to the Sun-satellite-Earth plane, and $\mathbf{e}_x$ lies in that plane, perpendicular to $\mathbf{e}_z$, on the Sun side, completing the right-handed frame.

Nominal yaw steering cannot be flown exactly near orbit noon and orbit midnight when the Sun is close to the orbital plane: the required yaw rate becomes larger than the satellite can follow, and every satellite block performs its own turn manoeuvre. GNX-py uses one attitude provider, `gnx_py.models.attitude`, for the PCO in SIS and for the satellite antenna terms (and the phase wind-up) of the positioning and TEC sessions. With the default setting it uses the published turn laws of the ANTEX block (GPS IIR, IIF and III, Galileo IOV and FOC, BeiDou BDS-2 and BDS-3), or the ORBEX attitude file given in `attitude_path` where the file comes from the analysis center of the SP3 product; elsewhere the satellite is in yaw steering. `satellite_attitude="nominal"` uses nominal yaw steering everywhere. The module documentation of `gnx_py.models.attitude` lists the laws and their references.

Denoting the satellite center-of-mass coordinates as XYZ_{COM}, the PCO in the body frame as ΔAPC and the rotation matrix with the columns $\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z$ of the body frame as R_{sat}, the antenna phase-center coordinates are

$$XYZ_{APC} = XYZ_{COM} + R_{sat}\cdot\Delta APC.$$

3.1.4 RAC frame and SISRE

After both products are expressed in a comparable coordinate convention, we subtract them and project the difference into the local orbital frame. This tutorial uses the usual RAC notation:

- **R**: radial, from the Earth center to the satellite (positive outward),
- **A**: along-track, approximately the direction of motion,
- **C**: cross-track, normal to the orbital plane.

A clean definition is

$$\mathbf{R} = \frac{\mathbf{r}_{sat}}{\|\mathbf{r}_{sat}\|},$$
$$\mathbf{v}_i = \mathbf{v}_{sat} + \omega_E \times \mathbf{r}_{sat},$$
$$\mathbf{C} = \frac{\mathbf{r}_{sat} \times \mathbf{v}_i}{\|\mathbf{r}_{sat} \times \mathbf{v}_i\|},$$
$$\mathbf{A} = \mathbf{C} \times \mathbf{R}.$$

Here $\mathbf{r}_{sat}$ and $\mathbf{v}_{sat}$ are the Earth-fixed position and velocity of the satellite (of the product `orb_path_0`) and ω_E is the Earth rotation vector. Adding $\omega_E \times \mathbf{r}_{sat}$ turns the Earth-fixed velocity into the inertial one, so that $\mathbf{C}$ is the normal of the inertial orbital plane; with the Earth-fixed velocity alone, $\mathbf{C}$ would tilt by tens of degrees.

The projected differences are then (HofmannWellenhof et al. 2008)

$$\Delta R = \mathbf{R} \cdot \Delta XYZ,$$
$$\Delta A = \mathbf{A} \cdot \Delta XYZ,$$
$$\Delta C = \mathbf{C} \cdot \Delta XYZ.$$

All differences are `orb_path_0` minus `orb_path_1`. With the differences determined in this way, GNX-py computes the SISRE of (Montenbruck et al. 2015, 2018):

$$SISRE = \sqrt{(w_R \cdot dR - dt'')^2 + w_{AC}^2 \cdot (dA^2 + dC^2)},$$
$$dt'' = (dt - \overline{dt}) - (dTGD - \overline{dTGD}),$$

where dt is the clock difference in metres, $dTGD$ the group-delay difference in metres, and $\overline{dt}$, $\overline{dTGD}$ the common clock and group-delay datums of the epoch (columns `dt_mean` and `dTGD_mean`, see the next section). The radial error and the clock error enter the user range with opposite signs, which is why they appear together in the first term. The weights follow from the satellite altitude: they are the projections of the radial and the transverse errors on the line of sight, averaged over the part of the Earth that sees the satellite.

satellites	w_R	w_{AC}^2
GPS	0.98	1/49
Galileo	0.98	1/61
BeiDou MEO	0.98	1/54
BeiDou IGSO	0.99	1/126

The BeiDou orbit type comes from the ANTEX block of the satellite (without an ANTEX file, from its orbit). Expanding the square gives the quadratic form $k_R\,dR^2 + k_t\,dt''^2 + k_{AC}(dA^2 + dC^2) + k_{Rt}\,dR\,dt''$ with $k_R = w_R^2$, $k_t = 1$, $k_{AC} = w_{AC}^2$ and $k_{Rt} = -2w_R$; `SISSession.run(kr, kb, kac, krb)` accepts own coefficients of this form. In the result dataframe, GNX-py reports the components as `dR`, `dA`, `dC`, `dt`, `dTGD`, `sisre`, `sisre_orb` and `sisre_notgd`: `sisre_orb` is $\sqrt{w_R^2 dR^2 + w_{AC}^2(dA^2 + dC^2)}$, and `sisre_notgd` uses the clock term $dt - \overline{dt}$ without the group delays.

One technical detail is worth keeping. The RAC frame needs the satellite velocity. For broadcast orbits this velocity can be generated together with the satellite position. For SP3 products, however, velocity is often not included. GNX-py therefore interpolates positions not only at the requested epoch t_n, but also in a tiny neighbourhood around that epoch, and uses the difference to approximate an instantaneous velocity. This is especially important for SP3-vs-SP3 comparisons, where no broadcast velocity is available.

3.1.5 Satellite clocks, TGD and biases

Satellite clocks need special treatment. Broadcast and precise clock products are not always expressed in exactly the same time-scale or clock-datum convention. A raw clock difference can therefore contain a constellation-wide bias that is not a satellite-specific clock error. A common SIS practice is to remove, for each epoch, the average or median clock difference over the constellation (Montenbruck et al. 2018, 2015). In GNX-py this is controlled by `clock_bias=True` and `clock_bias_function="mean"` or `"median"`. For BeiDou the datum is taken separately for the BDS-2 and the BDS-3 satellites, because the clocks of the two generations have independent datums; we will see this in the BeiDou example.

Group delays and code biases are the second part of the clock story. A broadcast navigation message may contain TGD-like parameters. Precise bias products may contain DCB/DSB pairwise biases or OSB observable-specific biases. They describe related hardware delays, but they are not interchangeable unless the signal pair, sign convention, clock reference and analysis center convention are understood.

GNX-py keeps a `TGD` column in metres on both sides of the comparison, chosen so that the clock minus `TGD` is the same clock on both sides: for GPS the L1 P(Y) clock, for Galileo the E1 clock, for BeiDou the clock of the signal or ionosphere-free combination of the selected mode. For a GPS product the conversion uses the relations

$$\begin{aligned} DSB(obs_1, obs_2) &= OSB(obs_1) - OSB(obs_2), \\ TGD_{GPS} &= k_2 \cdot DSB(C1W, C2W), \\ k_2 = \frac{-f_2^2}{f_1^2 - f_2^2} &= -\frac{1}{\gamma - 1}, \quad \gamma = \frac{f_1^2}{f_2^2}. \end{aligned}$$

This equation is a good mental model for GPS L1/L2, but it should not be copied blindly to every constellation and every signal. For BeiDou, especially, the selected signal mode decides which broadcast TGD/ISC fields are meaningful and whether the precise clock can be compared directly.

Bias products are read only with `compare_dcb=True`. With `compare_dcb=False` (the examples below), GPS and Galileo get a zero `TGD` placeholder on both sides, so `dTGD` is zero and `sisre` equals `sisre_notgd`; BeiDou navigation inputs keep the group delays of the message.

3.1.6 Satellite eclipse periods

The `orbits` module can also flag periods when a satellite is in the Earth's shadow. Entering eclipse can disturb satellite motion or introduce clock behaviour related to thermal changes. This is useful when studying the stability of satellite orbits around eclipse intervals. GNX-py uses the classical geometric condition of a cylindrical shadow (Mervart 1995):

$$\begin{aligned} \cos(\phi) = \frac{\mathbf{r}^{sat} \cdot \mathbf{r}^{sun}}{\|\mathbf{r}^{sat}\| \cdot \|\mathbf{r}^{sun}\|} < 0, \\ \|\mathbf{r}^{sat}\| \cdot \sqrt{1 - \cos^2(\phi)} < a_e. \end{aligned}$$

Here $\mathbf{r}^{sat}$ and $\mathbf{r}^{sun}$ are vectors from the geocentre to the satellite and to the Sun, and a_e is the equatorial radius of Earth (6378.137 km). The first condition puts the satellite behind the Earth, the second one inside the cylinder of radius a_e around the Earth-Sun line. If both inequalities are satisfied, the satellite is geometrically in eclipse: the column `eclipse_raw`. The column `eclipse` extends the flag after the shadow exit by `extension_time` minutes (30 by default, `extend_eclipse=True`), a buffer for the period in which the orbit and the clock may still be disturbed after the shadow exit. In practical SIS analysis, these flags can be used to inspect whether orbit or clock errors behave differently near shadow transitions.

Let us conclude the theoretical introduction here and move on to examples of using GNX-py for orbit analysis.

3.2 Notebook setup

The examples below use the repository `sample_data/` directory. The notebook should work when it is opened from the repository root or from the `tutorial/orbits/` directory. If your IDE starts Jupyter from

another directory, run the setup cell and read the error message: it will ask you to point `REPO_ROOT` to the cloned repository.

GNX-py reads the navigation, SP3 and ANTEX files with its own readers, so no external conversion tool is needed. Figures and table previews come from the GNX-py plotting package `gnx_py.viz` (`viz.plot_sisre`, `viz.plot_sv_stats`, `viz.df_head`). The only local helper is `stat_tools.py`, which computes the per-satellite statistics shown in the tables, while the numerical API stays in `gnx_py.orbits`.

```
from __future__ import annotations

from dataclasses import asdict
from datetime import datetime
from pathlib import Path
import importlib.util
import os
import textwrap
import warnings

import pandas as pd

import gnx_py as gnx
from gnx_py import viz
from gnx_py.models.antex import build_antenna_model, load_antex
from gnx_py.models.attitude import REGIMES
from gnx_py.models.biases import bds_satellite_generations
from gnx_py.orbits import SISConfig, SISSession


def find_repo_root(start: Path) -> Path:
    for candidate in [start.resolve(), *start.resolve().parents]:
        if (candidate / "pyproject.toml").exists() and (candidate / "gnx_py").is_dir():
            return candidate
    raise FileNotFoundError(
        "Run this notebook from inside the cloned GNX-py repository, "
        "or set REPO_ROOT manually to the repository directory."
    )


REPO_ROOT = find_repo_root(Path.cwd())
SAMPLE_DATA = REPO_ROOT / "sample_data"
TUTORIAL_DIR = REPO_ROOT / "tutorial" / "orbits"

try:
    GNX_LOCATION = Path(gnx.__file__).resolve().relative_to(REPO_ROOT)
except ValueError:  # gnx_py installed outside the repository (site-packages)
    GNX_LOCATION = gnx.__file__

print(f"gnx_py imported from: {GNX_LOCATION}")
print(f"sample data: {SAMPLE_DATA.relative_to(REPO_ROOT)}")
```

```
gnx_py imported from: gnx_py/__init__.py
sample data: sample_data
```

```
def load_tutorial_module(module_name: str, module_path: Path):
    spec = importlib.util.spec_from_file_location(module_name, module_path)
    module = importlib.util.module_from_spec(spec)
    if spec.loader is None:
        raise ImportError(f"Cannot load {module_name} from {module_path}")
    spec.loader.exec_module(module)
    return module


stat_tools = load_tutorial_module("orbits_stat_tools", TUTORIAL_DIR / "stat_tools.py")
```

```
orbit_stats = stat_tools.orbit_stats

viz.apply_publication_style(font_scale=1.3, mathtext="cm")
```

```
NAV_PATH = SAMPLE_DATA / "BRDC00IGS_R_20240350000_01D_MN.rnx"
COD_SP3 = SAMPLE_DATA / "COD0MGXFIN_20240350000_01D_05M_ORB.SP3"
COD_SP3_PREV = SAMPLE_DATA / "COD0MGXFIN_20240340000_01D_05M_ORB.SP3"
COD_SP3_NEXT = SAMPLE_DATA / "COD0MGXFIN_20240360000_01D_05M_ORB.SP3"
GRG_SP3 = SAMPLE_DATA / "GRG0MGXFIN_20240350000_01D_05M_ORB.SP3"
GRG_SP3_PREV = SAMPLE_DATA / "GRG0MGXFIN_20240340000_01D_05M_ORB.SP3"
GRG_SP3_NEXT = SAMPLE_DATA / "GRG0MGXFIN_20240360000_01D_05M_ORB.SP3"
ATX_PATH = SAMPLE_DATA / "igs20.atx"
CAS_DSB = SAMPLE_DATA / "CAS1OPSRAP_20240350000_01D_01D_DCB.BIA"
GRG_OSB = SAMPLE_DATA / "GRG0MGXFIN_20240350000_01D_01D_OSB.BIA"
GFZ_OSB = SAMPLE_DATA / "GFZ0MGXRAP_20240350000_01D_01D_OSB.BIA"

SMOKE_TLIM = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 4, 6, 0)]
FULL_DAY_TLIM = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 5, 0, 0)]
INTERVAL_SECONDS = 120

required_paths = [NAV_PATH, COD_SP3, COD_SP3_PREV, COD_SP3_NEXT, GRG_SP3, GRG_SP3_PREV,
    GRG_SP3_NEXT, ATX_PATH]
missing = [path for path in required_paths if not path.exists()]
if missing:
    raise FileNotFoundError("Missing sample files:\n" + "\n".join(str(path.relative_to(
    REPO_ROOT)) for path in missing))
```

3.3 Practical configuration with SISConfig

As with SPP and PPP, orbit comparison is controlled by a configuration object and a session. The current public entry points are `SISConfig` and `SISSession` from `gnx_py.orbits`. Every setting is validated when it is set: an unknown signal mode or a missing file raises an error naming the setting, and settings that must fit together (for example satellite PCO of an SP3 input without an ANTEX file) are checked at the start of `SISSession.run()`.

The parameters you will usually edit are:

parameter	role in the tutorial
`orb_path_0`, `orb_path_1`	the two products to compare; each can be a broadcast navigation file or an SP3 file
`prev_sp3`, `next_sp3`	adjacent-day SP3 files for product 1, useful near day boundaries and for interpolation
`prev_sp3_0`, `next_sp3_0`	adjacent-day SP3 files for product 0 when product 0 is also SP3
`system`	constellation: `"G"` for GPS, `"E"` for Galileo, `"C"` for BeiDou
`gps_mode`, `gal_mode`, `bds_mode`	signal/frequency convention used for clock and bias handling
`interval`	output sampling in seconds
`tlim`	time interval to process; short intervals are best for tutorial runs
`atx_path`	ANTEX file used for satellite PCO correction, BeiDou satellite generations and PRN changes
`compare_dcb`	whether external bias products should be read and compared/converted
`dcb_path_0`, `dcb_path_1`	DCB/DSB/OSB files for the two products when `compare_dcb=True`
`clock_bias`	remove the common clock datum per epoch
`clock_bias_function`	use `"mean"` or `"median"` for the common clock correction

parameter	role in the tutorial
`apply_satellite_pco`	apply satellite PCO to SP3 coordinates
`satellite_attitude`, `attitude_path`	attitude of the PCO step: turn laws (default), `"nominal"` yaw steering, or an ORBEX file
`missing_sat_antenna`	a satellite without an ANTEX entry: `"exclude"` (default, with a warning) or `"raise"`
`apply_eclipse`	flag eclipse periods
`extend_eclipse`, `extension_time`	extend the eclipse flag after the shadow exit (default: by 30 minutes)

A safe starting point is: GPS `L1L2`, Galileo `E1E5a`, and BeiDou `B1IB3I`. For BeiDou, `B1IB3I` is not just another label: it tells GNX-py to use the legacy B1I/B3I ionosphere-free clock convention that is currently the validated path for MEO/IGSO SIS comparison.

```
def run_sis(config: SISConfig):
    """Run SISSession and collect its warnings (each message once) to print them with `print_warnings`."""
    with warnings.catch_warnings(record=True) as captured:
        warnings.simplefilter("always")
        result = SISSession(config=config).run()
    warning_messages = list(dict.fromkeys(short_paths(str(item.message)) for item in captured))
    return result, warning_messages


def short_paths(text: str) -> str:
    return text.replace(str(REPO_ROOT) + os.sep, "")


def print_warnings(messages, width=88, shown=()):
    """Print every message; a message already printed for an earlier run (in `shown`) is only counted."""
    new = [message for message in messages if message not in shown]
    for number, message in enumerate(new, start=1):
        print(textwrap.fill(f"{number}. {message}", width=width, subsequent_indent="   "))
    repeated = len(messages) - len(new)
    if repeated:
        print(f"({repeated} more warning(s) identical to messages printed above)")


def build_broadcast_vs_precise_config(
    *,
    system: str,
    gps_mode: str = "L1L2",
    gal_mode: str = "E1E5a",
    bds_mode: str = "B1IB3I",
    tlim=SMOKE_TLIM,
    interval: int = INTERVAL_SECONDS,
    compare_dcb: bool = False,
    satellite_attitude=None,
):
    return SISConfig(
        orb_path_0=NAV_PATH,
        orb_path_1=COD_SP3,
        prev_sp3=COD_SP3_PREV,
        next_sp3=COD_SP3_NEXT,
        dcb_path_0=CAS_DSB if compare_dcb else None,
        dcb_path_1=CAS_DSB if compare_dcb else None,
        interval=interval,
        gps_mode=gps_mode,
        gal_mode=gal_mode,
        bds_mode=bds_mode,
        atx_path=ATX_PATH,
```

```
        system=system,
        tlim=tlim,
        compare_dcb=compare_dcb,
        clock_bias=True,
        clock_bias_function="mean",
        apply_eclipse=True,
        apply_satellite_pco=True,
        satellite_attitude=satellite_attitude,
    )


def key_columns(df: pd.DataFrame):
    preferred = [
        "dR", "dA", "dC", "dt", "dt_mean", "dTGD", "dTGD_mean",
        "sisre", "sisre_orb", "sisre_notgd", "clock_convention_status",
        "clock_convention_mode", "clock_convention_directly_comparable",
    ]
    return [column for column in preferred if column in df.columns]


def summarize_clock_convention(df: pd.DataFrame) -> pd.DataFrame:
    attrs = {
        key.replace("clock_convention_", ""): value
        for key, value in df.attrs.items()
        if key.startswith("clock_convention_")
    }
    return pd.DataFrame([attrs]) if attrs else pd.DataFrame()
```

3.4 Example 1 - Broadcast and precise products comparison

In the first example we compare broadcast navigation against a precise CODE MGEX SP3 product. This is the classical SIS question: if a receiver uses the broadcast message, how far is that signal-in-space information from a precise reference product?

The short time window below is intentional. It keeps the notebook light and repeatable. For a real validation report you would increase `tlim`, possibly use the whole day, and then save summary tables outside the notebook.

3.4.1 GPS: broadcast vs precise

Let us start with GPS. We select the `L1L2` mode, use the ANTEX file for satellite PCO, and remove the mean constellation clock offset per epoch. The precise SP3 product is treated as the reference, so in the interpretation below "error" means "broadcast minus precise" under this convention.

```
gps_config = build_broadcast_vs_precise_config(system="G", gps_mode="L1L2")
gps_result, gps_warnings = run_sis(gps_config)

print(f"rows={len(gps_result)}")
print(f"satellites={gps_result.index.get_level_values('sv').nunique()}")
print(f"captured warnings={len(gps_warnings)}")
print_warnings(gps_warnings)
viz.df_head(gps_result[key_columns(gps_result)], nrows=4, ncols=10)
```

```
rows=5611
satellites=31
captured warnings=1
1. GPS satellites without a usable broadcast ephemeris (left out): G01 (unhealthy,
   health=63).
```

sv	time	dR	dA	dC	dt	dt_mean	dTGD
G08	2024-02-04 00:00:00	0.231	-0.428	-0.832	-1.314	-0.227	0.000
G08	2024-02-04 00:02:00	0.228	-0.429	-0.829	-1.258	-0.225	0.000
G08	2024-02-04 00:04:00	0.224	-0.432	-0.827	-1.203	-0.224	0.000
G08	2024-02-04 00:06:00	0.220	-0.437	-0.826	-1.181	-0.224	0.000

PDF preview: 2 column(s) omitted from this wide table.

The columns `dR`, `dA` and `dC` are the orbit differences in the radial, along-track and cross-track directions. `dt` is the clock difference expressed in metres. `dt_mean` is the constellation-common clock correction removed at each epoch. `dTGD` is the group-delay / bias difference in metres; in this run (`compare_dcb=False`) it is the zero placeholder described above. The three SIS columns are useful for different questions:

- `sisre` includes orbit, clock and group-delay terms,
- `sisre_orb` keeps only the orbit contribution,
- `sisre_notgd` is a clock/orbit metric without the TGD term (here equal to `sisre`).

When `clock_bias=True`, the raw clock difference is not interpreted directly. We first remove the constellation datum, because otherwise a common offset could dominate the result while saying little about satellite-specific SIS quality.

The only warning of this run tells us that G01 is left out: all its navigation records of the day are flagged unhealthy. GNX-py gives each warning once per run, and `run_sis` collects them, so they do not interrupt the tables.

```
fig = viz.plot_sisre(gps_result)  # the time axis is GPS time, as the SIS epochs
```

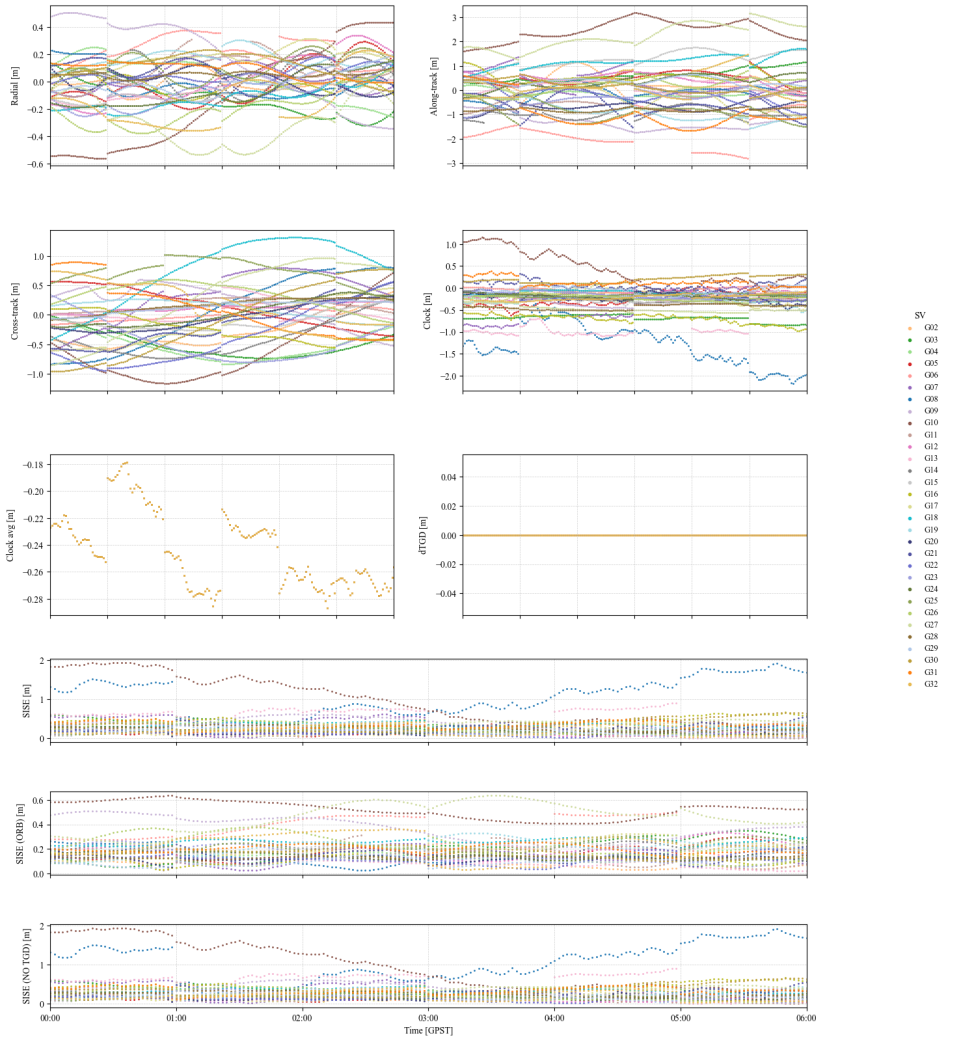


Figure 3.1: GPS broadcast-versus-precise SIS components over the tutorial interval.

The plot separates the geometry and clock pieces. In many SIS analyses the radial and clock components dominate the final range-like metric. Along-track and cross-track errors still matter, but their mapping into user range is weaker and is therefore weighted differently.

```
gps_stats = orbit_stats(
    gps_result,
    columns=["dR", "dA", "dC", "dt", "sisre"],
    stats=("mean", "95%", "rms"),
)
viz.df_head(gps_stats, nrows=8, ncols=10, reset_index=True)
```

sv	dR_mean	dR_95%	dR_rms	dA_mean	dA_95%	dA_rms	dC_mean
G02	-0.016	0.120	0.062	0.302	1.084	0.484	-0.274
G03	-0.126	0.077	0.176	0.620	1.065	0.667	-0.511
G04	0.032	0.243	0.152	-0.132	0.191	0.191	-0.523
G05	-0.004	0.277	0.154	0.488	0.824	0.554	0.143
G06	0.227	0.373	0.248	-1.614	-0.623	1.751	0.030
G07	0.029	0.187	0.101	0.361	1.163	0.569	0.334
G08	-0.019	0.208	0.110	-0.117	0.350	0.327	0.062
G09	0.155	0.502	0.329	-1.309	-0.725	1.342	0.193

PDF preview: 2 column(s) omitted from this wide table.

```
fig = viz.plot_sv_stats(
    gps_stats,
    "sisre_rms",
```

```
    title="GPS broadcast vs precise: SISRE RMS per satellite",
    ylabel="SISRE RMS [m]",
)
```

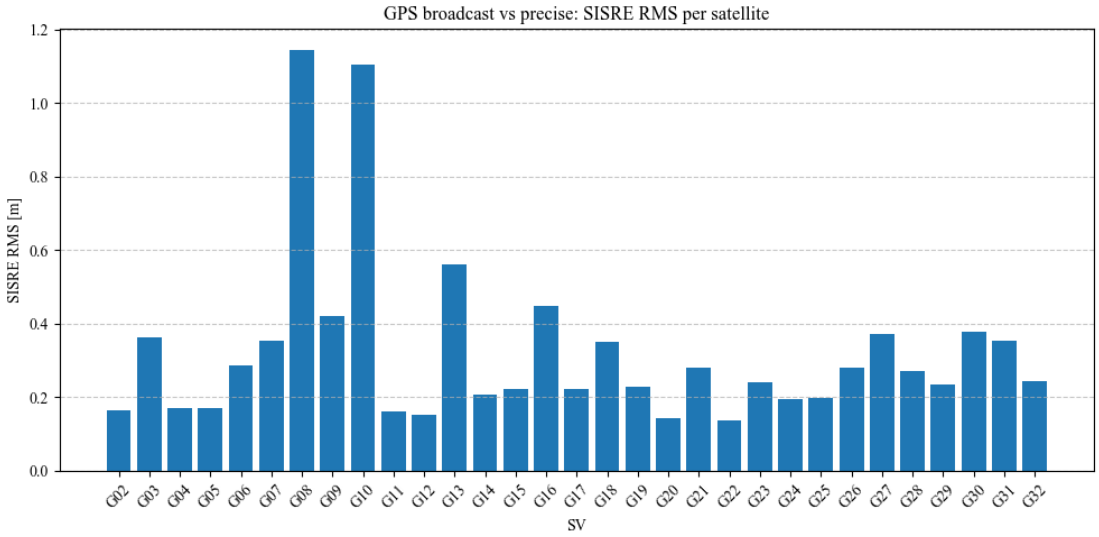


Figure 3.2: Per-satellite GPS SIS statistics used to identify satellite-specific behaviour.

The per-satellite statistics are often more useful than a single global number. A single problematic satellite can be hidden in an all-constellation average. When preparing a validation report, I would usually inspect both: a time series plot to see when something happened, and a per-satellite table to see where it happened.

3.4.2 Satellite attitude during noon and midnight turns

The GPS result carries one more column, `att_regime`: the attitude regime of the precise satellite at every epoch, used when the PCO was rotated from the body frame into the Earth-fixed frame. Code 0 means nominal yaw steering; the other codes are the turn regimes of `gnx_py.models.attitude` (`REGIMES`). Let us count them, and then repeat the run with `satellite_attitude="nominal"` to see how much the turn laws change the comparison.

```
regimes = (
    gps_result["att_regime"].dropna().astype(int).map(REGIMES)
    .value_counts().rename_axis("attitude regime").rename("rows").to_frame()
)
viz.df_head(regimes, reset_index=True)
```

attitude regime	rows
yaw steering	5541
auxiliary Sun	32
shadow	27
catch-up	11

```
nominal_result, nominal_warnings = run_sis(build_broadcast_vs_precise_config(system="G",
    satellite_attitude="nominal"))
print_warnings(nominal_warnings, shown=gps_warnings)

turn_rows = gps_result["att_regime"] != 0
columns = ["dR", "dA", "dC", "sisre"]
attitude_change = (gps_result[columns] - nominal_result[columns].reindex(gps_result.index)).abs
    ()
turn_table = attitude_change[turn_rows].groupby(level="sv").max().add_prefix("max abs change ")
viz.df_head(turn_table, reset_index=True)
```

```
print(f"largest change outside the turns: {attitude_change[~turn_rows].to_numpy().max():.4f} m"
    )
```

```
(1 more warning(s) identical to messages printed above)
```

sv	max abs change dR	max abs change dA	max abs change dC	max abs change sisre
G04	0.000	0.054	0.053	0.002
G09	0.000	0.217	0.210	0.015
G32	0.000	0.396	0.258	0.013

```
largest change outside the turns: 0.0000 m
```

Only three satellites pass a turn in these six hours. The attitude rotates the horizontal part of the PCO, so `dA` and `dC` change by up to about 0.4 m for G32 and about 0.2 m for G09, two GPS IIF satellites whose PCO has an x component of 0.394 m, and only by about 5 cm for G04 (GPS IIIA, x = -0.06 m). `dR` does not change, and SISRE changes by less than 2 cm, because the transverse components have small weights. Outside the turns the two runs are identical.

If the analysis center publishes its attitude in an ORBEX file, it can replace the laws for the satellites the file covers. The ORBEX file is not part of `sample_data`, so the following lines are only an example of the configuration (COD SP3 orbits with the COD attitude of the same day):

```
orbex_config = build_broadcast_vs_precise_config(system="G")
orbex_config.attitude_path = SAMPLE_DATA / "COD0MGXFIN_20240350000_01D_30S_ATT.OBX"  # ORBEX of
    the SP3's analysis center, copied there
orbex_result, orbex_warnings = run_sis(orbex_config)
```

The ORBEX attitude is used only when it comes from the analysis center of the SP3 product; such rows get `att_regime` 8.

3.4.3 Galileo: broadcast vs precise

The same workflow can be repeated for Galileo. The main practical difference is the selected signal/navigation convention. Here we use `E1E5a`, which is the usual safe starting point for this dataset.

```
gal_config = build_broadcast_vs_precise_config(system="E", gal_mode="E1E5a")
gal_result, gal_warnings = run_sis(gal_config)

print(f"rows={len(gal_result)}")
print(f"satellites={gal_result.index.get_level_values('sv').nunique()}")
print(f"captured warnings={len(gal_warnings)}")
print_warnings(gal_warnings)
viz.df_head(gal_result[key_columns(gal_result)], nrows=4, ncols=10)
```

```
rows=4163
satellites=23
captured warnings=0
```

sv	time	dR	dA	dC	dt	dt_mean	dTGD
E07	2024-02-04 00:00:00	0.035	0.019	0.190	-0.117	0.144	0.000
E07	2024-02-04 00:02:00	0.043	0.019	0.194	-0.120	0.141	0.000
E07	2024-02-04 00:04:00	0.025	0.086	0.175	-0.127	0.145	0.000
E07	2024-02-04 00:06:00	0.019	0.087	0.179	-0.131	0.145	0.000

PDF preview: 2 column(s) omitted from this wide table.

```
gal_stats = orbit_stats(
    gal_result,
    columns=["dR", "dA", "dC", "dt", "sisre"],
    stats=("mean", "95%", "rms"),
)
viz.df_head(gal_stats, nrows=8, ncols=10, reset_index=True)
```

sv	dR_mean	dR_95%	dR_rms	dA_mean	dA_95%	dA_rms	dC_mean
E02	0.319	0.387	0.321	-0.214	-0.045	0.241	0.089
E03	0.186	0.253	0.188	0.281	0.367	0.291	-0.123
E04	0.156	0.352	0.210	0.003	0.123	0.063	-0.004
E05	0.291	0.341	0.294	0.232	0.366	0.249	-0.128
E07	0.077	0.252	0.125	0.147	0.247	0.160	0.153
E08	0.256	0.408	0.272	0.066	0.400	0.190	0.065
E09	0.217	0.309	0.232	0.042	0.190	0.084	-0.012
E10	0.315	0.473	0.332	-0.205	-0.004	0.227	-0.144

PDF preview: 2 column(s) omitted from this wide table.

3.4.4 Galileo broadcast record selection

Let us check the record-selection rule on the two results. The broadcast navigation fields, among them `Toe` and `TransTime` (seconds of the week), are passed through to the result frame of a navigation input. They are not part of the guaranteed output columns, but they are convenient here: for every row we can compute how old the selected record is and whether it had already been transmitted at the epoch.

```
def week_minutes(seconds):
    # a difference of two seconds-of-week values, wrapped into -1/2 ... +1/2 week, in minutes
    return ((seconds + 302400) % 604800 - 302400) / 60


def record_selection(result: pd.DataFrame) -> dict:
    epochs = result.index.get_level_values("time")
    seconds_of_week = ((epochs - pd.Timestamp("1980-01-06")).total_seconds() % 604800).to_numpy()
    since_toe = week_minutes(seconds_of_week - result["Toe"].to_numpy(float))
    since_transmission = week_minutes(seconds_of_week - result["TransTime"].to_numpy(float))
    transmission_after_toe = week_minutes(result["TransTime"].to_numpy(float) - result["Toe"].to_numpy(float))
    return {
        "median epoch - Toe (min)": pd.Series(since_toe).median(),
        "rows with Toe after the epoch (%)": 100 * (since_toe < 0).mean(),
        "median TransTime - Toe (min)": pd.Series(transmission_after_toe).median(),
        "rows with a record already transmitted (%)": 100 * (since_transmission >= 0).mean(),
    }, since_transmission < 0


gps_selection, _ = record_selection(gps_result)
gal_selection, gal_not_transmitted = record_selection(gal_result)
viz.df_head(pd.DataFrame({"GPS": gps_selection, "Galileo": gal_selection}).T.rename_axis("system"), reset_index=True)

late = gal_result[gal_not_transmitted].index
print(f"Galileo rows with a record not yet transmitted: {len(late)}, "
      f"{late.get_level_values('time').min():%H:%M}-{late.get_level_values('time').max():%H:%M}")
print(f"satellites: {', '.join(sorted(set(late.get_level_values('sv'))))}")
```

system	median epoch Toe (min)	rows with Toe after the epoch (%)	median TransTime Toe (min)	rows with a record already transmitted
GPS	0.000	49.706	-119.700	100.000
Galileo	18.000	1.249	11.667	97.862

```
Galileo rows with a record not yet transmitted: 89, 00:00-00:34
satellites: E02, E04, E12, E13, E15, E25
```

The two systems behave very differently. A GPS record is transmitted about two hours before its Toe, so the nearest Toe lies in the future at about half of the epochs and still every selected record had already been transmitted. A Galileo record is transmitted about 12 minutes after its Toe; with the transmitted-first rule the selected record is typically some 15-20 minutes old, and almost every row uses a record

that was on the air at the epoch. The few exceptions are all in the first 35 minutes of the day: for these satellites the daily file holds no earlier record within the 3 h window, so GNX-py falls back to the nearest Toe.

From the library point of view, GPS and Galileo are the straightforward branches of this tutorial: the main decisions are the signal mode, whether to use external bias products, and whether to include satellite PCO and eclipse flags. BeiDou adds one more layer: the clock datum and group-delay convention have to be read before the clock error is interpreted.

3.5 BeiDou in SIS analysis

BeiDou support is one of the important current additions to GNX-py. It is also the constellation where this tutorial has to be most careful. For BDS, a large clock error can be a real product issue, but it can also be a sign that we compared two clocks expressed in different conventions.

3.5.1 Legacy BDS B1IB3I

The stable default for current BDS SIS validation is `B1IB3I`. This mode represents the legacy B1I/B3I ionosphere-free convention. In the present implementation it is the recommended path for comparing BDS broadcast navigation with precise SP3 clocks for MEO/IGSO satellites.

The reason is not only numerical. Broadcast BDS navigation includes legacy group-delay fields such as `TGD1` and `TGD2`, both relative to B3I, the signal of the broadcast clock. The precise BeiDou clocks of the 2024 MGEX products (COD, GRG, GFZ) refer to the ionosphere-free B1I/B3I combination (the C2I/C6I observables). `B1IB3I` tells the pipeline to use the matching legacy IF correction, while also annotating the output with `clock_convention_status`.

GNX-py does not take the datum of the precise clocks for granted. Once per run it reads it from a bias product of the same analysis center that declares it, or tests it on the broadcast clocks. Some products refer the BDS-3 clocks to the IF(B1C,B2a) combination instead (the COD MGEX products of 2026). With a RINEX 4 navigation file, whose B-CNAV records carry the group delays `TGD_B1Cp` and `TGD_B2ap`, GNX-py converts these clocks to IF(B1I,B3I) and the status stays `ok`; with a RINEX 3 file, which has no B-CNAV records, the status is `warning`.

A single-signal mode such as `B1I` can be useful for receiver-style user corrections, but it is not automatically comparable with a precise ionosphere-free clock. If we mix those conventions, the clock term that enters SISRE (after the datum and group-delay corrections) can exceed 20 m for some satellites even when the orbit is not the problem; we will measure it below. In this tutorial, therefore, `B1IB3I` is the default BDS example.

```
bds_config = build_broadcast_vs_precise_config(system="C", bds_mode="B1IB3I")
bds_result, bds_warnings = run_sis(bds_config)

print(f"rows={len(bds_result)}")
print(f"satellites={bds_result.index.get_level_values('sv').nunique()}")
print(f"captured warnings={len(bds_warnings)}")
print_warnings(bds_warnings)
viz.df_head(bds_result[key_columns(bds_result)], nrows=4, ncols=12)
```

```
rows=6516
satellites=36
captured warnings=4
1. sample_data/BRDC00IGS_R_20240350000_01D_MN.rnx: broadcast ionosphere record header
   BDSA C50 rejected: BeiDou Klobuchar amplitude A2 <= 0 for every |latitude| <= 1/3
   semicircle (no daytime term)
2. BeiDou broadcast interpolation currently supports BDS MEO/IGSO only; skipping GEO
   satellites: C01, C02, C03, C04, C05, C59, C60, C62.
3. SP3 files disagree on the position at an epoch both carry (more than 1 m): C11
   2024-02-04 00:00:00 (14.61 m); that position node is not used.
4. SP3 interpolation: 181 satellite-epochs left out: 181 in gaps of missing SP3
   positions/clocks: C11 (position).
```

sv	time	dR	dA	dC	dt	dt_mean	dTGD
C06	2024-02-04 00:00:00	1.697	0.113	0.114	22.320	22.471	7.501
C06	2024-02-04 00:02:00	1.697	0.104	0.119	22.320	22.473	7.501
C06	2024-02-04 00:04:00	1.695	0.104	0.111	22.317	22.474	7.501
C06	2024-02-04 00:06:00	1.693	0.103	0.104	22.322	22.477	7.501

PDF preview: 4 column(s) omitted from this wide table.

```
clock_convention = summarize_clock_convention(bds_result)
viz.df_head(clock_convention.drop(columns="note", errors="ignore"), ncols=len(clock_convention.
    columns), reset_index=False)
if "note" in clock_convention:
    print(textwrap.fill(clock_convention.loc[0, "note"], width=88))
```

system	mode	status	directly comparable
C	B1IB3I	ok	True

```
BeiDou B1I/B3I ionosphere-free mode uses the legacy IF broadcast datum correction a*TGD1
and is the validated legacy mode for precise C2I/C6I-referenced clocks (precise clock
datum IF(B1I,B3I) detected).
```

The `clock_convention_status` metadata is part of the interpretation, not just a technical detail. For the `B1IB3I` example it should report that the comparison is directly comparable under the validated legacy convention, and the note tells us that the IF(B1I,B3I) datum of the precise clocks was detected, not assumed. The result table also carries per-row `clock_convention_*` columns so that downstream summaries can keep the warning/status information with the numerical values.

The warnings of this run are worth reading one by one:

- the BeiDou GEO satellites (C01-C05, C59, C60 and C62) are skipped. This is expected in the current SIS/SISRE path: MEO and IGSO are processed; GEO is explicitly excluded because the current coefficient and validation stack is not meant for GEO geometry;
- C11 is left out: the COD SP3 files of two consecutive days disagree by about 15 m on its position at midnight, and the positions of C11 have gaps in this window;
- a BeiDou broadcast ionosphere record of the navigation header is rejected; it does not matter for SIS.

When reading the BeiDou numbers, keep in mind which clock quantity each column holds. `dt` is the raw broadcast-minus-precise clock difference before any datum correction, so for BDS it reaches tens of metres: its per-satellite mean ranges from -27 m (C33) to 32 m (C07) in this window, about 22 m for C06 in the table above. Most of it is the common datum `dt_mean`, which for BeiDou is taken per epoch and satellite generation (about 22 m for BDS-2 and 10 m for BDS-3 here), together with the broadcast group-delay term `dTGD`. The SIS columns use the corrected clock term `(dt - dt_mean) - (dTGD - dTGD_mean)`, so a large raw `dt` does not by itself contradict the `directly_comparable` status. In the per-satellite table below, `dt_mean` is the mean of `dt` for each satellite, computed by `orbit_stats`, not the per-epoch datum column with the same name.

```
bds_stats = orbit_stats(
    bds_result,
    columns=["dR", "dA", "dC", "dt", "dTGD", "sisre"],
    stats=("mean", "95%", "rms"),
)
viz.df_head(bds_stats, nrows=10, ncols=12, reset_index=True)
```

sv	dR_mean	dR_95%	dR_rms	dA_mean	dA_95%	dA_rms	dC_mean
C06	1.723	1.790	1.724	-0.251	0.099	0.392	-0.511
C07	1.846	1.970	1.847	-0.190	0.582	0.495	0.148
C08	1.166	1.469	1.178	0.982	1.372	1.008	-0.267
C09	1.721	1.800	1.722	-0.338	-0.043	0.431	0.125
C10	1.772	1.919	1.774	-0.169	0.607	0.664	0.269
C12	1.489	1.742	1.523	1.328	1.592	1.341	0.035
C13	1.862	1.952	1.863	-7.246	-6.945	7.250	-9.494
C14	0.623	1.223	0.721	-0.440	1.459	1.091	-0.570
C16	1.353	1.443	1.355	-0.547	-0.266	0.574	-0.093
C19	0.084	0.163	0.094	0.264	0.332	0.267	0.141

PDF preview: 4 column(s) omitted from this wide table.

3.5.2 Antenna reference point and satellite generations

BeiDou is two constellations in one: the regional BDS-2 satellites and the global BDS-3 satellites. They differ in their antennas, in their broadcast clock datums and in the quality of their broadcast orbits, so the comparison is easier to read per generation. The generation of a satellite is taken from its ANTEX block (`BEIDOU-2*` or `BEIDOU-3*`), else from the B-CNAV records of the navigation file (only BDS-3 MEO/IGSO satellites transmit them), and only as the last resort from the PRN.

```
generation, _ = bds_satellite_generations(
    bds_result.index.get_level_values("sv").unique(), antex=load_antex(ATX_PATH), epoch=
    SMOKE_TLIM[0]
)
by_generation = bds_result.assign(generation=bds_result.index.get_level_values("sv").map(
    generation))
generation_summary = by_generation.groupby("generation").agg(
    satellites=("sisre", lambda column: column.index.get_level_values("sv").nunique()),
    mean_dR=("dR", "mean"),
    mean_dR_apc_signal=("dR_apc_signal", "mean"),
    mean_dt_datum=("dt_mean", "mean"),
    mean_dTGD_datum=("dTGD_mean", "mean"),
    median_sisre=("sisre", "median"),
)
generation_summary["mean_IF_clock_datum"] = generation_summary["mean_dt_datum"] -
    generation_summary["mean_dTGD_datum"]

print(f"precise orbit moved to the APC of: {bds_result.attrs['apc_reference']}")
print(f"dR_apc_signal: radial offset of the {bds_result.attrs['apc_signal']} APC from it")
viz.df_head(generation_summary, ncols=len(generation_summary.columns) + 1, reset_index=True)
```

```
precise orbit moved to the APC of: B3I
dR_apc_signal: radial offset of the IF(B1I,B3I) APC from it
```

generation	satellites	mean_dR	mean_dR_apc signal	mean_dt datum	mean_dTGD datum	median_sisre	mean_IF clock_datum
BDS-2	9	1.506	-1.789	22.480	4.148	1.046	18.332
BDS-3	27	0.015	-0.409	10.168	-2.180	0.506	12.348

Three things are visible here.

1. **Antenna reference point.** The precise orbit was moved to the B3I phase center, the reference of the BeiDou broadcast orbit and clock. The BDS-3 broadcast orbits sit at that point: their mean radial difference is a few centimetres. For BDS-2 it is about 1.5 m in this window. With the igs20 antenna offsets of the BDS-2 satellites, no single phase center brings the BDS-2 broadcast orbits to zero (on other days of 2024, outside `sample_data`, the mean radial offset at B3I ranged from about 0.3 to 1.7 m), so read the BDS-2 radial component with this in mind and evaluate the two generations separately.
2. **Signal APC.** The IF(B1I,B3I) phase center of the selected mode lies about 0.4 m (BDS-3) and 1.8 m (BDS-2) closer to the Earth than the B3I phase center (negative `dR_apc_signal`). This is reported only; the user corrects it with the broadcast group delays.
3. **Clock datum.** The epoch datum `dt_mean` is about 22 m for BDS-2 and 10 m for BDS-3. Even after the group-delay datum is removed, the ionosphere-free clock datums of BDS-2 and BDS-3 against the same precise product differ from each other by about 6 m (about 18 m and 12 m here). A single datum for both generations would leave this offset in the clock term of every satellite, so GNX-py takes the datum per epoch and generation. If a generation has fewer than 3 satellites at an epoch, it gets no datum there: `sisre` and `sisre_notgd` are NaN at that epoch, with one warning.

The per-satellite table above shows two more things worth knowing. C13 (a BDS-2 IGSO satellite) has along-track and cross-track errors of 7-10 m: this is its broadcast orbit, because the GRG and COD precise orbits of C13 agree within about 0.2 m in this window. The small transverse weight of the IGSO class ($w_{AC}^2 = 1/126$) keeps its SISRE at about 2 m (RMS 1.8 m). And among all BeiDou satellites, C06

has the largest SISRE (about 5 m RMS) because of the reference: the COD clock of C06 is about 4 m off the GRG clock on this day, and with the GRG product as `orb_path_1` the SISRE RMS of C06 drops to about 1 m. A comparison is never better than its reference product.

3.5.3 Clock datum, TGD, DCB, OSB and DSB

This is the part of SIS analysis where it is easy to fool oneself. A broadcast clock, a precise SP3 clock, a TGD value, a DSB value and an OSB value may all refer to satellite hardware delays, but they do not necessarily live in the same convention.

- `TGD` is a broadcast group-delay correction, represented internally by GNX-py in metres.
- `DCB` / `DSB` is a differential bias between two observables.
- `OSB` is an observable-specific bias for one signal/code observable.
- `clock_bias=True` removes a constellation-common clock offset per epoch.
- `clock_convention_status` tells us whether the BDS clock term can be interpreted directly.

The important rule is: compare clocks only after they have been expressed in the same convention. For BDS this often means using `B1IB3I` for legacy validation, or using a modern RINEX 4 + OSB/DSB stack only when the clock reference is documented and consistent.

```
bias_catalog = []
for label, product_path in {
    "CAS DSB": CAS_DSB,
    "GRG OSB": GRG_OSB,
    "GFZ OSB": GFZ_OSB,
}.items():
    bias_df, bias_kind = gnx.GNSSDataProcessor2().read_bia(product_path)
    bds_columns = [col for col in bias_df.columns if "C2I" in col or "C6I" in col or "C1X" in
    col or "C5X" in col or "C7Z" in col]
    bias_catalog.append(
        {
            "product": label,
            "kind": bias_kind,
            "rows": len(bias_df),
            "example_columns": ", ".join(bds_columns[:6]),
        }
    )

viz.df_head(pd.DataFrame(bias_catalog), reset_index=False, truncate_str=None)
```

product	kind	rows	example columns
CAS DSB	DSB	99	BIAS_C1C_C5X, BIAS_C1P_C6I, BIAS_…
GRG OSB	OSB	94	OSB_C1X, OSB_C2I, OSB_C5X, OSB_C6I
GFZ OSB	OSB	95	OSB_C2I, OSB_C6I

The table above is intentionally modest. It does not pretend that every listed bias column can be dropped into every SIS run. For example, a DSB product can describe a pairwise code bias, but it does not by itself define a complete single-signal clock datum. An OSB product can be more flexible, but only if the selected observables and precise clock convention match.

This is why the BDS clock status is reported in the output. If the status is `requires_validation`, the numbers are still useful for diagnostics, but they should not be presented as trusted clock/SIS errors without a documented bias and clock-datum conversion.

```
b1i_config = build_broadcast_vs_precise_config(
    system="C",
    bds_mode="B1I",
    tlim=FULL_DAY_TLIM,
    interval=1800,
)
b1i_result, b1i_warnings = run_sis(b1i_config)
b1ib3i_result, b1ib3i_warnings = run_sis(
```

```
    build_broadcast_vs_precise_config(system="C", bds_mode="B1IB3I", tlim=FULL_DAY_TLIM, interval=1800)
)
print("B1I warnings:")
print_warnings(b1i_warnings, shown=bds_warnings)
print("B1IB3I warnings (full day):")
print_warnings(b1ib3i_warnings, shown=bds_warnings + b1i_warnings)

status_demo = summarize_clock_convention(b1i_result)
viz.df_head(status_demo[["mode", "status", "directly_comparable"]], reset_index=False)


def clock_term(df: pd.DataFrame) -> pd.Series:
    # the clock term of sisre: dt'' = (dt - dt_mean) - (dTGD - dTGD_mean)
    return ((df["dt"] - df["dt_mean"]) - (df["dTGD"] - df["dTGD_mean"])).abs()


mode_comparison = pd.DataFrame(
    {
        mode: {
            "largest abs(dt'') (m)": clock_term(df).max(),
            "satellite": clock_term(df).groupby(level="sv").max().idxmax(),
            "median per-satellite SISRE RMS (m)": orbit_stats(df, columns="sisre", stats=("rms",))["sisre_rms"].median(),
        }
        for mode, df in {"B1I": b1i_result, "B1IB3I": b1ib3i_result}.items()
    }
).T.rename_axis("mode")
viz.df_head(mode_comparison, reset_index=True)
```

```
B1I warnings:
1. SP3 interpolation: 16 satellite-epochs left out: 16 in gaps of missing SP3
   positions/clocks: C11 (position).
2. BeiDou SIS clock convention status requires_validation for mode B1I: BeiDou single-
   signal mode B1I is not directly comparable with a precise IF clock without an
   explicit clock-datum conversion from a matching OSB/DSB product. Use B1IB3I for
   legacy BDS SIS clock validation, or enable compare_dcb with a clock-reference
   consistent product. BDS precise clocks are tied to an IF clock reference chosen by
   the analysis centre: IF(B1I,B3I) (C2I/C6I) in the MGEX products of 2024, IF(B1C,B2a)
   for the BDS-3 clocks of COD MGEX in 2026 (the bias product declares it,
   SATELLITE_CLOCK_REFERENCE_OBSERVABLES; mode B1IB3I converts them with the B-CNAV
   group delays of a RINEX 4 navigation file). Broadcast TGD is a user correction for a
   selected signal or IF combination, not a generic DCB/OSB replacement. SIS clock
   errors are directly comparable only after both clocks are expressed in the same
   datum.
(3 more warning(s) identical to messages printed above)
B1IB3I warnings (full day):
(4 more warning(s) identical to messages printed above)
```

mode	status	directly comparable
B1I	requires_validation	False

mode	largest abs(dt”) (m)	satellite	median per satellite SISRE RMS (m)
B1I	26.115	C33	3.972
B1IB3I	3.882	C06	0.593

The `requires_validation` warning printed above ends with the clock-convention note that the result also carries in its metadata (`clock_convention_note`). The `B1I` example is deliberately a coarse diagnostic: the whole day sampled every 30 minutes instead of every 2 minutes. It shows the policy difference without

turning the notebook into a long benchmark. With the same orbits, the clock term of `B1I` reaches more than 20 m and the median SISRE grows several times compared with `B1IB3I`. The expected lesson is not "never use B1I"; the lesson is that a single-signal BDS clock comparison against a precise IF clock needs an explicit clock-datum conversion. Without that, the clock term may be dominated by convention mismatch rather than physical orbit/clock quality.

3.5.4 Modern BDS and GEO

Modern BDS signals such as `B1C`, `B2a` and `B2b` are important, but in the current GNX-py SIS path they should be treated as validation experiments. They require navigation data with the relevant RINEX 4 CNAV / ISC / TGD fields and a bias stack that is consistent with the precise clock product. They should not silently reuse legacy `TGD1` / `TGD2` as if the same correction applied to modern signals. When a bias product is used for `B1C` or `B2a`, GNX-py takes its pilot observable first (P, then X, then D), because the broadcast group delays `TGD_B1Cp` and `TGD_B2ap` refer to the pilot component; which B2b observable fits `TGD_B2bI` best is not settled yet.

For satellite orbit types, the current practical rule is also explicit:

- BDS **MEO** and **IGSO** are supported in the validated legacy SIS path.
- BDS **GEO** is skipped / unsupported in SIS/SISRE comparison, because the current coefficient and validation setup is not designed for GEO geometry.
- Modern BDS modes can be useful for experiments, but the output should be labelled as `requires_validation` unless the navigation, clock and bias products have been validated together.

3.5.5 PRN reassignment

A PRN identifies a signal, not a satellite. When a PRN is given to another satellite, the new satellite brings its own antenna offsets, clock and group delays. BeiDou reassigned several PRNs in April 2026, and the igs20 ANTEX file in `sample_data` already records it. Let us look at C12:

```
antex = load_antex(ATX_PATH)
c12_rows = []
for epoch in [datetime(2024, 2, 4), datetime(2026, 4, 21, 3), datetime(2026, 4, 21, 6),
    datetime(2026, 4, 21, 9)]:
    entry = antex.satellite("C12", epoch)
    c12_rows.append(
        {
            "epoch": epoch,
            "SVN": entry.svn.strip() if entry else "-",
            "ANTEX block": entry.type.strip() if entry else "no entry",
            "generation": bds_satellite_generations(["C12"], antex=antex, epoch=epoch)[0]["C12"
    ] if entry else "-",
        }
    )
viz.df_head(pd.DataFrame(c12_rows), reset_index=False)
```

epoch	SVN	ANTEX block	generation
2024-02-04 00:00:00	C013	BEIDOU-2M	BDS-2
2026-04-21 03:00:00	C013	BEIDOU-2M	BDS-2
2026-04-21 06:00:00	-	no entry	-
2026-04-21 09:00:00	C233	BEIDOU-3M-CAST	BDS-3

Until 21 April 2026, 04:00, C12 was a BDS-2 MEO satellite (SVN C013); from 08:00 the same PRN belongs to a BDS-3 satellite (SVN C233). The old rule "PRN below 19 means BDS-2" is therefore wrong for recent data, which is why GNX-py takes the generation from the ANTEX block.

Every session that reads an ANTEX file checks the PRNs for its own time span: SIS with `atx_path`, and SPP, PPP and TEC with `atx_path` and satellite or receiver PCO. If the ANTEX entry of the satellite that holds a PRN at the start of the run ends at an epoch inside the run (the PRN is reassigned to another satellite, or no entry follows a VALID UNTIL), the session gives one `UserWarning` naming the PRN, the satellites and the epoch, and excludes the rows of that PRN after that epoch from both products. Consecutive entries of the same satellite (a new calibration) are not a change, and without an ANTEX

file nothing can be checked, because orbit products carry no satellite numbers. The sample day has no PRN change, so let us ask the antenna model directly for 21 April 2026:

```
with warnings.catch_warnings(record=True) as captured:
    warnings.simplefilter("always")
    antenna_model = build_antenna_model(
        ATX_PATH, None, datetime(2026, 4, 21, 0, 0), ("C",), epoch_end=datetime(2026, 4, 21, 23, 55)
    )
print_warnings([short_paths(str(item.message)) for item in captured])
viz.df_head(pd.DataFrame([asdict(change) for change in antenna_model.prn_changes.values()]), reset_index=False)
```

```
1. ANTEX igs20.atx: the satellite of PRN C04 (SVN C006 valid until 2026-04-21 08:00:00, then C217), C05 (SVN C011 valid until 2026-04-21 14:00:00, no later entry within the session), C12 (SVN C013 valid until 2026-04-21 04:00:00, then C233), C48 (SVN C233 valid until 2026-04-21 04:00:00, no later entry within the session), C62 (SVN C217 valid until 2026-04-21 08:00:00, no later entry within the session) changes or loses its entry during the session: their observations (orbit rows) after that epoch are excluded (a reassigned PRN belongs to another satellite with its own antenna, clock and group delays)
```

prn	svn	change	next_svn	next_from
C04	C006	2026-04-21 08:00:00	C217	2026-04-21 12:00:00
C05	C011	2026-04-21 14:00:00		NaT
C12	C013	2026-04-21 04:00:00	C233	2026-04-21 08:00:00
C48	C233	2026-04-21 04:00:00		NaT
C62	C217	2026-04-21 08:00:00		NaT

On that day five BeiDou PRNs change or lose their satellite. Note C48 and C12: the satellite C233 transmitted as C48 until 04:00 and as C12 from 08:00, so a run that ignored the change would join the clocks and group delays of two different satellites under one PRN.

3.6 Example 2 - SP3 vs SP3 product comparison

The second classical use case is comparing two precise SP3 products. The logic is the same as before, but both products are interpolated SP3 files. In this case, GNX-py uses adjacent-day products when provided, because interpolation and velocity estimation near product boundaries benefit from more context.

Here we compare GRG and COD MGEX SP3 products over a short GPS window. The example disables eclipse flagging to keep the SP3-vs-SP3 tutorial focused on product-to-product RAC/SISRE differences.

```
sp3_config = SISConfig(
    orb_path_0=GRG_SP3,
    orb_path_1=COD_SP3,
    prev_sp3_0=GRG_SP3_PREV,
    next_sp3_0=GRG_SP3_NEXT,
    prev_sp3=COD_SP3_PREV,
    next_sp3=COD_SP3_NEXT,
    interval=INTERVAL_SECONDS,
    gps_mode="L1L2",
    gal_mode="E1E5a",
    bds_mode="B1IB3I",
    atx_path=ATX_PATH,
    system="G",
    tlim=SMOKE_TLIM,
    compare_dcb=False,
    clock_bias=True,
    clock_bias_function="mean",
    apply_eclipse=False,
    apply_satellite_pco=False,
)

```

```
sp3_result, sp3_warnings = run_sis(sp3_config)

print(f"rows={len(sp3_result)}")
print(f"satellites={sp3_result.index.get_level_values('sv').nunique()}")
print(f"captured warnings={len(sp3_warnings)}")
viz.df_head(sp3_result[key_columns(sp3_result)], nrows=4, ncols=10)
```

```
rows=5792
satellites=32
captured warnings=0
```

sv	time	dR	dA	dC	dt	dt_mean	dTGD
G01	2024-02-04 00:00:00	0.003	-0.034	0.002	-0.041	-0.100	0.000
G01	2024-02-04 00:02:00	0.005	-0.033	0.004	-0.041	-0.100	0.000
G01	2024-02-04 00:04:00	0.004	-0.033	0.003	-0.040	-0.100	0.000
G01	2024-02-04 00:06:00	0.003	-0.033	0.002	-0.039	-0.100	0.000

PDF preview: 2 column(s) omitted from this wide table.

```
sp3_stats = orbit_stats(
    sp3_result,
    columns=["dR", "dA", "dC", "dt", "sisre"],
    stats=("mean", "95%", "rms"),
)
viz.df_head(sp3_stats, nrows=8, ncols=10, reset_index=True)
```

sv	dR_mean	dR_95%	dR_rms	dA_mean	dA_95%	dA_rms	dC_mean
G01	-0.000	0.009	0.006	-0.021	0.001	0.026	-0.007
G02	-0.014	0.013	0.025	-0.018	0.014	0.024	0.002
G03	0.004	0.025	0.017	-0.012	0.006	0.016	-0.022
G04	-0.028	-0.001	0.033	-0.027	0.003	0.031	0.005
G05	0.009	0.016	0.012	0.002	0.007	0.004	-0.010
G06	0.010	0.020	0.013	0.002	0.018	0.010	-0.009
G07	0.017	0.022	0.017	-0.008	0.014	0.014	0.006
G08	-0.009	0.007	0.011	0.008	0.055	0.031	0.016

PDF preview: 2 column(s) omitted from this wide table.

```
fig = viz.plot_sv_stats(
    sp3_stats,
    "sisre_rms",
    title="SP3 vs SP3: SISRE RMS per GPS satellite",
    ylabel="SISRE RMS [m]",
)
```

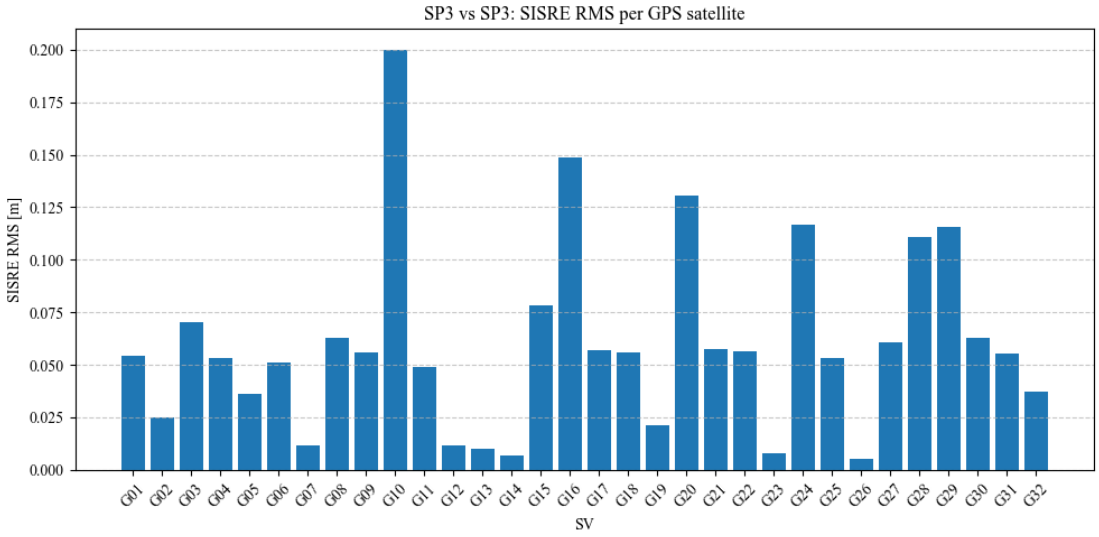


Figure 3.3: Per-satellite statistics for the SP3-versus-SP3 comparison example.

In SP3-vs-SP3 comparisons the numbers are usually much smaller than in broadcast-vs-precise comparisons. This is expected: we are comparing two precise analysis products rather than a user broadcast message against a precise reference. Still, the same interpretation applies. Inspect the time series for temporal structure, inspect per-satellite statistics for outliers, and remember that the clock datum correction can dominate the interpretation.

3.7 What to try next

We have discussed the orbit/SIS module and the `gnx_py.viz` functions used to inspect its results. The main ideas to keep are:

- always know which product is treated as reference, and remember that a comparison is never better than its reference,
- compare APC with APC (the broadcast reference APC: IF(L1,L2) for GPS, B3I for BeiDou), or know when PCO was not applied,
- interpret `dR`, `dA`, `dC` in the local satellite frame rather than only as Cartesian differences,
- treat `dt` and `dTGD` as convention-sensitive quantities,
- for BDS start from `B1IB3I` MEO/IGSO, inspect `clock_convention_status` and read BDS-2 and BDS-3 separately,
- read the warnings of a run: they name the satellites that were left out and why,
- use modern BDS modes only as clearly labelled validation experiments until the RINEX 4 and bias stack is validated.

For practice, extend the time window from six hours to the whole day and compare the per-satellite SISRE statistics. You can also switch the `system` field between `G`, `E` and `C`, compare different SP3 products (for BeiDou, GRG against COD shows the C06 clock offset directly), or use the RINEX 4 navigation file `BRD400DLR_S_20240350000_01D_MN.rnx` from `sample_data` as `orb_path_0`. In the next tutorial step, we will move toward the ionosphere module, where the same bias-convention discipline becomes even more important.

Chapter 4

Ionosphere Introduction

4.1 Ionospheric delay in GNSS observations

Ionospheric delay is one of the key factors degrading GNSS positioning. Long-established methods of dealing with it include modelling ionospheric delay or using a combination of observations at two frequencies. The ionosphere-free combination removes the first-order ionospheric delay, but it does so at the cost of increased observation noise (Teunissen and Montenbruck 2017; HofmannWellenhof et al. 2008). Ionospheric modelling and monitoring is a broad field of GNSS-related research (e.g. (Mannucci et al. 1998; Schaer 1999)).

GNX-py offers functionality for measuring, modelling and monitoring the spatial and temporal changes of the ionosphere. In this notebook we will focus on the introductory workflow: measuring Slant TEC (STEC) for one station, comparing it with ionosphere models, and understanding the diagnostic columns that the current library returns.

Current version note. The current `gnx_py.ionosphere` workflow supports GPS, Galileo and BeiDou through the shared signal registry. `TECSession` processes one constellation at a time, so for a multi-GNSS comparison we run separate sessions for `G`, `E` and `C`. For BDS, `B1IB3I` is the best-tested STEC pair in the repository at the moment.

Notation

- f_i: carrier frequency i [Hz],
- $\lambda_i = c/f_i$: wavelength,
- c: speed of light,
- $N_e(s)$: electron density [el/m^3],
- STEC $= \int_{\mathrm{LOS}} N_e(s)\, ds$ [el/m^2],
- 1 TECU $= 10^{16}$ el/m^2,
- ρ: geometric range,
- T: tropospheric delay,
- $\delta t_r, \delta t_s$: receiver/satellite clock offsets,
- P_i: code measurement on f_i [m],
- $\Phi_i = \lambda_i \varphi_i$: carrier phase observation [m],
- $d^P_{(\cdot),i}, d^L_{(\cdot),i}$: hardware delays,
- N_i: carrier ambiguity.

4.1.1 Definition of STEC

Ionospheric delay on the satellite-receiver path is defined as the line integral of electron density along the signal path:

$$\mathrm{STEC} = \int_{\mathrm{LOS}} N_e(s)\, ds \qquad [\mathrm{el/m^2}].$$

The commonly accepted unit for describing STEC is TECU, Total Electron Content Unit:

$$1 \text{ TECU} = 10^{16} \text{ el/m}^2.$$

STEC is a slant quantity. It depends on the satellite-receiver geometry. When the satellite is low above the horizon, the signal travels a longer path through the ionosphere and STEC increases even if the vertical ionosphere above the station is unchanged.

4.1.2 Ionospheric delay in code and phase

In GNSS processing we usually want the ionospheric delay in metres for a given signal. For first-order ionospheric delay:

$$I_i = \frac{40.3 \cdot 10^{16}}{f_i^2} \text{ STEC}.$$

Here I_i is the ionospheric delay in metres for signal i, while STEC is expressed in TECU. The sign is different for code and phase observations: the ionosphere delays the group velocity of code observations and advances the carrier phase.

$$P_i : +I_i \quad \text{(group delay)}, \qquad \Phi_i : -I_i \quad \text{(phase advance)}.$$

This sign difference is the reason why dual-frequency code and phase observations are so useful for ionospheric work. The ionosphere is dispersive, while many other terms in the observation equation are not.

4.1.3 Observation equations

Recording code and phase observations as P_i and Φ_i:

$$\begin{aligned} P_i &= \rho + c(\delta t_r - \delta t_s) + T + I_i + d^P_{r,i} - d^P_{s,i} + \varepsilon_{P_i}, \\ \Phi_i &= \rho + c(\delta t_r - \delta t_s) + T - I_i + \lambda_i N_i + d^L_{r,i} - d^L_{s,i} + \varepsilon_{\Phi_i}. \end{aligned}$$

The terms ρ, receiver/satellite clock and troposphere are non-dispersive. They appear in the same way on both frequencies. The first-order ionosphere and hardware delays are frequency-dependent, which is why a two-frequency combination can expose them.

4.1.4 Geometry-free combination

STEC measurement using GNSS is possible thanks to the geometry-free combination. It removes all non-dispersive terms, such as the geometric range, clocks and tropospheric delay. For two frequencies:

$$\begin{aligned} P_{\text{GF}} &= P_1 - P_2 = (I_1 - I_2) + \left[(d^P_{r,1} - d^P_{r,2}) - (d^P_{s,1} - d^P_{s,2})\right] + \eta_P, \\ \Phi_{\text{GF}} &= \Phi_1 - \Phi_2 = -(I_1 - I_2) + (\lambda_1 N_1 - \lambda_2 N_2) + \left[(d^L_{r,1} - d^L_{r,2}) - (d^L_{s,1} - d^L_{s,2})\right] + \eta_\Phi. \end{aligned}$$

The code geometry-free observable is noisy but absolute after bias correction. The phase geometry-free observable is much cleaner, but it contains an unknown constant ambiguity term for each continuous satellite arc. GNX-py therefore detects cycle slips and tracking gaps, splits observations into arcs, and levels the phase combination to the code combination.

4.1.5 Biases and STEC observables

The remaining biases in the geometry-free combination are differences in hardware delays between two frequencies for satellites and receivers. Satellite DCB/DSB/OSB values can be introduced from external products. Receiver DCB values can be taken from GIM/IONEX auxiliary blocks, manually configured, estimated in a calibration workflow, or set to zero when we only need a diagnostic run.

GNX-py currently uses the geometry-free code convention

```
P4 = P2 - P1
```

inside the STEC bias policy. OSB values are differenced as $OSB(P_1) - OSB(P_2)$, while DCB/DSB products are treated as differential pair biases for the selected code pair. The source order is controlled by `TECConfig.stec_bias_sources`, typically:

```
("product", "gim", "config", "zero")
```

After applying the selected bias convention, code-derived STEC can be written as

$$\mathrm{STEC}^{(P)} = \frac{P_{\mathrm{GF}} - \mathrm{DCB}_{12}^{P}}{40.3 \cdot 10^{16} \left(\frac{1}{f_1^2} - \frac{1}{f_2^2}\right)} \ [\mathrm{TECU}].$$

For phase observations, with constant B_Φ:

$$\mathrm{STEC}^{(\Phi)} = -\frac{\Phi_{\mathrm{GF}} - B_\Phi}{40.3 \cdot 10^{16} \left(\frac{1}{f_1^2} - \frac{1}{f_2^2}\right)} \ [\mathrm{TECU}].$$

Current version note. The output dataframe now includes explicit bias diagnostics such as `stec_sat_bias_m`, `stec_station_bias_m`, `stec_total_bias_m`, `stec_sat_bias_source`, `stec_station_bias_source`, `stec_bias_code_1` and `stec_bias_code_2`. These columns are part of the scientific interpretation. If you change the signal pair or the product source, inspect them before interpreting the STEC values. Each satellite arc receives one satellite and one station bias constant, and the output keeps four calibration levels of the same levelled phase: `stec_uncal` (no bias removed), `stec_sat` (satellite bias only), `stec_rcv` (receiver bias only) and `stec_full` (both; this is `leveled_tec`). The constants are in `stec_sat_bias_tecu` and `stec_rcv_bias_tecu`, and `stec_bias_datum` names the datum of the satellite biases.

4.1.6 Phase-to-code leveling

B_Φ is a linear combination of phase ambiguities at both frequencies. Because of this bias, the phase geometry-free observation is precise but not on the correct absolute scale. To remove it, we use phase-to-code leveling: the phase observation is shifted to match the code observation over a local window (Mannucci et al. 1998).

GNX-py uses an elevation-weighted form:

$$\tilde{L}_4^{(n)} = L_4^{(n)} + \frac{\sum_{k=1}^{K} w_k \left[P_4^{(k)} - L_4^{(k)}\right]}{\sum_{k=1}^{K} w_k}, \qquad w_k = \sin^4 E_k.$$

where:

- $\tilde{L}_4^{(n)}$ is the levelled carrier phase geometry-free observation at epoch n,
- $P_4^{(k)}$ is the code geometry-free observation at epoch k,
- $L_4^{(k)}$ is the phase geometry-free observation at epoch k,
- K is the number of samples used for levelling,
- E_k is the satellite elevation at epoch k; low samples, where code multipath is largest, get small weights.

By default (`leveling_ws=None`) GNX-py levels each continuous arc as a whole, so one constant is added to the phase of the arc. An integer `leveling_ws` levels over a moving window of that many samples instead, which lets part of the code multipath back into the result. `median_leveling_ws` controls the median variant (`median_leveled_tec`) in the same way. The column `leveling_std_tecu` gives, for each arc, the weighted scatter of the code around the levelled phase, a quick quality indicator. The output keeps several versions of the measured TEC, for example `code_tec`, `phase_tec`, `leveled_tec`, `median_leveled_tec`, `leveled_tec_sg` and `leveled_tec_cheby`. The point is not that one column is always best. The point is that the dataframe exposes enough diagnostic information to see how the measurement was built.

4.1.7 Mapping STEC to VTEC

In ionosphere modelling using GNSS, it is usually assumed that electrons are concentrated in an infinitely thin shell at a certain altitude, for example 350, 400 or 450 km. The conversion between STEC and VTEC is based on this geometry. Assume shell height h_{ion}, Earth radius R_E and satellite elevation E:

$$M(E) = \frac{1}{\sqrt{1 - \left(\frac{R_E}{R_E + h_{\text{ion}}} \cos E\right)^2}}.$$

Then

$$\text{STEC} \approx M(E) \cdot \text{VTEC}, \qquad \text{VTEC} \approx \frac{\text{STEC}}{M(E)}.$$

This mapping function is simple, but it is good enough for an introductory tutorial. More advanced mapping functions and shell assumptions can be found in the literature and are important when building operational ionosphere products.

4.2 Current GNX-py ionosphere workflow

The introductory STEC pipeline in GNX-py is controlled by `TECConfig` and executed by `TECSession`:

1. `TECConfig` selects the station file, constellation, signal pair, orbit source, ionosphere model and bias policy.
2. `TECSession` reads the observations with GNX-py's own RINEX reader, loads the products, interpolates satellite geometry, applies selected corrections and detects cycle slips.
3. `STECMonitor` groups observations by satellite arc, applies the configured bias policy, performs phase-to-code leveling, and returns a dataframe with both STEC values and diagnostics.

Important practical points in the current version:

- `TECSession` processes one constellation at a time: use `use_sys="G"`, `"E"` or `"C"`.
- GPS and Galileo are the historical paths; BeiDou is active, with `B1IB3I` currently the best-tested BDS pair.
- `compare_models` can add `klobuchar`, `ntcm`, `bdgim` and/or `gim` comparison columns, but this path should be treated as model-comparison support, not as a certification of every product combination.
- Bias source selection is explicit through `stec_bias_sources` and `stec_missing_bias`.
- Activity indexes and kriging work on a network of stations; they are the topics of the later chapters.

4.3 Notebook setup

The notebook uses the repository `sample_data/` directory and avoids local absolute paths. The main station in this tutorial is `BRUX`, because the current sample data contains a compact observation file for that station together with navigation (RINEX 3 and RINEX 4), SP3, ANTEX, DCB/OSB and GIM products.

```
from __future__ import annotations

from datetime import datetime
```

```
from pathlib import Path
import contextlib
import importlib.util
import io
import os
import warnings

import numpy as np
import pandas as pd

import gnx_py as gnx
from gnx_py import viz
from gnx_py.ionosphere import TECConfig, TECSession

viz.apply_publication_style(font_scale=1.3, mathtext="cm")


def find_repo_root(start: Path) -> Path:
    for candidate in [start.resolve(), *start.resolve().parents]:
        if (candidate / "pyproject.toml").exists() and (candidate / "gnx_py").is_dir():
            return candidate
    raise FileNotFoundError(
        "Run this notebook from inside the cloned GNX-py repository, "
        "or set REPO_ROOT manually to the repository directory."
    )


REPO_ROOT = find_repo_root(Path.cwd())
SAMPLE_DATA = REPO_ROOT / "sample_data"
TUTORIAL_DIR = REPO_ROOT / "tutorial" / "ionosphere"

## Only repository-relative paths are printed, so no machine-specific path ends up in the output.
local_checkout = Path(gnx.__file__).resolve().parent == (REPO_ROOT / "gnx_py").resolve()
print(f"gnx_py imported from: "
      f"{'gnx_py/ of this repository' if local_checkout else 'an installed package'}")
print(f"sample data: {SAMPLE_DATA.relative_to(REPO_ROOT)}")
```

```
gnx_py imported from: gnx_py/ of this repository
sample data: sample_data
```

```
def load_tutorial_module(module_name: str, module_path: Path):
    spec = importlib.util.spec_from_file_location(module_name, module_path)
    module = importlib.util.module_from_spec(spec)
    if spec.loader is None:
        raise ImportError(f"Cannot load {module_name} from {module_path}")
    spec.loader.exec_module(module)
    return module


## tools.py holds only the rough zenith-VTEC estimator used below; every figure
## and table preview in this notebook comes from gnx_py.viz.
tools = load_tutorial_module("ionosphere_tutorial_tools", TUTORIAL_DIR / "tools.py")
VTECZenithEstimator = tools.VTECZenithEstimator
```

```
STATION = "BRUX"
DOY = 35

OBS_PATH = SAMPLE_DATA / "BRUX00BEL_R_20240350000_01D_30S_MO.crx.gz"
NAV_PATH = SAMPLE_DATA / "BRDC00IGS_R_20240350000_01D_MN.rnx"
## RINEX 4 navigation file: it also carries every broadcast ionosphere set as a '> ION' record (BDGIM below)
```

```
NAV4_PATH = SAMPLE_DATA / "BRD400DLR_S_20240350000_01D_MN.rnx"
SP3_PATHS = [
    SAMPLE_DATA / "GRGOMGXFIN_20240340000_01D_05M_ORB.SP3",
    SAMPLE_DATA / "GRGOMGXFIN_20240350000_01D_05M_ORB.SP3",
    SAMPLE_DATA / "GRGOMGXFIN_20240360000_01D_05M_ORB.SP3",
]
ATX_PATH = SAMPLE_DATA / "igs20.atx"
DCB_PATH = SAMPLE_DATA / "CAS1OPSRAP_20240350000_01D_01D_DCB.BIA"
GIM_PATH = SAMPLE_DATA / "COD0OPSFIN_20240350000_01D_01H_GIM.INX"

## time_limit [start, end] is GPST and inclusive: GPS for the whole day, BeiDou for 20 minutes
GPS_TLIM = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 4, 23, 59, 30)]
BDS_TLIM = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 4, 0, 20)]

required_paths = [OBS_PATH, NAV_PATH, NAV4_PATH, ATX_PATH, DCB_PATH, GIM_PATH, *SP3_PATHS]
missing = [path for path in required_paths if not path.exists()]
if missing:
    raise FileNotFoundError("Missing sample files:\n" + "\n".join(str(path.relative_to(
    REPO_ROOT)) for path in missing))
```

4.4 Example of STEC processing in GNX-py

Let's now move on to an example of STEC measurement in GNX-py. The procedure is:

- obtain satellite coordinates at the time of signal transmission,
- introduce corrections that matter for the geometry-free combination, especially PCO and code biases,
- detect cycle slips and loss of tracking,
- split observations into continuous satellite arcs,
- level phase observations to code observations,
- convert metres/electrons into TECU and inspect the result.

In the dataframe you will see arc identifiers such as `G02_0_1`. I read them as: satellite `G02`, tracking-gap arc `0`, cycle-slip arc `1`. This is why one physical satellite can appear as several arc-labelled entries.

For simplicity, the whole pipeline is enclosed in `TECSession`, managed by `TECConfig`. We will start with GPS because it is the clearest introductory case, and then run a BeiDou `B1IB3I` example to show how the current bias policy generalizes to BDS.

4.4.1 TECConfig for this tutorial

Below we build a small helper around `TECConfig`. The configuration is intentionally explicit, because STEC values depend on processing choices:

parameter	why it matters here
`sys` / `use_sys`	`TECSession` processes one constellation at a time
`gps_freq`, `gal_freq`, `bds_freq`	select the two signals used in the geometry-free combination
`nav_path`	broadcast ephemerides and the broadcast ionosphere coefficients (Klobuchar, NTCM, BDGIM)
`orbit_type`	`broadcast` uses NAV interpolation; `precise` uses SP3 interpolation
`sat_pco`	`"los"` applies the satellite antenna offsets of both signals along the line of sight; `False` leaves the satellite term in STEC (about 4.8 TECU for GPS III on L1/L2)
`ionosphere_model`	background model of the `ion` column (`gim`, `klobuchar`, `ntcm`, `bdgim`); the GIM file can also provide DCB data
`compare_models`	requests extra model columns: `klobuchar`, `ntcm`, `bdgim`, `gim`
`add_dcb`, `add_sta_dcb`	enable satellite and receiver/station bias corrections

parameter	why it matters here
`stec_bias_sources`	defines fallback order: product, GIM, config, zero
`stec_missing_bias`	choose whether missing bias should warn/use zero or raise
`leveling_ws`, `median_leveling_ws`	phase-to-code levelling: `None` (default) levels each arc as a whole, an integer is an opt-in moving window
`time_limit`	restricts the processed interval `[start, end]` (GPST, inclusive); applied while the observation file is read

The tutorial uses `orbit_type="broadcast"` to keep the examples fast. The same `TECConfig` can be adapted to precise SP3 geometry by changing `orbit_type` and supplying `sp3_path`. With broadcast orbits the cycle-slip detector uses the same layers as with precise orbits: the loss-of-lock flags of the RINEX file, a third carrier when one is tracked and, at 30 s sampling, the geometry layer, which starts a new piece at every switch of the broadcast ephemeris record.

```
def run_tec(config: TECConfig, use_sys: str):
    """Run TECSession while keeping notebook output compact."""
    stream = io.StringIO()
    with warnings.catch_warnings(record=True) as captured, contextlib.redirect_stdout(stream):
        warnings.simplefilter("always")
        result = TECSession(config=config, use_sys=use_sys).run()
    # messages name input files by their full path: show them relative to the repository
    root = f"{REPO_ROOT}{os.sep}"
    warning_messages = list(dict.fromkeys(str(item.message).replace(root, "") for item in
    captured))
    return result, warning_messages


def build_tec_config(
    *,
    system: str = "G",
    time_limit=GPS_TLIM,
    compare_models=("ntcm", "klobuchar"),
    ev_mask: int = 30,
    nav_path=NAV_PATH,
) -> TECConfig:
    return TECConfig(
        obs_path=OBS_PATH,
        nav_path=nav_path,
        sp3_path=SP3_PATHS,
        atx_path=ATX_PATH,
        dcb_path=DCB_PATH,
        gim_path=GIM_PATH,
        sys=system,
        gps_freq="L1L2",
        gal_freq="E1E5a",
        bds_freq="B1IB3I",
        orbit_type="broadcast",
        ionosphere_model="gim",
        troposphere_model=False,
        time_limit=time_limit,
        day_of_year=DOY,
        station_name=STATION,
        screen=False,
        ev_mask=ev_mask,
        windup=True,
        rel_path=True,
        sat_pco="los",
        rec_pco=True,
        add_dcb=True,
        add_sta_dcb=True,
        rcv_dcb_source="gim",
```

```
        stec_bias_sources=("product", "gim", "config", "zero"),
        stec_missing_bias="warn_zero",
        compare_models=list(compare_models) if compare_models else False,
        use_iono_rms=True,
        min_arc_len=2,
    )


TEC_ELECTRON_COLUMNS = [
    "leveled_tec",
    "median_leveled_tec",
    "code_tec",
    "phase_tec",
    "leveled_tec_sg",
    "leveled_tec_cheby",
]
GPS_L1_HZ = 1575.42e6
L1_TECU_TO_M = 40.3e16 / GPS_L1_HZ**2


def with_tecu_units(df: pd.DataFrame) -> pd.DataFrame:
    """Return a display copy with the measured STEC columns in TECU and every model as slant
    TEC in TECU.

    compare_models already returns <model>_tecu columns; only the background model `ion` needs
    one here
    (its delay is on the first signal of the pair, L1 for L1L2)."""
    out = df.copy()
    for column in TEC_ELECTRON_COLUMNS:
        if column in out.columns:
            out[column] = out[column] / 1e16
    for column in ["ion", "klobuchar", "ntcm", "bdgim"]:
        if column in out.columns and f"{column}_tecu" not in out.columns:
            out[f"{column}_tecu"] = out[column] / L1_TECU_TO_M
    return out


def compact_columns(df: pd.DataFrame, columns: list[str]) -> list[str]:
    return [column for column in columns if column in df.columns]
```

4.4.2 Running GPS STEC

We will use the BRUX observation file from `sample_data`. GNX-py reads it with its own RINEX reader (`gnx_py.io.rinex_obs`): the Hatanaka-compressed `.crx.gz` file is decompressed in memory, nothing is written next to the data, and `time_limit` is applied while the file is read. `GPS_TLIM` covers the whole day of sample data, so the figures below span 24 hours. For a real analysis you would choose the interval deliberately and store summary products outside the notebook.

`run_tec` collects the warnings of the run instead of letting them interrupt the notebook; we print them below, because they are part of the result.

```
gps_config = build_tec_config(system="G", time_limit=GPS_TLIM, compare_models=("ntcm", "
    klobuchar"))
obs_tec, gps_warnings = run_tec(gps_config, use_sys="G")

print(f"rows={len(obs_tec)}")
print(f"satellite arcs={obs_tec.index.get_level_values('sv').nunique()}")
print(f"unique captured warnings={len(gps_warnings)}")
for message in gps_warnings:
    print(f"warning: {message}")
```

```
rows=14510
```

```
satellite arcs=42
unique captured warnings=1
warning: GPS satellites without a usable broadcast ephemeris (left out): G01 (unhealthy, health
    =63).
```

The resulting dataframe contains both the original observation/preprocessing columns and the ionosphere-specific outputs. The columns `ion`, `klobuchar` and `ntcm` are modelled delays in metres on the first signal of the pair (L1 here); `compare_models` also returns each comparison model as slant TEC in TECU (`klobuchar_tecu`, `ntcm_tecu`). The measured STEC columns are initially in electrons per square metre, so in the next step we create a display copy in TECU, with `ion_tecu` for the background model.

```
raw_preview_cols = compact_columns(
    obs_tec,
    [
        "ion", "klobuchar", "ntcm", "leveled_tec", "code_tec", "phase_tec",
    ],
)
## Measured STEC is still in el/m^2 here (~1e17), so use significant digits.
viz.df_head(obs_tec[raw_preview_cols], nrows=5, ncols=8, floatfmt=".4g", truncate_str=14)
```

sv	time	ion	klobuchar	ntcm	leveled_tec	code_tec	phase_tec
G02_0_1	2024-02-04 00:00:00	2.173	2.056	1.815	1.388e+17	1.416e+17	-4.52e+17
G02_0_1	2024-02-04 00:00:30	2.185	2.064	1.818	1.394e+17	1.019e+17	-4.514e+17
G02_0_1	2024-02-04 00:01:00	2.197	2.071	1.822	1.399e+17	1.289e+17	-4.509e+17
G02_0_1	2024-02-04 00:01:30	2.209	2.078	1.825	1.406e+17	1.21e+17	-4.502e+17
G02_0_1	2024-02-04 00:02:00	2.221	2.086	1.828	1.413e+17	1.711e+17	-4.496e+17

```
obs_tec_tecu = with_tecu_units(obs_tec)

preview_cols = compact_columns(
    obs_tec_tecu,
    [
        "leveled_tec", "median_leveled_tec", "code_tec", "phase_tec",
        "ion_tecu", "klobuchar_tecu", "ntcm_tecu", "ev",
    ],
)
viz.df_head(obs_tec_tecu[preview_cols], nrows=5, ncols=8, floatfmt=".2f", truncate_str=14)
```

sv	time	leveled_tec	median leveled_tec	code_tec	phase_tec	ion_tecu	klobuchar tecu
G02_0_1	2024-02-04 00:00:00	13.88	14.00	14.16	-45.20	13.39	12.66
G02_0_1	2024-02-04 00:00:30	13.94	14.06	10.19	-45.14	13.46	12.71
G02_0_1	2024-02-04 00:01:00	13.99	14.11	12.89	-45.09	13.53	12.75
G02_0_1	2024-02-04 00:01:30	14.06	14.18	12.10	-45.02	13.60	12.80
G02_0_1	2024-02-04 00:02:00	14.13	14.24	17.11	-44.96	13.68	12.85

The contrast between `code_tec` and `phase_tec` is the practical version of the theory above. Code STEC is noisy but absolute after bias correction. Phase STEC is precise but shifted by ambiguity. `leveled_tec` is the phase-derived observable placed on the code scale.

4.5 Inspecting STEC arcs

Before plotting a STEC series, it is useful to look at the arcs that survived the preprocessing. A long continuous arc is easier to interpret than a very short arc near a cycle slip.

```
arc_counts = obs_tec_tecu.groupby(level="sv").size().sort_values()
selected_arc = arc_counts.idxmax()

fig = viz.plot_sv_stats(
    arc_counts,
    "n_obs",
    kind="barh",
    title="Observation count per satellite arc",
```

```
    xlabel="Number of observations",
    ylabel="Satellite arc",
    figsize=(8, 7),
)

print(f"Selected arc for the STEC plot: {selected_arc}")
```

```
Selected arc for the STEC plot: G21_0_1
```

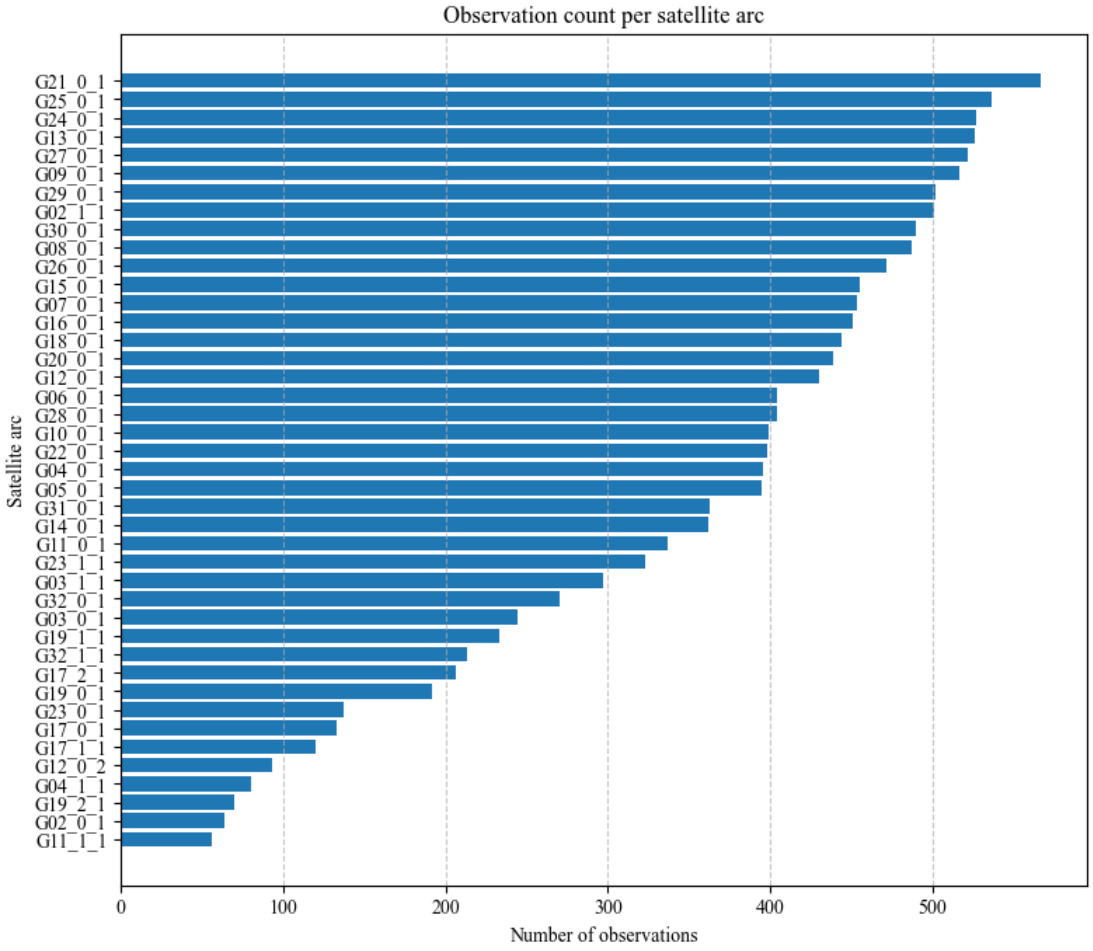


Figure 4.1: Number of usable observations per satellite arc after preprocessing.

`viz.plot_stec` shows the same arc through several lenses: code STEC, phase STEC, levelled STEC and smoothed variants. This is not just a pretty plot. It is a sanity check that cycle slip detection and leveling produced a physically plausible arc. The series drawn on each axis are configurable through `left_series` and `right_series`; below we pick the colours explicitly. The time axis is the `LT` column of the output: local solar time, i.e. GPST plus 4 minutes per degree of the station longitude.

```
stec_series = [
    {"col": "code_tec", "label": "Code STEC", "color": "tab:blue", "y_label": "CODE STEC [TECU]
    "},
    {"col": "leveled_tec", "label": "Phase levelled STEC", "color": "tab:green"},
    {"col": "leveled_tec_sg", "label": "S-G filter smoothed STEC", "color": "orange"},
]

fig = viz.plot_stec(obs_tec_tecu, selected_arc, time_col="LT", left_series=stec_series)
```

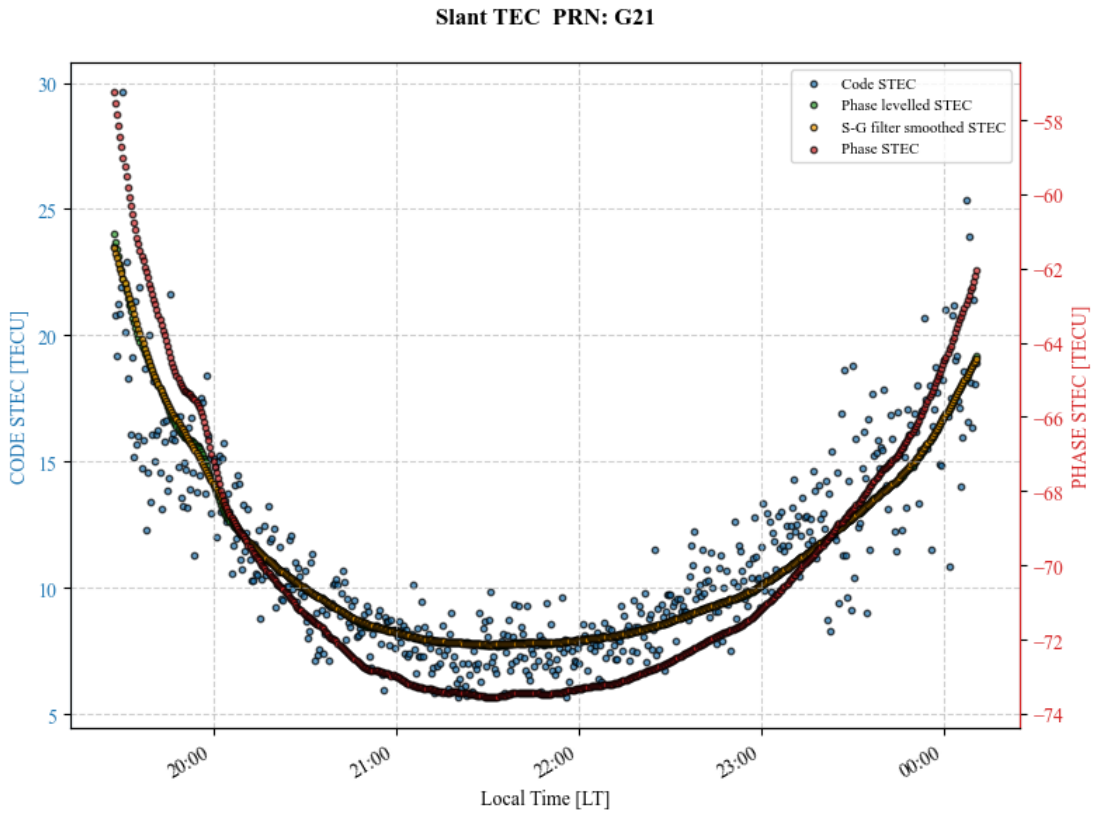


Figure 4.2: Selected STEC arc showing code, phase, levelled and smoothed TEC estimates.

4.6 Estimating VTEC above the station

With STEC measured, corrected for satellite and receiver biases, and levelled with phase observations, we can estimate a rough VTEC series above the station. The helper `VTECZenithEstimator` uses the thin-shell mapping function and selects the highest-elevation observations at each epoch. Then it interpolates and smooths the resulting series. The estimator is a small tutorial helper kept in `tools.py`; the figure is composed from two `gnx_py.viz` calls: `viz.plot_stec_arcs` draws the STEC of every satellite and `viz.plot_vtec_zenith` lays the zenith VTEC line over the same axes.

This is intentionally presented as a teaching tool, not as the most rigorous VTEC estimator. It helps connect the geometry from the first half of the notebook with a concrete time series above one receiver.

```
estimator = VTECZenithEstimator(
    df=obs_tec_tecu,
    time_col="LT",
    elev_col="ev",
    stec_col="leveled_tec",
    sv_col="sv",
    min_elev_deg=30,
    resample_freq="5min",
    smooth_method="whittaker",
    smooth_params={"lam": 50, "order": 2},
)

df_vtec, zen = estimator.run()

vtec_title = f"STEC measured at station {STATION} and rough zenith VTEC"
fig = viz.plot_stec_arcs(
    obs_tec_tecu,
    value_col="leveled_tec",
    time_col="LT",
```

```
    group_by="prn",
    legend=False,
    figsize=(8, 5),
)
fig = viz.plot_vtec_zenith(
    zen,
    time_col="LT",
    ax=fig.axes[0],
    color="black",
    lw=2,
    label="Smoothed VTEC",
    title=vtec_title,
    xlabel="Local time",
    ylabel="TEC [TECU]",
    legend_ncol=2,
    legend_fontsize=7,
)
```

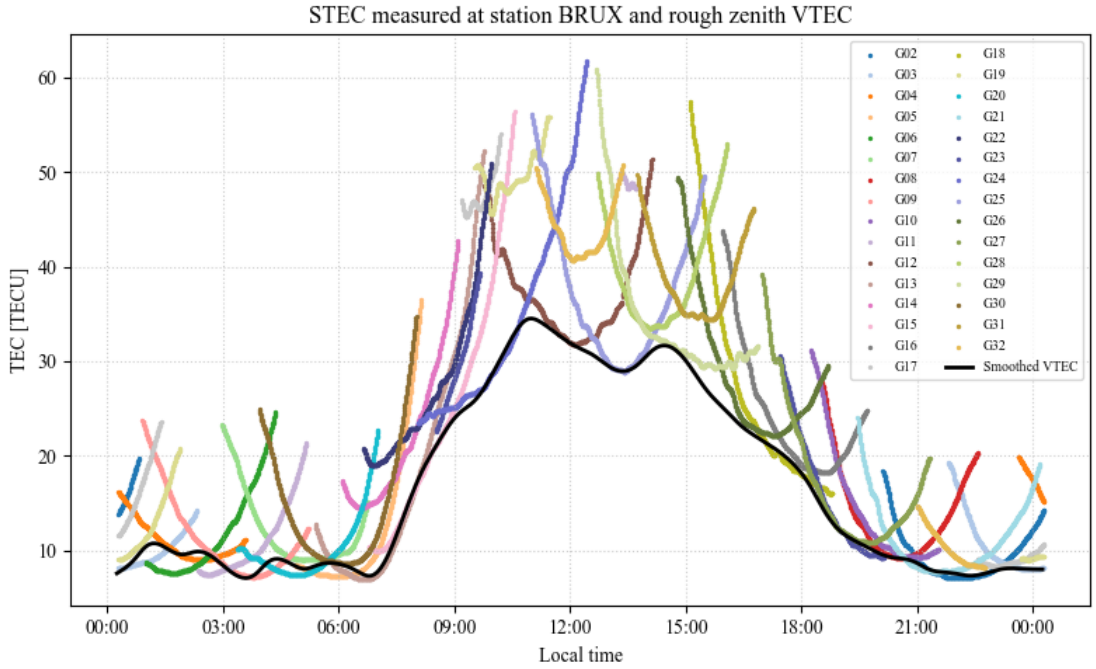


Figure 4.3: Measured STEC links and a rough zenith VTEC estimate above the station.

```
viz.df_head(zen, nrows=5, ncols=7, floatfmt=".2f", truncate_str=14)
```

LT	vtec	ev	M	nsat	vtec_smooth
2024-02-04 00:15:00	8.04	80.21	1.01	5.00	7.58
2024-02-04 00:20:00	8.03	78.56	1.02	5.00	7.79
2024-02-04 00:25:00	8.04	76.46	1.02	5.00	8.02
2024-02-04 00:30:00	8.03	74.31	1.03	5.00	8.26
2024-02-04 00:35:00	8.06	72.15	1.04	5.00	8.55

In the `zen` dataframe, `vtec` is the mapped value at the selected high-elevation link, while `vtec_smooth` is the smoothed zenith series. The result should be interpreted gently: for a single station and a single day of data this is more of an intuition-building exercise than a final ionosphere product.

4.7 Comparing measured STEC with models

At the beginning we asked `TECConfig` for the GIM as the background model (`ion`) and for Klobuchar and NTCM comparison columns. Both broadcast models take their coefficients from the navigation file; this RINEX 3 file has one GPS and one Galileo set in its header, used for the whole day. Now we can compare those modelled values with measured `leveled_tec`. The model columns were returned in metres of L1 delay, so we use the TECU-converted display columns created earlier: `ion_tecu`, `klobuchar_tecu` and `ntcm_tecu`.

```
model_cols = compact_columns(obs_tec_tecu, ["ion_tecu", "klobuchar_tecu", "ntcm_tecu"])
model_names = [
    {"ion_tecu": "GIM", "klobuchar_tecu": "Klobuchar", "ntcm_tecu": "NTCM G", "bdgim_tecu": "
    BDGIM"}[column]
    for column in model_cols
]

comparator = viz.STECModelComparator(
    df=obs_tec_tecu,
    x_col="leveled_tec",
    model_cols=model_cols,
    model_names=model_names,
    scale=1.0,
    gridsize=50,
    mincnt=1,
)

## report() returns the statistics table and both figures (hexbin, error histogram).
stats_table, hexbin_fig, hist_fig = comparator.report(
    title="Modelled vs measured STEC",
    show_hex=True,
    show_hist=True,
)
```

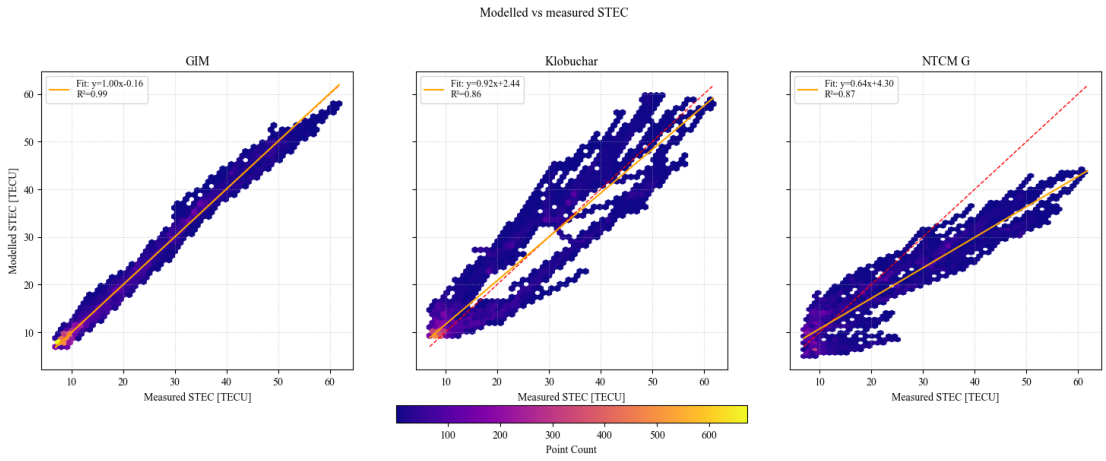


Figure 4.4: Measured STEC compared with ionospheric model estimates.

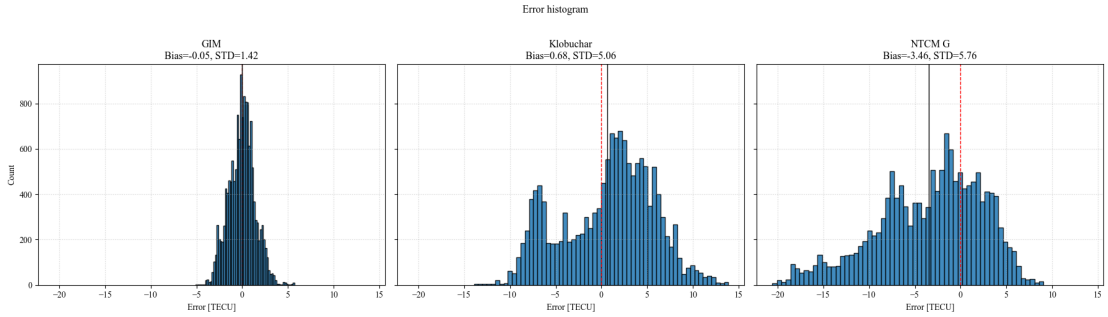


Figure 4.5: Model-comparison residuals for the selected STEC processing example.

```
viz.df_head(stats_table, ncols=10, floatfmt=".2f")
```

Model	N	Bias	STD	RMS	MAE	Corr	R2
GIM	14510	-0.05	1.42	1.42	1.12	0.99	0.99
Klobuchar	14510	0.68	5.06	5.11	4.28	0.92	0.86
NTCM G	14510	-3.46	5.76	6.72	5.15	0.93	0.87

PDF preview: 2 column(s) omitted from this wide table.

The point of this comparison is not to declare a global winner from one station and one day of data. It is to learn how to read the diagnostics. A good model should stay near the line $y = x$, have a small bias, and show a relatively compact error histogram. A GIM product will often agree well when its own DCB information participates in the processing chain, so the bias-source diagnostics are part of the interpretation.

4.7.1 BDGIM from a RINEX 4 navigation file

BDGIM is the global ionosphere model broadcast by BeiDou-3 in its B-CNAV messages (BDS-SIS-ICD-B2b-1.0, section 7.7): nine coefficients, each set tagged with its transmission time, so a daily file holds many sets. GNX-py reads them from the `> ION Cxx CNVX` records of a RINEX 4 navigation file, such as `BRD400DLR_S_20240350000_01D_MN.rnx` in `sample_data`. Records that fail the check against the coefficient grid and ranges of the ICD, filler records among them, are rejected (with a warning when BeiDou is the processed system or BDGIM is the background `ionosphere_model`; the kriging chapter prints such warnings), and each epoch takes the last plausible set broadcast before it. Because the model is global, GNX-py evaluates it for GPS, Galileo and BeiDou alike. `ionosphere_model="bdgim"`, or `"bdgim"` in `compare_models`, needs such a RINEX 4 file and raises a readable error otherwise; a negative model value is set to 0 in the session columns.

The rule of the set in force at each epoch also applies to the GPS Klobuchar and Galileo NTCM coefficients of a RINEX 4 file when they are the background `ionosphere_model`. The `compare_models` columns `klobuchar` and `ntcm` use one set of the file, the first plausible one.

Below we repeat the GPS run with the RINEX 4 file and request only the BDGIM column; the background `ion` is still the GIM.

```
bdgim_config = build_tec_config(
    system="G",
    time_limit=GPS_TLIM,
    compare_models=("bdgim",),
    nav_path=NAV4_PATH,
)
bdgim_tec, bdgim_warnings = run_tec(bdgim_config, use_sys="G")
bdgim_tec_tecu = with_tecu_units(bdgim_tec)
for message in bdgim_warnings:
    print(f"warning: {message}")

bdgim_comparator = viz.STECModelComparator(
    df=bdgim_tec_tecu,
```

```
    x_col="leveled_tec",
    model_cols=["ion_tecu", "bdgim_tecu"],
    model_names=["GIM", "BDGIM"],
)
## statistics only (Bias = model - measured); the figures would look like the ones above
viz.df_head(bdgim_comparator.compute_stats(), ncols=10, floatfmt=".2f")
```

```
warning: GPS satellites without a usable broadcast ephemeris (left out): G01 (unhealthy, health
    =63).
```

Model	N	Bias	STD	RMS	MAE	Corr	R2
GIM	14510	-0.05	1.42	1.42	1.12	0.99	0.99
BDGIM	14510	-7.76	4.87	9.16	7.97	0.95	0.90

PDF preview: 2 column(s) omitted from this wide table.

BDGIM follows the daily course of the measured STEC, but it sits several TECU below it on average (the `Bias` column). This is a property of the broadcast coefficients, not of the implementation: the global mean of the BDGIM map equals its first broadcast coefficient, which on 2024-035 is about 8 TECU (7-9 TECU) below the global mean of the CODE GIM. Broadcast models serve single-frequency users in real time; for absolute STEC the calibrated measurement remains the reference.

4.8 Bias policy diagnostics

STEC can look convincing even when the wrong bias convention was used. For this reason the current GNX-py output includes bias-source columns. Let us summarize them for the GPS run.

```
bias_diag_cols = compact_columns(
    obs_tec_tecu,
    [
        "stec_sat_bias_source", "stec_station_bias_source",
        "stec_bias_code_1", "stec_bias_code_2",
        "stec_sat_bias_m", "stec_station_bias_m", "stec_total_bias_m",
    ],
)

bias_summary = (
    obs_tec_tecu.reset_index()
    .groupby([col for col in ["stec_sat_bias_source", "stec_station_bias_source", "
    stec_bias_code_1", "stec_bias_code_2"] if col in obs_tec_tecu.columns])
    .size()
    .reset_index(name="rows")
)

viz.df_head(bias_summary, nrows=10, ncols=8, floatfmt=".3f", reset_index=False)
```

stec_sat bias_source	stec_station bias_source	stec_bias code_1	stec_bias code_2	rows
gim	gim	C1W	C2W	14510

The summary tells us which source was actually used. In the GPS run both sources are `gim`: with `rcv_dcb_source="gim"` the receiver DCB comes from the IONEX header, and GNX-py then takes the satellite DCBs from the same IONEX file, so that the satellite and receiver biases share one datum (`stec_bias_datum`). `product_dsb` means that the DCB/DSB product (`dcb_path`) provided a matching differential bias for the selected code pair, as in the BeiDou run below. If you see `config` or `zero`, that does not automatically mean the run is wrong; it means the interpretation must include that fallback.

4.9 BeiDou B1IB3I STEC smoke example

BeiDou is supported in the current ionosphere workflow. The safest default pair for the sample data is `B1IB3I`, which maps to the legacy B1I/B3I observable pair. This mirrors the caution from the SIS

tutorial: modern BDS signals can be configured when the observations and bias products support them, but they require validation before scientific use.

The BDS run below uses `BDS_TLIM`, so it covers 00:00-00:20 GPST. Its purpose is to show that the same `TECConfig` pattern works for `system="C"`, and that the output carries the same STEC and bias diagnostics. Read the printed warnings: with broadcast orbits GNX-py skips the BeiDou GEO satellites, and the CODE GIM carries no BeiDou receiver DCB, so the station bias comes from the next source of `stec_bias_sources`. That last message appears twice: GNX-py writes it to the log (the line printed above the output) and also raises it as a warning, which `run_tec` prints. The first warning comes from the navigation reader: the header of this RINEX 3 file holds a BeiDou Klobuchar set (`BDSA`/`BDSB`) for every transmitting satellite, and the set of C50 fails the plausibility check (its amplitude has no daytime term), so it is rejected and the other sets remain. This run uses no broadcast ionosphere model, so the rejection does not change its STEC; the reader reports it because BeiDou is the processed system.

```
bds_config = build_tec_config(
    system="C",
    time_limit=BDS_TLIM,
    compare_models=False,
    ev_mask=20,
)
bds_tec, bds_warnings = run_tec(bds_config, use_sys="C")
bds_tec_tecu = with_tecu_units(bds_tec)

print(f"rows={len(bds_tec)}")
print(f"BDS satellite arcs={bds_tec.index.get_level_values('sv').nunique()}")
print(f"unique captured warnings={len(bds_warnings)}")
for message in bds_warnings:
    print(f"warning: {message}")

viz.df_head(
    bds_tec_tecu[compact_columns(
        bds_tec_tecu,
        [
            "leveled_tec", "code_tec",
            "stec_sat_bias_source", "stec_station_bias_source",
            "stec_bias_code_1", "stec_bias_code_2",
        ],
    )],
    nrows=5,
    ncols=8,
    floatfmt=".2f",
    truncate_str=14,
)
```

```
rcv_dcb_source='gim': the GIM gives no receiver DCB for BRUX (BeiDou, C2I-C6I); the station
    bias comes from the next source of stec_bias_sources ('product', 'config', 'zero')


rows=410
BDS satellite arcs=10
unique captured warnings=3
warning: sample_data/BRDC00IGS_R_20240350000_01D_MN.rnx: broadcast ionosphere record header
    BDSA C50 rejected: BeiDou Klobuchar amplitude A2 <= 0 for every |latitude| <= 1/3
    semicircle (no daytime term)
warning: BeiDou broadcast interpolation currently supports BDS MEO/IGSO only; skipping GEO
    satellites: C01, C02, C03, C04, C05, C59, C60, C62.
warning: rcv_dcb_source='gim': the GIM gives no receiver DCB for BRUX (BeiDou, C2I-C6I); the
    station bias comes from the next source of stec_bias_sources ('product', 'config', 'zero')
```

sv	time	leveled_tec	code_tec	stec_sat bias_source	stec_station bias_source	stec_bias code_1	stec_bias code_2
C06_0_1	2024-02-04 00:00:00	5.64	0.56	product_dsb	product_dsb	C2I	C6I
C06_0_1	2024-02-04 00:00:30	5.65	-5.29	product_dsb	product_dsb	C2I	C6I
C06_0_1	2024-02-04 00:01:00	5.66	1.26	product_dsb	product_dsb	C2I	C6I
C06_0_1	2024-02-04 00:01:30	5.63	6.13	product_dsb	product_dsb	C2I	C6I
C06_0_1	2024-02-04 00:02:00	5.58	3.54	product_dsb	product_dsb	C2I	C6I

```
bds_bias_summary = (
    bds_tec_tecu.reset_index()
    .groupby([col for col in ["stec_sat_bias_source", "stec_station_bias_source", "
    stec_bias_code_1", "stec_bias_code_2"] if col in bds_tec_tecu.columns])
    .size()
    .reset_index(name="rows")
)

viz.df_head(bds_bias_summary, nrows=10, ncols=8, floatfmt=".3f", reset_index=False)
```

stec_sat bias_source	stec_station bias_source	stec_bias code_1	stec_bias code_2	rows
product_dsb	product_dsb	C2I	C6I	410

For BDS `B1IB3I`, the diagnostic code pair should correspond to the legacy BDS observables with which GNX-py was validated. If you change `bds_freq` to a modern pair such as `B1C`, `B2a` or `B2b`, first check whether the observation file and bias product contain the matching code observables. During validation runs, `stec_missing_bias="raise"` is safer than quietly falling back to zero.

4.10 What to try next

This concludes the introductory ionosphere tutorial. You have seen the theory behind STEC, the geometry-free combination, bias handling, phase-to-code leveling, a rough VTEC estimate, model comparison including BDGIM, and a BeiDou `B1IB3I` run.

For your own experiments, I recommend changing only one thing at a time:

- change `GPS_TLIM` to a shorter window and inspect whether the same arcs remain stable,
- switch `system` between `G`, `E` and `C`, remembering that `TECSession` processes one constellation at a time,
- test `stec_missing_bias="raise"` to catch missing bias products early,
- use `ionosphere_model="bdgim"` with `nav_path=NAV4_PATH` as the background model,
- compare `leveled_tec`, `median_leveled_tec`, `leveled_tec_sg` and `code_tec`,
- move to the calibration notebook when you want to estimate receiver DCB instead of using GIM/product values.

The next chapters work with a network of stations: ionospheric activity indexes (SIDX, ROTI, GIX) and regional VTEC maps with kriging.

Chapter 5

Ionosphere Calibration

5.1 Receiver calibration

In this tutorial, I will discuss the issue of calibrating GNSS receivers for TEC measurement. Several scientific articles have been devoted to this topic, and they are worth reading for a deeper understanding of the subject (Sardón et al. 1994; Arikan et al. 2008). Here we will keep the same style as in the previous ionosphere notebook: first the observation model, then the intuition, then a small GNX-py example.

The practical question is simple: when we form a geometry-free code combination, the ionospheric delay is exposed, but so are satellite and receiver hardware delays. If the receiver delay is unknown, the STEC time series can be shifted by a constant amount. For relative short-term variability that may be acceptable; for absolute STEC it is not.

5.2 Why receiver DCB matters

Let us start from code observations at two frequencies:

$$P_1 = \rho + c(\delta t_r - \delta t_s) + T + I_1 + b^P_{r,1} + b^P_{s,1} + \varepsilon_{P_1} \quad [m]$$

$$P_2 = \rho + c(\delta t_r - \delta t_s) + T + I_2 + b^P_{r,2} + b^P_{s,2} + \varepsilon_{P_2} \quad [m]$$

and the geometry-free code combination:

$$P_{\mathrm{GF}} = P_1 - P_2 = (I_1 - I_2) + (b^P_{r,1} - b^P_{r,2}) + (b^P_{s,1} - b^P_{s,2}) + (\varepsilon_{P_1} - \varepsilon_{P_2}) \quad [m].$$

where:

- P_1, P_2 are code observations on two frequencies,
- P_{GF} is the geometry-free code combination,
- I_n is the ionospheric delay in metres at frequency n,
- $b^P_{r,n}$ and $b^P_{s,n}$ are receiver and satellite hardware delays.

The non-dispersive range, clocks and troposphere disappeared, but the hardware delays remained. In other words, the geometry-free combination is exactly the place where the ionosphere becomes visible and where DCB errors become dangerous.

During the generation of global ionosphere maps, receiver and satellite biases are estimated as part of the state vector. That is why IONEX/GIM files often contain satellite and receiver DCB entries in the header. You can also find SINEX BIAS products with `.BIA` or `.BSX` extensions, containing DSB or OSB values for satellites and sometimes receivers (Schaer 2018). What we need for absolute STEC is the bias in the same geometry-free convention used by our selected signal pair.

Current version note. GNX-py now exposes this explicitly through the STEC bias policy. `TECConfig.stec_bias_sources` chooses the fallback order, while output columns such as `stec_sat_bias_source`, `stec_station_bias_source`, `stec_bias_code_1` and `stec_bias_code_2` tell us which convention was actually used.

5.3 Calibration as model alignment

The receiver calibration method used by GNX-py can be summarized as levelling the measured geometry-free ionosphere to a background model. This is similar in spirit to phase-to-code levelling from the previous notebook, but now the unknown constant is the receiver DCB.

In the weighted-average method, GNX-py uses a background GIM value and satellite DCB information. For each satellite link above an elevation cutoff, it forms an epoch-wise receiver DCB candidate. In simplified notation:

$$\hat{d}_r(t,s) = I_s^{\mathrm{model}}(t) - \left(L_4(t,s)/c + d_{12}^{sat}(s)\right),$$

where L_4 is the levelled geometry-free phase observable expressed as a time delay, d_{12}^{sat} is the satellite differential code bias in seconds, and I^{model} is the modelled ionospheric delay converted to the same convention.

The epoch value is an elevation-weighted average:

$$\hat{d}_r(t) = \frac{\sum_s w_s(t)\hat{d}_r(t,s)}{\sum_s w_s(t)}, \qquad w_s(t) = \sin^4(E_s(t)).$$

Finally, the receiver DCB is taken as a robust median over epochs:

$$\hat{d}_r = \mathrm{median}_t\left(\hat{d}_r(t)\right).$$

The exact sign convention follows GNX-py's internal `P4/L4` convention and the selected signal pair. The important practical point is consistency: GIM model, satellite bias, receiver bias and geometry-free observable must all be interpreted in the same convention.

This method is model-dependent. If the background GIM is biased, the receiver DCB estimate will inherit part of that bias. That is why GIM is a reasonable background for this tutorial, while Klobuchar or NTCM would be too rough for calibration.

5.4 PPP-UDUC as an advanced DCB source

The GIM-based weighted calibration can also be compared with a receiver DCB estimated by PPP-UDUC. The PPP output such an estimate comes from is not part of `sample_data`, so the PPP comparison is kept here as a workflow note rather than a mandatory executable cell.

The idea remains useful. In uncombined PPP, receiver hardware delays can be part of the estimated state. If one observable-specific bias is fixed as a reference, the second one can be interpreted as a DCB-like quantity for the receiver. A small post-processing step (not part of GNX-py) can turn such PPP output into a station DCB table. You can then pass such a value back into `TECConfig` through `rcv_dcb_source="defined"` and `define_station_dcb=...`.

This PPP route can be more robust than a simple GIM alignment for some stations, but it is also more complex and still depends on the chosen ionospheric constraints, bias products and PPP convergence. In this tutorial we keep the executable part focused on the current GIM-based calibration path and show how a user-defined DCB would be injected.

5.5 Current GNX-py calibration workflow

In the current API, the automatic receiver calibration path is activated through `TECConfig`:

```
add_sta_dcb=True
rcv_dcb_source="calibrate"
```

Then `TECSession` levels the arcs twice, with the calibration between the two passes:

1. It levels the arcs with the satellite biases of `stec_bias_sources` and without a station bias (a preliminary STEC).
2. It runs `Calibration` against the GIM background to estimate a receiver DCB, with the same satellite biases, so the receiver DCB is in their datum; samples below `calibration_min_elev` (30 deg by default) are left out.
3. It stores the estimate in `sta_dcb` and levels the arcs again (second pass) with the normal bias policy, now including the station bias.

Other receiver-bias choices are also available:

`rcv_dcb_source`	meaning
`"calibrate"`	estimate station DCB from measured STEC and GIM background (the default of `rcv_dcb_source`; it takes effect with `add_sta_dcb=True`)
`"gim"`	use receiver DCB parsed from GIM/IONEX auxiliary data
`"defined"`	use `define_station_dcb` from the configuration
`"none"`	do not apply station DCB

This makes the calibration notebook part of the same bias-policy story as the introduction notebook. We do not hide the chosen DCB source; we inspect it.

5.6 Notebook setup

The notebook uses the station `BRUX` from `sample_data`, so it runs from a fresh clone of the repository. The calculations avoid local absolute paths and do not write output files. Figures and table previews come from `gnx_py.viz`, the plotting package of GNX-py.

```
from __future__ import annotations

from datetime import datetime
from pathlib import Path
import contextlib
import io
import os
import warnings

import numpy as np
import pandas as pd

import gnx_py as gnx
from gnx_py.core.gnss import frequency_hz, mode_signals
from gnx_py.ionosphere import TECConfig, TECSession


def find_repo_root(start: Path) -> Path:
    for candidate in [start.resolve(), *start.resolve().parents]:
        if (candidate / "pyproject.toml").exists() and (candidate / "gnx_py").is_dir():
            return candidate
    raise FileNotFoundError(
        "Run this notebook from inside the cloned GNX-py repository, "
        "or set REPO_ROOT manually to the repository directory."
```

```
    )


REPO_ROOT = find_repo_root(Path.cwd())
SAMPLE_DATA = REPO_ROOT / "sample_data"

## Only repository-relative paths are printed, so no machine-specific path ends up in the output.
local_checkout = Path(gnx.__file__).resolve().parent == (REPO_ROOT / "gnx_py").resolve()
print(f"gnx_py imported from: "
      f"{'gnx_py/ of this repository' if local_checkout else 'an installed package'}")
print(f"sample data: {SAMPLE_DATA.relative_to(REPO_ROOT)}")
```

```
gnx_py imported from: gnx_py/ of this repository
sample data: sample_data
```

```
from gnx_py import viz

## Notebook-wide figure style: serif fonts and Computer Modern mathtext.
viz.apply_publication_style(font_scale=1.3, mathtext="cm")
```

```
STATION = "BRUX"
SYSTEM = "G"
GPS_MODE = "L1L2"
DOY = 35
C = 299_792_458.0

OBS_PATH = SAMPLE_DATA / "BRUX00BEL_R_20240350000_01D_30S_MO.crx.gz"
NAV_PATH = SAMPLE_DATA / "BRDC00IGS_R_20240350000_01D_MN.rnx"
SP3_PATHS = [
    SAMPLE_DATA / "GRG0MGXFIN_20240340000_01D_05M_ORB.SP3",
    SAMPLE_DATA / "GRG0MGXFIN_20240350000_01D_05M_ORB.SP3",
    SAMPLE_DATA / "GRG0MGXFIN_20240360000_01D_05M_ORB.SP3",
]
ATX_PATH = SAMPLE_DATA / "igs20.atx"
DCB_PATH = SAMPLE_DATA / "CAS1OPSRAP_20240350000_01D_01D_DCB.BIA"
GIM_PATH = SAMPLE_DATA / "COD0OPSFIN_20240350000_01D_01H_GIM.INX"
PPP_DCB_TABLE = SAMPLE_DATA / "dcb_table.csv"

## time_limit [start, end] (GPST, inclusive) is applied while the observation file is read. The calibration
## uses every sample of this interval above calibration_min_elev, so we give it the whole day of sample data.
TLIM = [datetime(2024, 2, 4, 0, 0), datetime(2024, 2, 4, 23, 59, 30)]

required_paths = [OBS_PATH, NAV_PATH, ATX_PATH, DCB_PATH, GIM_PATH, PPP_DCB_TABLE, *SP3_PATHS]
missing = [path for path in required_paths if not path.exists()]
if missing:
    raise FileNotFoundError("Missing sample files:\n" + "\n".join(str(path.relative_to(REPO_ROOT)) for path in missing))
```

```
def run_tec(config: TECConfig, use_sys: str = SYSTEM):
    """Run TECSession while keeping notebook output compact."""
    stream = io.StringIO()
    with warnings.catch_warnings(record=True) as captured, contextlib.redirect_stdout(stream):
        warnings.simplefilter("always")
        result = TECSession(config=config, use_sys=use_sys).run()
    # messages name input files by their full path: show them relative to the repository
    root = f"{REPO_ROOT}{os.sep}"
    warning_messages = list(dict.fromkeys(str(item.message).replace(root, "") for item in captured))
    return result, warning_messages
```

```

def build_calibration_config(
    *,
    rcv_dcb_source: str = "calibrate",
    define_station_dcb=None,
    add_sta_dcb: bool = True,
) -> TECConfig:
    return TECConfig(
        obs_path=OBS_PATH,
        nav_path=NAV_PATH,
        sp3_path=SP3_PATHS,
        atx_path=ATX_PATH,
        dcb_path=DCB_PATH,
        gim_path=GIM_PATH,
        sys=SYSTEM,
        gps_freq=GPS_MODE,
        gal_freq="E1E5a",
        bds_freq="B1IB3I",
        orbit_type="broadcast",
        ionosphere_model="gim",
        troposphere_model=False,
        time_limit=TLIM,
        day_of_year=DOY,
        station_name=STATION,
        screen=False,
        ev_mask=30,
        windup=True,
        rel_path=False,
        sat_pco="los",
        rec_pco=True,
        add_dcb=True,
        add_sta_dcb=add_sta_dcb,
        rcv_dcb_source=rcv_dcb_source,
        define_station_dcb=define_station_dcb,
        stec_bias_sources=("product", "gim", "config", "zero"),
        stec_missing_bias="warn_zero",
        compare_models=False,
        use_iono_rms=True,
        min_arc_len=2,
    )


def tec_factor_for_mode(mode: str) -> float:
    sig1, sig2 = mode_signals(mode)
    f1, f2 = frequency_hz(sig1), frequency_hz(sig2)
    return (1 / 40.3) * ((f1**2 * f2**2) / (f1**2 - f2**2))


TEC_FACTOR = tec_factor_for_mode(GPS_MODE)
NS_TO_TECU = C * 1e-9 * TEC_FACTOR / 1e16


def to_tecu(series: pd.Series) -> pd.Series:
    return series / 1e16
```

5.7 Weighted-average receiver calibration

We now run the current GNX-py calibration path. The important configuration choices are `add_sta_dcb =True` and `rcv_dcb_source="calibrate"`. The result dataframe should contain both the GIM receiver bias (`sta_bias`) and the calibrated receiver bias (`sta_dcb`).

```
calibration_config = build_calibration_config(rcv_dcb_source="calibrate")
obs_tec, calibration_warnings = run_tec(calibration_config)

print(f"rows={len(obs_tec)}")
print(f"satellite arcs={obs_tec.index.get_level_values('sv').nunique()}")
print(f"unique captured warnings={len(calibration_warnings)}")
for message in calibration_warnings:
    print(f"warning: {message}")
```

```
rows=14510
satellite arcs=42
unique captured warnings=1
warning: GPS satellites without a usable broadcast ephemeris (left out): G01 (unhealthy, health
    =63).
```

```
calibrated_bias_ns = float(obs_tec["sta_dcb"].dropna().iloc[0])
gim_bias_ns = float(obs_tec["sta_bias"].dropna().iloc[0])
bias_error_ns = gim_bias_ns - calibrated_bias_ns

bias_table = pd.DataFrame(
    [
        {"source": "GIM / IONEX header", "receiver_dcb_ns": gim_bias_ns},
        {"source": "GNX weighted calibration", "receiver_dcb_ns": calibrated_bias_ns},
    ]
)
bias_table["difference_to_gim_ns"] = bias_table["receiver_dcb_ns"] - gim_bias_ns
bias_table["stec_shift_vs_calibrated_tecu"] = (
    bias_table["receiver_dcb_ns"] - calibrated_bias_ns
) * NS_TO_TECU

viz.df_head(bias_table, nrows=5, ncols=4, floatfmt=".3f", reset_index=False)
print(f"Calibration difference for station {STATION}: {bias_error_ns:.3f} ns (GIM - calibrated)
    ")
```

source	receiver_dcb ns	difference to_gim_ns	stec_shift vs calibrated tecu
GIM / IONEX header	7.850	0.000	0.003
GNX weighted calibration	7.849	-0.001	0.000

```
Calibration difference for station BRUX: 0.001 ns (GIM - calibrated)
```

A difference below about one nanosecond is generally a good sign for this simple method. Keep in mind that both numbers are tied to the same CODE solution: the CODE GIM is the background of the calibration, and its header holds the GIM receiver DCB. Close agreement is therefore expected here; a calibration against another GIM is a more independent check. In STEC units, the same difference is not abstract: for GPS L1/L2 it maps to a nearly constant shift in TECU. This is why receiver calibration matters even when the shape of the STEC series looks correct.

```
diagnostic_cols = [
    "sta_bias", "sta_dcb", "stec_station_bias_source", "stec_sat_bias_source",
    "stec_bias_code_1", "stec_bias_code_2", "stec_station_bias_m",
    "stec_sat_bias_m", "stec_total_bias_m",
]
diagnostic_cols = [column for column in diagnostic_cols if column in obs_tec.columns]
viz.df_head(obs_tec[diagnostic_cols], nrows=5, ncols=11, floatfmt=".3f", truncate_str=16)
```

sv	time	sta_bias	sta_dcb	stec_station bias_source	stec_sat bias_source	stec_bias code_1	stec_bias code_2
G02_0_1	2024-02-04 00:00:00	7.850	7.849	calibrate	product_dsb	C1W	C2W
G02_0_1	2024-02-04 00:00:30	7.850	7.849	calibrate	product_dsb	C1W	C2W
G02_0_1	2024-02-04 00:01:00	7.850	7.849	calibrate	product_dsb	C1W	C2W
G02_0_1	2024-02-04 00:01:30	7.850	7.849	calibrate	product_dsb	C1W	C2W
G02_0_1	2024-02-04 00:02:00	7.850	7.849	calibrate	product_dsb	C1W	C2W

PDF preview: 3 column(s) omitted from this wide table.

The diagnostic columns confirm which path was used. For the calibrated run, `stec_station_bias_source` should be `calibrate`. The satellite biases come from the first source of `stec_bias_sources` that has the code pair, here the CAS DSB product (`product_dsb`); the calibration used the same satellite biases, so receiver and satellite biases share one datum.

5.8 Comparing calibrated, GIM and user-defined DCB

Instead of reprocessing STEC once per receiver DCB (calibrated, GIM, PPP-UDUC), which would triple the run time, we use the calibrated STEC as the base series and compute the constant STEC shift implied by alternative receiver DCB values. This is equivalent for illustrating the main effect: changing a receiver DCB shifts the absolute STEC level.

The formula used in the next cell is:

$$\Delta\mathrm{STEC}[\mathrm{TECU}] = \Delta d_r[\mathrm{ns}] \cdot c \cdot 10^{-9} \cdot \frac{K_{GF}}{10^{16}},$$

where K_{GF} is the geometry-free conversion factor for the selected frequency pair.

```
arc_counts = obs_tec.groupby(level="sv").size().sort_values()
selected_arc = arc_counts.idxmax()

## Keep the (sv, time) MultiIndex so that stec.loc[selected_arc] selects the arc.
stec = obs_tec.loc[[selected_arc], ["leveled_tec"]].rename(columns={"leveled_tec": "calibrated"
    })
stec["calibrated"] = to_tecu(stec["calibrated"])
stec["gim_header"] = stec["calibrated"] + (gim_bias_ns - calibrated_bias_ns) * NS_TO_TECU
stec["no_station_dcb"] = stec["calibrated"] + (0.0 - calibrated_bias_ns) * NS_TO_TECU

fig = viz.plot_stec(
    stec,
    selected_arc,
    time_col="time",
    left_series=[
        {"col": "calibrated", "label": "Weighted calibration", "color": "tab:red", "marker": "o
    ",
         "s": 5, "alpha": 1.0, "edgecolor": "none", "y_label": "STEC [TECU]", "y_color": "black
    "},
        {"col": "gim_header", "label": "GIM receiver DCB", "color": "black", "marker": "x",
         "s": 5, "alpha": 1.0, "edgecolor": None},
        {"col": "no_station_dcb", "label": "No station DCB", "color": "tab:blue", "marker": "."
    ,
         "s": 5, "alpha": 1.0, "edgecolor": "none"},
    ],
    right_series=[],
    title=f"Receiver DCB effect on STEC for arc {selected_arc}",
    figsize=(9, 5),
)

arc_times = stec.index.get_level_values("time")
print(f"Selected arc: {selected_arc} ({arc_times.min():%H:%M}-{arc_times.max():%H:%M} GPST)")
```

```
Selected arc: G21_0_1 (19:10-23:53 GPST)
```

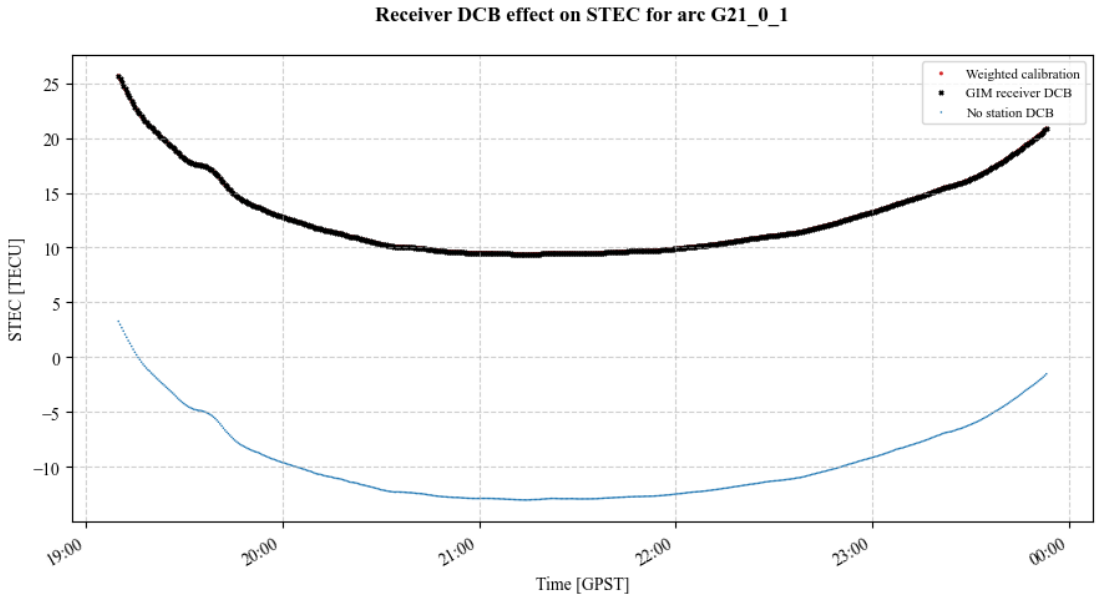


Figure 5.1: Effect of receiver DCB choices on the absolute STEC level for one satellite arc.

```
shift_table = pd.DataFrame(
    [
        {"series": "GIM header vs calibrated", "shift_tecu": (gim_bias_ns - calibrated_bias_ns)
     * NS_TO_TECU},
        {"series": "No station DCB vs calibrated", "shift_tecu": (0.0 - calibrated_bias_ns) *
    NS_TO_TECU},
    ]
)
viz.df_head(shift_table, nrows=5, ncols=2, floatfmt=".3f", reset_index=False)
```

series	shift_tecu
GIM header vs calibrated	0.003
No station DCB vs calibrated	-22.400

This plot is intentionally simple. The horizontal axis shows the `TECSession` epochs, which are GPS time (GPST), as in the RINEX observation file. `TLIM` covers the whole day, and the arc shown is the longest one of the run (its span is printed above). The plot shows that receiver DCB acts as an almost constant vertical offset in absolute STEC. That offset may be small when calibrated and GIM values agree, or very large if station DCB is omitted. The shape of the arc can remain similar, while the absolute TEC level becomes wrong.

5.9 PPP-UDUC comparison when available

The repository includes `sample_data/dcb_table.csv`, a compact table of PPP-derived receiver DCB values for the EPN stations of the network chapters. BRUX is not among them, so with the sample data the cell below only explains what to provide; with a station of your own in such a table it prints the comparison. A receiver DCB refers to the code pair of the processing that produced it (for GPS L1/L2, C1C-C2W or C1W-C2W differ by the receiver's C1C-C1W bias), so check that it matches the pair in `stec_bias_code_1` and `stec_bias_code_2` before you pass it.

```
ppp_table = pd.read_csv(PPP_DCB_TABLE)
ppp_match = ppp_table.loc[ppp_table["Station"].str.upper().eq(STATION)]

if ppp_match.empty:
    print(
        f"No PPP-derived DCB row for {STATION} in {PPP_DCB_TABLE.name}. "
```

```
        "To compare PPP-UDUC here, provide a station DCB value and pass it as
    define_station_dcb."
    )
else:
    ppp_bias_ns = float(ppp_match["gps_dcb_mean"].iloc[0])
    ppp_shift_tecu = (ppp_bias_ns - calibrated_bias_ns) * NS_TO_TECU
    ppp_summary = pd.DataFrame(
        [{
            "source": "PPP-UDUC table mean",
            "receiver_dcb_ns": ppp_bias_ns,
            "difference_to_calibrated_ns": ppp_bias_ns - calibrated_bias_ns,
            "stec_shift_vs_calibrated_tecu": ppp_shift_tecu,
        }]
    )
    viz.df_head(ppp_summary, nrows=1, ncols=4, floatfmt=".3f", reset_index=False)
```

```
No PPP-derived DCB row for BRUX in dcb_table.csv. To compare PPP-UDUC here, provide a station
    DCB value and pass it as define_station_dcb.
```

5.10 How to pass your own receiver DCB

When you have a trusted receiver DCB, for example from PPP-UDUC or from an external calibration product, you can pass it through `TECConfig` as a user-defined station bias. The value is in nanoseconds.

The cell below builds the configuration but does not run another full STEC session by default. It is meant as a clear template that you can enable after replacing `user_receiver_dcb_ns` with your own value.

```
user_receiver_dcb_ns = calibrated_bias_ns

defined_config = build_calibration_config(
    rcv_dcb_source="defined",
    define_station_dcb=user_receiver_dcb_ns,
    add_sta_dcb=True,
)

print(f"rcv_dcb_source={defined_config.rcv_dcb_source}")
print(f"define_station_dcb={defined_config.define_station_dcb:.3f} ns")

RUN_DEFINED_EXAMPLE = False
if RUN_DEFINED_EXAMPLE:
    obs_tec_defined, defined_warnings = run_tec(defined_config)
    print(f"defined run rows={len(obs_tec_defined)} warnings={len(defined_warnings)}")
```

```
rcv_dcb_source=defined
define_station_dcb=7.849 ns
```

For validation runs, consider setting `stec_missing_bias="raise"`. That makes missing bias products fail loudly instead of silently falling back to zero. During exploration, `warn_zero` is convenient; during scientific reporting, explicit failure is usually safer.

5.11 What to try next

This concludes the calibration tutorial. Its main idea: receiver DCB calibration is a levelling problem, and the selected ionosphere model matters. The executable part uses `sample_data`, `TECConfig` and explicit bias-policy diagnostics.

The main lessons are:

- a receiver DCB error produces an almost constant STEC offset,
- `rcv_dcb_source="calibrate"` estimates the station bias against the GIM background,

- `rcv_dcb_source="gim"` uses receiver DCB from the GIM/IONEX header,
- `rcv_dcb_source="defined"` is the path for PPP-derived or externally supplied DCB values,
- PPP-UDUC comparison is still relevant, but it needs a current PPP output or station DCB table for the same station.

The next two chapters use calibrated STEC of a whole network: the EPN stations in `tutorial/ionosphere/network` were processed with `rcv_dcb_source="calibrate"` against the CODE GIM, i.e. with the method of this chapter.

Chapter 6

Ionospheric Activity Indexes

6.1 Ionospheric activity indices

One of the ways in which we can use GNSS TEC measurements is to calculate ionospheric activity indices. GNX-py implements the calculation and visualisation of SIDX (Sudden Ionosphere Disturbance Index), GIX (Gradient Ionosphere Index), and ROT/ROTI (Rate of TEC, Rate of TEC Index) indices. These indices are described in detail in the following works: (Jakowski and Hoque 2019; Cherniak et al. 2018; Carmo et al. 2021). In this tutorial, we will discuss how to calculate them and show how to monitor ionospheric activity using GNX-py.

The examples below use measured STEC files from the local `network/` folder: one file per station and system for 38 EPN stations on 2024-035 (GPS L1/L2 and Galileo E1/E5a, 30 s sampling, elevation mask 30 deg), computed with `TECSession` of this GNX-py version, with precise CODE orbits and the receiver DCB calibrated against the CODE GIM (`rcv_dcb_source="calibrate"`, the method of the calibration chapter), and with levelling windows of 50 samples instead of the whole-arc levelling of the introduction (`leveling_ws=50`; `network/README.md` lists all settings). The folder holds 75 files: LAMA has no Galileo file, because its receiver tracks Galileo E5a only on part of the epochs and flags almost all of those phase observations with a loss of lock, so no Galileo arc survives the cycle-slip detection. The way the files were made matters because temporal indices such as SIDX and ROTI mostly use changes along one receiver-satellite link, while spatial indices such as GIX compare different links at the same epoch and therefore need calibrated or at least consistently levelled STEC.

First, let us begin by discussing and deriving equations relating to SIDX, i.e. the index describing temporal changes in TEC in a given area.

Let us denote STEC, total electron content along the signal transmission path measured at time t, as:

$$\mathrm{STEC}(t) = \frac{\tilde{L}_4^{(n)}(t)}{40.3 \cdot 10^{16} \left(\frac{1}{f_2^2} - \frac{1}{f_1^2}\right)} \ [\mathrm{TECU}]$$

where $\tilde{L}_4^{(n)}(t)$ is the code-levelled geometry-free phase $L_1 - L_2$ in metres and f_1, f_2 are the frequencies of the received signals in Hz (the ionosphere advances the phase of f_2 more, so $L_1 - L_2$ grows with STEC; the factor 10^{16} gives TECU). Conversion from STEC to VTEC is performed using the mapping function $M(e; H)$, where e and H are, respectively: the satellite elevation angle and the height of the ionospheric shell.

$$\mathrm{VTEC}(t) = \frac{\mathrm{STEC}(t)}{M(e; H)} \quad [\mathrm{TECU}]$$

$$M(\varepsilon; H) = \frac{1}{\sqrt{1 - \left(\frac{R_E \cos e}{R_E + H}\right)^2}}$$

The SIDX index describes the rate of change of TEC in a given area. Imagine that we have a network of GNSS stations and we record observations at 30-second intervals. Let us assume that we are tracking a certain satellite and recording the measured STEC at epochs t_0 and t_1 with one of the receivers.

```python
from pathlib import Path

import numpy as np
import pandas as pd
from IPython.display import Image, display

import gnx_py as gnx
from gnx_py import viz

## One notebook-wide figure style: serif fonts, Computer Modern mathtext.
viz.apply_publication_style(font_scale=1.3, mathtext="cm")


def find_repo_root(start: Path | None = None) -> Path:
    """Find the GNX repository root from the current notebook working directory."""
    start = (start or Path.cwd()).resolve()
    for candidate in (start, *start.parents):
        if (candidate / "gnx_py").is_dir() and (candidate / "tutorial").is_dir():
            return candidate
    raise RuntimeError(
        "Could not find the GNX repository root. Start Jupyter from inside the cloned repository."
    )


REPO_ROOT = find_repo_root()
TUTORIAL_DIR = REPO_ROOT / "tutorial" / "ionosphere"
NETWORK_DIR = TUTORIAL_DIR / "network"
FIGURE_DIR = TUTORIAL_DIR / "figures"
ROTI_FIGURE_DIR = FIGURE_DIR / "roti"
GIX_FIGURE_DIR = FIGURE_DIR / "gix"
REGENERATE_FIGURES = True

## Paths are printed relative to the repository, so no machine-specific paths end up in the outputs.
print(f"Network folder:  {NETWORK_DIR.relative_to(REPO_ROOT)}")
```

```
Network folder:  tutorial/ionosphere/network
```

```python
fig = viz.plot_stec_change()
```

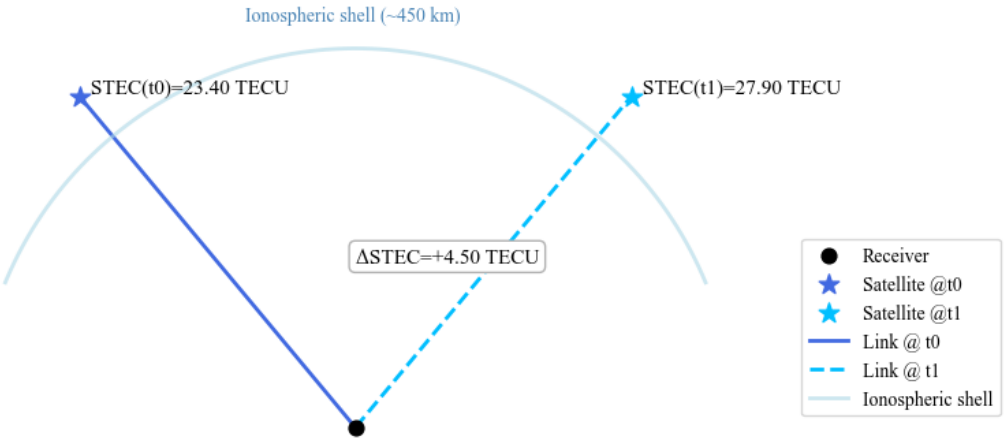


Figure 6.1: Example STEC time variation used to introduce temporal ionospheric activity indices.

By converting to VTEC and differentiating the observation from epoch t_1 and t_0 and dividing by the time interval, we obtain an approximate value for the rate of change of VTEC. Of course, satellites in neighbouring epochs are located in different places in space, and thus the location of the IPP changes, but we assume that in short intervals (at short distances between IPPs) the change in VTEC is minimal. The graph above illustrates this type of situation at the level of a single link. With the aforementioned network of stations and observing many satellites in different locations in neighbouring epochs, we obtain a set of such VTEC differences. Let us write this down step by step:

6.1.1 SIDX (temporal VTEC change)

At the level of a single link we begin with the difference between two neighbouring epochs. Written as a continuous rate, the expression has units of TECU/s:

$$\mathrm{SID}_{\mathrm{link}}(t_0, t_1) = \frac{\Delta \mathrm{STEC}_{t_0,t_1}}{M(e;H)\,\Delta t} = \frac{\Delta \mathrm{VTEC}_{t_0,t_1}}{\Delta t} \quad [\mathrm{TECU/s}]$$

In GNX-py the practical monitoring output is reported in TECU/min, so the implementation multiplies the per-second rate by 60:

$$\mathrm{SID}_{\mathrm{link,min}}(t_0, t_1) = 60\,\frac{\Delta \mathrm{STEC}_{t_0,t_1}}{M(e;H)\,\Delta t} \quad [\mathrm{TECU/min}]$$

With a set of links between two neighbouring epochs, we can assign a given difference to epoch t_1 and use an aggregation function to represent all recorded VTEC changes in a given area.

$$\mathrm{SIDX}(t_1) = \mathrm{Agg}\left\{\mathrm{SID}_{\mathrm{link,min},\,\ell}(t_0, t_1) \;:\; \ell \in \mathcal{L}(t_0, t_1)\right\} \quad [\mathrm{TECU/min}]$$

Where ℓ is a single link and $\mathcal{L}(t_0, t_1)$ is the set of all links between two neighbouring epochs and $\mathrm{Agg} \in \{\mathrm{median}, \mathrm{mean}, \mathrm{p95}, \mathrm{max}\}$ is the aggregation function.

Current version note. `gnx.compute_sid` returns a pandas Series named `SIDX_tecu_per_min`. If you want to compare the result with a paper or product expressed in mTECU/s, use

$$\mathrm{SIDX}_{\mathrm{mTECU/s}}(t) = \frac{1000}{60}\,\mathrm{SIDX}_{\mathrm{TECU/min}}(t)$$

Finally, all calculated and aggregated differences constitute our SIDX index for a given reference epoch (t_1). Note that receiver and satellite DCB terms are omitted in the equations because they cancel during differentiation if they are approximately constant over the analysed arc. This is why temporal indices can be useful even before full absolute TEC calibration, although cycle slips, re-levelling jumps and day boundaries must still be handled carefully.

6.1.2 ROT (Rate Of TEC)

SIDX is a certain (mean, median, percentile) measure representing temporal changes in VTEC in a given area; it can be represented as an aggregation of ROT values. ROT itself is usually computed along one receiver-satellite link:

$$\mathrm{ROT}(t) \approx \frac{\Delta \mathrm{TEC}}{\Delta t} = \frac{\mathrm{TEC}(t) - \mathrm{TEC}(t - \Delta t)}{\Delta t} \quad [\mathrm{TECU/s}]$$

$$\mathrm{ROT}_{\mathrm{min}}(t) = 60 \cdot \mathrm{ROT}(t) \quad [\mathrm{TECU/min}]$$

ROTI, on the other hand, is calculated as the standard deviation of ROT in a given time window, e.g. 5 minutes. In GNX-py, SIDX is derived from mapped VTEC changes, while ROT and ROTI are usually computed from STEC along the same link, following the convention used in ROTI mapping workflows (Cherniak et al. 2018; Carmo et al. 2021).

6.1.3 ROTI (Rate Of TEC Index)

$$\mathrm{ROTI}(t) = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(\mathrm{ROT}_{\min}(t_k) - \overline{\mathrm{ROT}_{\min}}\right)^2} \quad [\mathrm{TECU/min}]$$

where: - t_k — epochs in given time window, - $\overline{\mathrm{ROT}_{\min}}$ — mean ROT in given time window.

Current version note. `gnx.compute_roti_links` uses a rolling time window, population standard deviation (`ddof=0`), optional 5-minute detrending, and protects the differentiation against large time gaps. The output contains both `ROT_tecu_per_min` / `ROTI_tecu_per_min` and mTECU/s variants.

As with SIDX, in the case of ROT we also work with receiver-satellite pairs. In GNX-py, ROTI can be presented in the form of a coordinate grid in a geographic or solar-geomagnetic system. The ROTI values calculated in the window can be assigned to grid points according to one of two approaches, depending on how you want to interpret the temporal support of the map: - the ROTI values come from a window centred on the plotted epoch (`mode="center"`), - the ROTI values come from a trailing window ending at the plotted epoch (`mode="trailing"`).

6.1.4 GIX (VTEC spatial gradient)

The last of the indices available in GNX-py is the GIX family of indices: GIX, GIXS, and GIX95. These are indices describing the spatial variation of VTEC in a given area. Unlike ROT and SIDX, in case of GIX we process data from a single epoch. Again, let us assume that we have a network of stations recording GNSS satellites signals. In each epoch, we observe a set of satellites from each station and obtain the corresponding STEC measurements along with their IPP according to the assumed model (by default, thin-shell 450 km). The GIX is calculated as follows:

Having pair of IPPs (i, j) (see figure below):

$$\Delta\mathrm{VTEC} = \mathrm{VTEC}_i - \mathrm{VTEC}_j$$

We can calculate the VTEC gradient in the direction of $ j →i $:

$$g_{ij} = \frac{\Delta\mathrm{VTEC}}{\Delta s} \quad [\mathrm{TECU/km}] \qquad \Delta s \approx s_{ij}$$

where Δs is the distance between IPPs measured on the ionospheric shell.

By calculating the azimuth $\alpha_{j\to i}$ of the line connecting two points, we can decompose the gradient into a gradient in the direction WE/NS:

$$g_{WE} = g_{ij}\sin\alpha_{j\to i}, \qquad g_{NS} = g_{ij}\cos\alpha_{j\to i} \quad [\mathrm{TECU/km}]$$

The calculated gradient is located at the central point $CP_{i,j}$ (see figure) between the IPP points. Then all pairs (i, j) from the set $\mathcal{P}(t)$ from a given epoch are aggregated using the mean function:

$$\overline{g}_{WE}(t) = \mathrm{mean}\{g_{WE}\}_{(i,j)\in\mathcal{P}(t)}, \qquad \overline{g}_{NS}(t) = \mathrm{mean}\{g_{NS}\}_{(i,j)\in\mathcal{P}(t)}$$

Finally, the GIX index for a given epoch is calculated as:

$$\mathrm{GIX}(t) = \sqrt{\overline{g}_{WE}(t)^2 + \overline{g}_{NS}(t)^2} \quad [\mathrm{TECU/km}]$$

$$\mathrm{GIXS}(t) = \mathrm{std}\{|g_{ij}|\}_{(i,j)\in\mathcal{P}(t)} \quad [\mathrm{TECU/km}]$$

$$\mathrm{GIXP95}(t) = \mathrm{percentile}_{95}\{|g_{ij}|\}_{(i,j)\in\mathcal{P}(t)} \quad [\mathrm{TECU/km}]$$

Current version note. GNX-py reports the GIX family in mTECU/km, so the values returned by `gnx.compute_gix` are the equations above multiplied by 1000. The function also stores directional statistics, signed percentile components and the number of accepted IPP pairs per epoch.

The figure below illustrates an example on a micro scale – for a pair of points. In GNX-py, criterion for selecting pairs is based on the distance between points. The user can set this range themselves or select the default, e.g. a range from 30 to 250 km. The above calculations are therefore performed for all pairs created in a given epoch.

```
fig = viz.plot_gix_two_points(
    draw_gc_arc=True,
    show_cp_arrow=False,
    title="TEC gradient between a pair of IPPs",
)
```

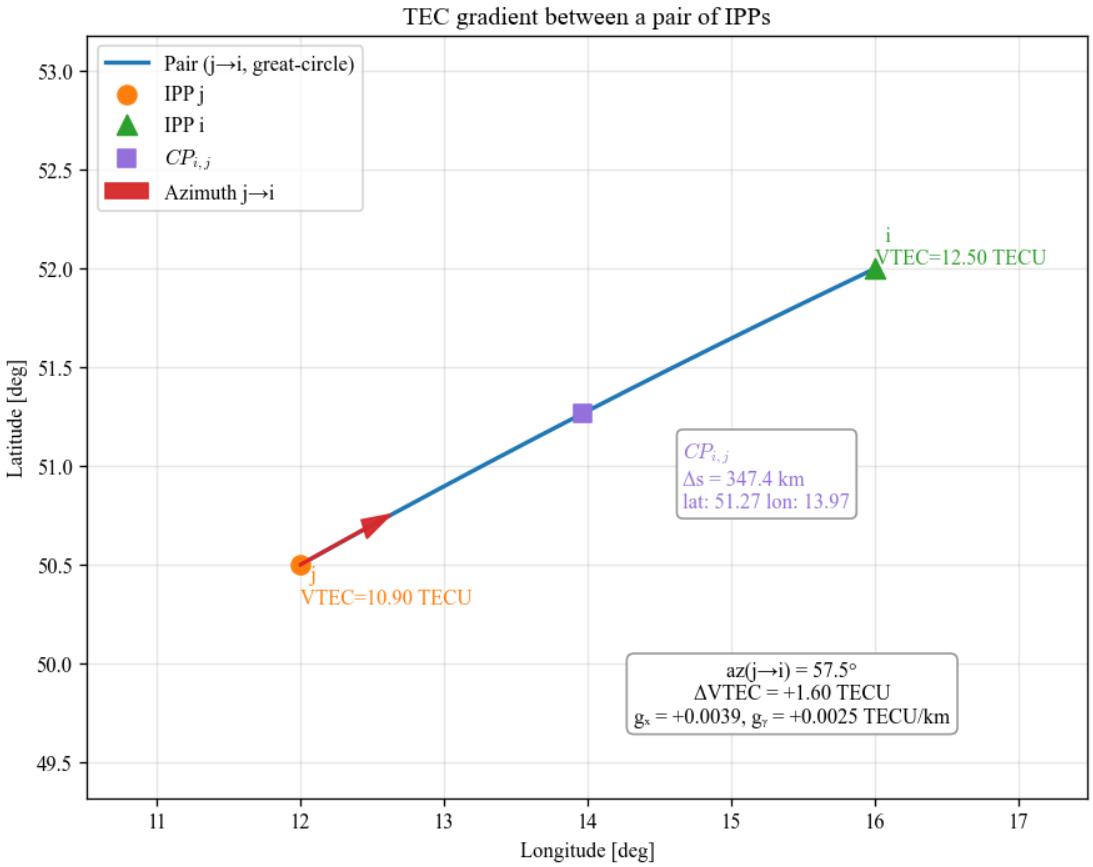


Figure 6.2: Two-point geometry used to build intuition for spatial VTEC gradients.

To learn more about the issues related to the indices described above, please refer to works (Jakowski and Hoque 2019; Cherniak et al. 2018; Carmo et al. 2021). Most of the methodology is based on these works. Now we will show how the above indices can be obtained in GNX-py.

The `network/` folder used here contains a real, measured STEC network, so the computations use real data. To keep the notebook suitable for interactive work and later PDF conversion, the default examples use a four-hour window for SIDX/ROTI and a shorter window for GIX. You can extend the time limits to the full day after you understand the workflow, but dense network GIX can become computationally expensive because it forms IPP pairs inside each epoch.

6.1.5 Processing example

```
## The imports, the figure style and the paths were set in the first code cell,
```

```
## so the explanatory figures could already be drawn in the theory section.
local_checkout = Path(gnx.__file__).resolve().parent == (REPO_ROOT / "gnx_py").resolve()
print(f"GNX-py module: "
      f"{'gnx_py/ of this repository' if local_checkout else 'an installed package'}")
print(f"Network files: {len(list(NETWORK_DIR.glob('*STEC.parquet.gzip')))}")
```

```
GNX-py module: gnx_py/ of this repository
Network files: 75
```

Let's start by loading the data. To calculate indices representing ionospheric activity over a given region, we must have access to the network of stations in that region. In this example, the local `network/` folder contains files with processed STEC measurements from EUREF network stations. These data were prepared with the STEC processing workflow, with receiver DCBs calibrated against the CODE GIM (see above).

Carefully studying the equations presented above, you will notice that receiver and satellite DCBs, as approximately constant biases, cancel during time-differentiation for SIDX and ROTI. We can therefore use temporal changes even when the absolute level is less important. For the GIX family, however, we compare different receiver-satellite links at the same epoch, so the STEC measurements should be consistently levelled/calibrated.

Data loading is performed using the `load_stec_folder` function. The important parameters here are the elevation mask, optional `network_filter`, `skip_negative`, and the file suffix used to detect STEC parquet files.

```
data, info = gnx.load_stec_folder(
    folder=NETWORK_DIR,
    file_suffix="STEC.parquet.gzip",
    min_elev_deg=30,
    station_name_len=6,
    unique_id_len=4,
    quiet=True,
    skip_negative=True,
    keep_columns=("lat_ipp", "lon_ipp", "leveled_tec", "ev", "sv", "time"),
)

summary_keys = ["files_scanned", "files_accepted", "files_rejected"]
{k: info[k] for k in summary_keys}
```

```
{'files_scanned': 75, 'files_accepted': 60, 'files_rejected': 15}
```

This function is designed to read sets of output files from individual station processing, saved by default in Parquet format. This format takes up less space than CSV and is useful when we usually want to access more than one station. The function arguments shown are close to default values, except for the folder path and the stricter 30-degree elevation mask.

`skip_negative=True` rejects an entire station/system file if it contains negative levelled TEC after the elevation filter. This is intentionally conservative for a tutorial: negative TEC often points to a levelling, DCB, or measurement problem that should be inspected before the file is used in spatial gradients. With this network it rejects 15 of the 75 files, all of them GPS: their lowest values lie between -0.1 and -1.9 TECU, in at most 1.5 % of the samples of a file, at night (19-06 GPST), when the true STEC is only a few TECU. That is the size of the remaining levelling and calibration errors rather than a gross error; set `skip_negative=False` to keep these stations and compare the results.

Let's take a look at the obtained data. The `info` dictionary contains information about the upload process — the number of files accepted and rejected, station identifiers, rejected files, and parsing errors if any.

```
viz.df_head(data[["leveled_tec", "lat_ipp", "lon_ipp", "ev", "name"]], nrows=4, ncols=7)
```

sv	time	leveled_tec	lat_ipp	lon_ipp	ev	name
E02_0_1	2024-02-04 08:35:30	49.058	46.225	9.380	30.000	BAUT_E
E02_0_1	2024-02-04 08:36:00	48.960	46.261	9.398	30.181	BAUT_E
E02_0_1	2024-02-04 08:36:30	48.805	46.297	9.415	30.363	BAUT_E
E02_0_1	2024-02-04 08:37:00	48.766	46.333	9.432	30.544	BAUT_E

```
info.keys()
```

```
dict_keys(['files_scanned', 'files_accepted', 'files_rejected', 'stations_unique', '
    stations_accepted', 'stations_rejected', 'rejected_files', 'errors'])
```

As you can see, our data includes the satellite PRN, station ID, STEC measurements, elevation, and the geocentric IPP coordinates. Let's perform some additional preparations for calculating the indices: calculate the mapping function for a 450 km thin shell, create a `stec` column expected by the monitoring helpers, and derive VTEC.

The full data set spans the entire day. For the tutorial run below we select a four-hour analysis window. All epochs are GPS time (GPST), the time scale of the TEC session; we keep them as naive timestamps, which is what the monitoring functions expect (a timezone-aware time would be converted from UTC to GPST, i.e. moved by 18 s, with a warning). This keeps the notebook responsive while still preserving the network character of the problem.

```
## load_stec_folder indexes the frame by (sv, time) and also keeps `sv` and `time`
## as ordinary columns. A plain reset_index() would try to insert them a second
## time and fail, so the duplicate index levels are dropped instead.
df = data.reset_index(drop=True)
df["time"] = pd.to_datetime(df["time"])  # naive GPST, as written by TECSession
df["stec"] = df["leveled_tec"]
df["M"] = gnx.mapping_M(df["ev"].to_numpy(), H_m=450e3)
df["vtec"] = df["stec"] / df["M"]

ANALYSIS_START = pd.Timestamp("2024-02-04 10:00:00")  # GPST
ANALYSIS_END = pd.Timestamp("2024-02-04 14:00:00")
analysis_mask = (df["time"] >= ANALYSIS_START) & (df["time"] < ANALYSIS_END)
df_window = df.loc[analysis_mask].copy()

print(f"Full data set: {df['time'].min()} to {df['time'].max()}, rows={len(df):,}")
print(f"Tutorial window: {ANALYSIS_START} to {ANALYSIS_END}, rows={len(df_window):,}")
```

```
Full data set: 2024-02-04 00:00:00 to 2024-02-04 23:59:30, rows=730,372
Tutorial window: 2024-02-04 10:00:00 to 2024-02-04 14:00:00, rows=117,207
```

Using the `Region` class from the `monitoring.py` module, we define the area over which we want to determine our indexes. This area is already implicitly defined by the selected stations, but when processing larger networks, it is worth specifying the study area more precisely. Otherwise, distant IPPs can increase computation time and may make the spatial interpretation less clear.

Here we focus on central Europe / Poland-like longitudes and latitudes, which match the supplied EUREF subset.

```
reg = gnx.Region(lat_min=50.0, lat_max=57.5, lon_min=10.0, lon_max=30.0)
extent = [9.0, 29.0, 49.0, 58.5]
reg
```

```
Region(lat_min=50.0, lat_max=57.5, lon_min=10.0, lon_max=30.0)
```

We're ready to calculate ROTI. We do this in the `compute_roti_links` function. We define:

- `df` — our dataframe with observations,
- `region` — the region class instance used to keep IPPs in the study area,
- `tec_source` — source observation column (`stec` or `vtec`),
- `tec_scale` — scale in which we report STEC or VTEC (`1` for TECU, `1e-16` if values were still in electrons),

- `window_min` and `min_samples` — rolling window and the minimum number of ROT samples needed for ROTI,
- `max_gap_s` — maximum allowed gap between neighbouring epochs inside one link,
- `min_elev_deg` — elevation mask,
- `coord_mode` — reference system (`GEO` or `SM`),
- `detrend_5min` — whether to remove a 5-minute moving mean before differentiating.

The detrending flag is useful when we want ROTI to emphasize short-term variability rather than the slowly changing background STEC.

```
roti = gnx.compute_roti_links(
    df=df_window,
    region=reg,
    tec_source="stec",
    tec_scale=1.0,
    window_min=5,
    min_samples=10,
    max_gap_s=31,
    split_on_day_change=True,
    min_elev_deg=30,
    detrend_5min=True,
    coord_mode="GEO",
)
```

```
viz.df_tail(
    roti[["time", "lat_ipp", "lon_ipp", "ROT_tecu_per_min", "ROTI_tecu_per_min"]],
    nrows=3, ncols=10, reset_index=False, index=False, truncate_str=3,
)
```

time	lat_ipp	lon_ipp	ROT_tecu_per_min	ROTI_tecu_per_min
2024-02-04 13:59:30	52.910	26.567	-0.144	0.127
2024-02-04 13:59:30	50.085	14.535	-0.148	0.123
2024-02-04 13:59:30	52.782	11.948	0.068	0.180

Let's look at the results. Each row represents a satellite-receiver pair at a given epoch. Instead of hard-coding one station and satellite, we choose a link that has many finite ROTI samples in the current data set. This makes the notebook more robust if the `network/` folder is replaced by another campaign.

As you can see, ROTI is calculated within a 5-minute window. The first finite value appears only after enough ROT samples have been accumulated, and each subsequent ROTI value represents the variability of ROT inside the rolling window.

```
finite_roti = roti.dropna(subset=["ROTI_tecu_per_min"])
if finite_roti.empty:
    raise RuntimeError("No finite ROTI values were produced for the selected window.")

sample_name, sample_sv = (
    finite_roti.groupby(["name", "sv"])
    .size()
    .sort_values(ascending=False)
    .index[0]
)
roti_sample = (
    roti[(roti["name"] == sample_name) & (roti["sv"] == sample_sv)]
    .sort_values(by="time")[["time", "name", "sv", "ROTI_tecu_per_min"]]
)
viz.df_head(roti_sample, nrows=12, ncols=10, reset_index=False, index=False)
```

time	name	sv	ROTI_tecu per_min
2024-02-04 10:00:00	POTS_E	E02_0_1	nan
2024-02-04 10:00:30	POTS_E	E02_0_1	nan
2024-02-04 10:01:00	POTS_E	E02_0_1	nan
2024-02-04 10:01:30	POTS_E	E02_0_1	nan
2024-02-04 10:02:00	POTS_E	E02_0_1	nan
2024-02-04 10:02:30	POTS_E	E02_0_1	nan
2024-02-04 10:03:00	POTS_E	E02_0_1	nan
2024-02-04 10:03:30	POTS_E	E02_0_1	nan
2024-02-04 10:04:00	POTS_E	E02_0_1	nan
2024-02-04 10:04:30	POTS_E	E02_0_1	nan
2024-02-04 10:05:00	POTS_E	E02_0_1	0.028
2024-02-04 10:05:30	POTS_E	E02_0_1	0.043

`viz.plot_roti_series` gives a first simple visualization of our data — it collapses the per-link ROTI to the mean and the 95th percentile of ROTI in each epoch. The mean tells us about the network-wide background level, while the upper percentile is more sensitive to localized disturbances or a subset of links with stronger short-term variability.

```
fig = viz.plot_roti_series(
    roti,
    value_col="ROTI_tecu_per_min",
    quantile=0.95,
    title=(
        f"Mean and 95% ROTI over time\n{reg}\n"
        "Moving standard deviation over a 5 min window"
    ),
    figsize=(10, 5),
)
```

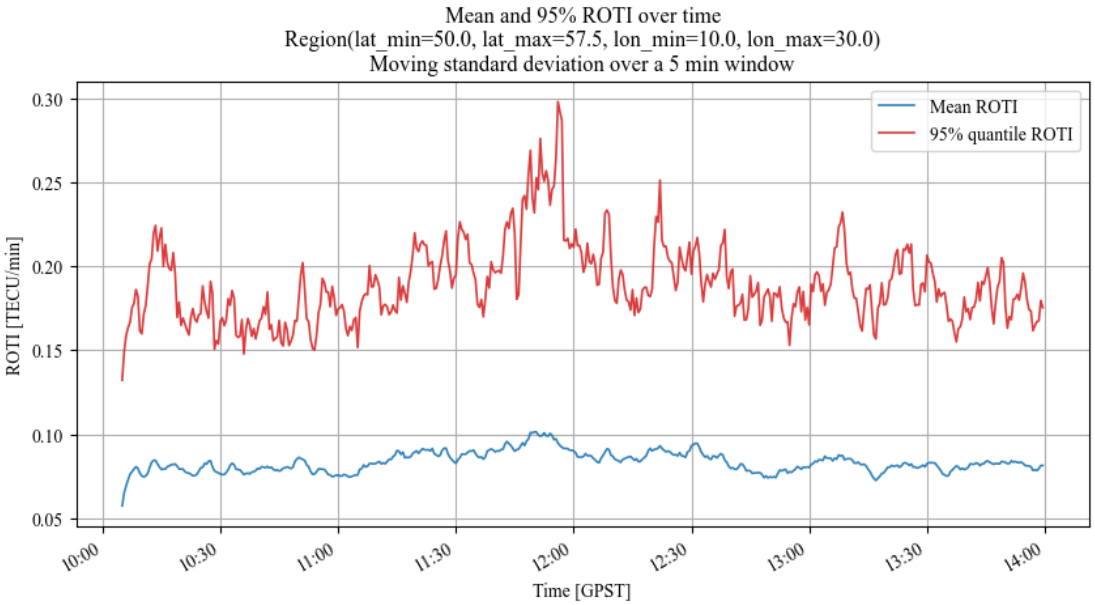


Figure 6.3: ROTI summary statistics over the selected regional network window.

As you can see, the graph allows you to observe the evolution of ROTI over time. Day 2024-035 is a relatively quiet day for geomagnetic activity, so we expect stable values rather than dramatic spikes. You can also inspect the `ROTI_mtecu_per_s` column if you want to express the same variability at a different time scale.

A key feature of the module in case of ROTI is the ability to visualize it in the form of snapshot images covering a given time window. This is achieved using `viz.plot_roti_snapshot` for one epoch or `viz.make_roti_snapshots` for a sequence. The most important parameters are:

- `roti_df` — a dataframe with ROTI results,

- `window_min` — width of the time window used for the map,
- `mode` (`center` or `trailing`) — whether the time window is centred on the map epoch or ends at it,
- `vmin` / `vmax` — explicit colour scale for one frame; `global_scale` is available in `make_roti_snapshots` when all frames should share one scale,
- `agg` — aggregation function inside each map tile (`median`, `mean`, `max`, `Q95`),
- `res_deg` — grid resolution,
- `min_points` — minimum number of ROTI samples required in one tile,
- `coord_mode` — geographic or solar-geomagnetic coordinate mode,
- `figsize` / `dpi` — figure size and resolution; the defaults are 10 × 10 in at 600 dpi. With `save_path` the figure is cropped to its content, so the PNG is about 6000 px wide and its height follows the map extent (`make_roti_snapshots` keeps the full 6000 × 6000 px canvas so that all frames match).

The maps draw coastlines and borders from Natural Earth through Cartopy. Cartopy downloads these files on first use and keeps them in its data directory, so the first map needs an internet connection once (see `tutorial/README.md` for fetching them in advance).

For PDF-oriented output, the next cell saves one deterministic PNG (`save_path`), closes the live figure (`close=True`) and then displays that PNG at a fixed width. This gives better control over the final image size during Markdown/PDF conversion than relying on an automatically embedded matplotlib output.

```
ROTI_FIGURE_DIR.mkdir(parents=True, exist_ok=True)
roti_epoch = pd.Timestamp("2024-02-04 12:00:00")  # GPST
roti_figure_path = ROTI_FIGURE_DIR / "roti_20240204_120000.png"

if REGENERATE_FIGURES or not roti_figure_path.exists():
    # close=True: the saved PNG is displayed below, so the live figure is not shown twice.
    fig = viz.plot_roti_snapshot(
        roti_df=roti,
        epoch=roti_epoch,
        window_min=5,
        mode="center",
        res_deg=2,
        min_points=3,
        agg="Q95",
        coord_mode="GEO",
        extent=extent,
        save_path=roti_figure_path,
        close=True,
    )
    if fig is None:
        raise RuntimeError("No ROTI samples fall in the snapshot window around the selected epoch.")

display(Image(filename=str(roti_figure_path), width=760))
```

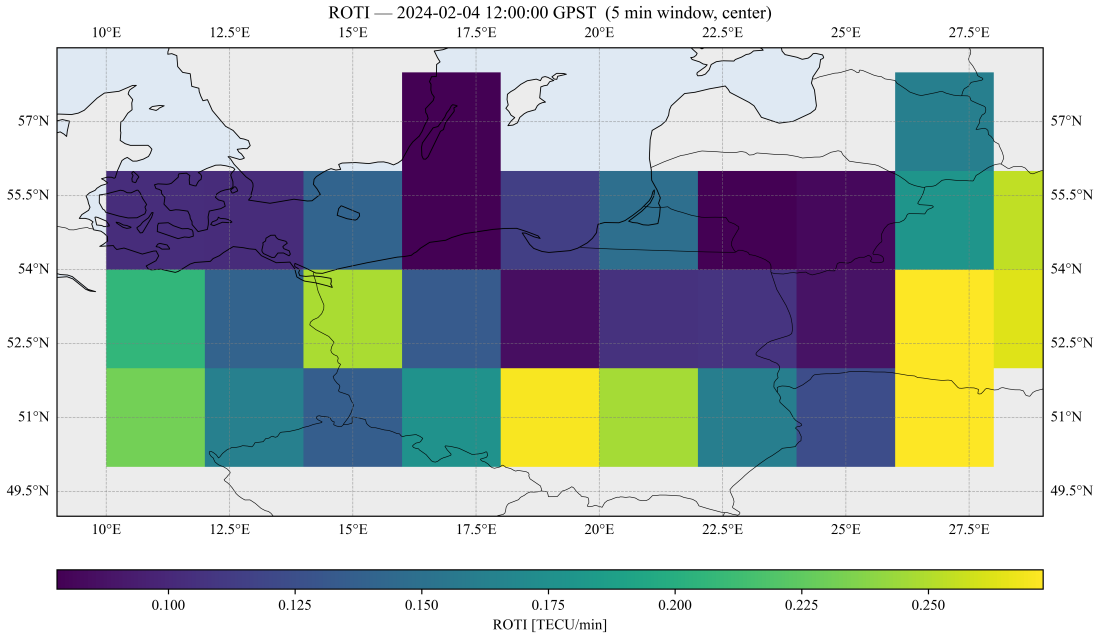


Figure 6.4: ROTI snapshot rendered from the generated tutorial figure file.

You can view the created images in the `figures/roti` directory. The images are generated as follows: each square with dimensions corresponding to the resolution `res_deg` is assigned a value aggregated using the `agg` function. If we have at least `min_points` samples in a tile, the values are aggregated and assigned to that tile. Together with the line graph above, this map is useful for spatial visualisation.

If you want to create a full sequence of frames, use `viz.make_roti_snapshots(...)` with `out_dir=ROTI_FIGURE_DIR` and a larger set of epochs. It saves and closes every frame and returns the list of written paths. For the tutorial we generate only one frame by default to avoid filling the repository with images.

```
## Optional heavier export pattern for a sequence of frames:
## frames = viz.make_roti_snapshots(
##     roti_df=roti,
##     every_n=30,
##     window_min=5,
##     mode="center",
##     res_deg=2,
##     min_points=3,
##     global_scale=False,
##     q_scale=(5, 99),
##     out_dir=ROTI_FIGURE_DIR,
##     agg="Q95",
##     coord_mode="GEO",
##     extent=extent,
## )
## len(frames)
```

As explained in the introduction, SIDX is also an indicator of TEC temporal variability. With data from multiple stations available, we calculate it in GNX-py in a way similar to ROTI, but here the STEC change is mapped to VTEC change through the thin-shell mapping function. The output unit is TECU/min.

```
sidx = gnx.compute_sid(
    df=df_window,
    region=reg,
    min_elev_deg=30,
    iono_h_m=450e3,
```

```
    stec_scale=1.0,
    max_gap_s=31,
    split_on_day_change=True,
    agg="p95",
)
```

```
viz.df_head(sidx.to_frame(), nrows=3, ncols=2)
```

time	SIDX_tecu per_min
2024-02-04 10:00:30	0.220
2024-02-04 10:01:00	0.212
2024-02-04 10:01:30	0.240

The result is a pandas Series with SIDX calculated for each epoch. `viz.plot_sidx_series` draws it as a time series (epochs without a value are dropped). The example below keeps the unit used by GNX-py, i.e. TECU/min.

```
fig = viz.plot_sidx_series(
    sidx,
    title=(
        f"SIDX for {reg}\n"
        f"Period: {ANALYSIS_START:%Y-%m-%d %H:%M-}{ANALYSIS_END:%H:%M} GPST\n"
        "Sampling rate: 30 s, aggregation: Q95"
    ),
    figsize=(12, 6),
)
```

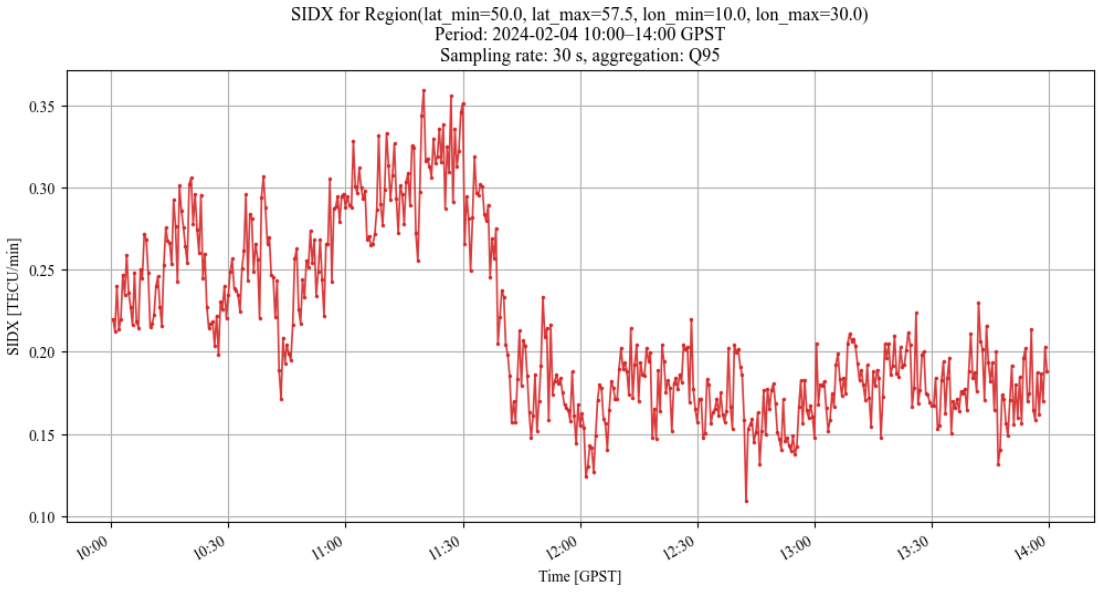


Figure 6.5: SIDX time series after removing invalid or missing epochs.

The `gnx_py.viz` functions are a convenience, not a requirement: every GNX result is an ordinary pandas object, so the built-in pandas plotting works as well. As an alternative, the same SIDX series can be drawn directly with `Series.plot`. This is the only figure in the tutorial made without `gnx_py.viz`.

```
## Alternative: plain pandas plotting of the same SIDX series.
ax = sidx.dropna().plot(
    figsize=(12, 4),
    color="tab:red",
    lw=1.0,
    grid=True,
```

```
    xlabel="Time [GPST]",
    ylabel="SIDX [TECU/min]",
    title="SIDX drawn with pandas Series.plot",
)
```

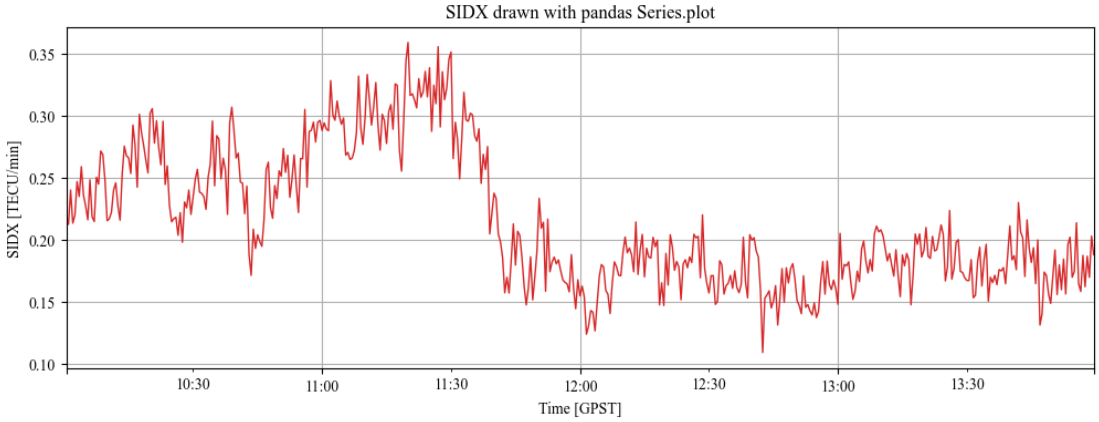


Figure 6.6: The same SIDX series drawn directly with pandas Series.plot.

The GIX index, like the previous ones, is calculated using a dedicated function. The function arguments are similar to those for ROTI and SIDX, with a few important exceptions:

- `dipole_preset` — predefined IPP pair distance range. Current presets include `30-250`, `50-500`, and `50-1000` km.
- `dmin_km`, `dmax_km` — custom lower and upper distance limits for IPP pairs; these override the preset.
- `max_delta_M` — maximum allowed difference in mapping function between two IPPs. Values around 0.0–0.2 are useful for reducing sensitivity to incorrectly levelled or biased links.
- `max_pairs_per_epoch` and `stratified_sampling` — pair thinning controls. They are important for larger networks because the number of possible pairs grows quickly inside each epoch. The thinning is random; `rng` fixes its random generator, so the result is reproducible.

Unlike ROTI and SIDX, GIX compares different receiver-satellite pairs at the same epoch. That is why calibrated or consistently levelled STEC is much more important here.

```
GIX_START = pd.Timestamp("2024-02-04 12:00:00")  # GPST
GIX_END = pd.Timestamp("2024-02-04 12:15:00")
df_gix = df[(df["time"] >= GIX_START) & (df["time"] < GIX_END)].copy()
print(f"GIX tutorial window rows: {len(df_gix):,}")
```

```
GIX tutorial window rows: 7,639
```

```
dmin, dmax = 150, 500

times, pairs = gnx.compute_gix(
    df=df_gix,
    region=reg,
    min_elev_deg=30,
    dmin_km=dmin,
    dmax_km=dmax,
    iono_h_m=450e3,
    stec_scale=1.0,
    max_delta_M=0.1,
    max_pairs_per_epoch=50_000,
    stratified_sampling=True,
    rng=np.random.default_rng(0),
)
```

Our output consists of two dataframes. The `times` dataframe contains a family of GIX indices for each epoch. It includes:

- `GIX_mtecu_km`,
- average GIX components in the E-W and N-S directions (`x` and `y`),
- `GIXS_mtecu_km` and its directional components,
- positive/negative directional percentile components,
- `GIXP95_mtecu_km`,
- `n_pairs` — number of accepted IPP pairs in a given epoch,
- `GIX_LS_mtecu_km` and its components — the least-squares gradient from the same pairs. GIX averages the projected components, so for pairs spread evenly in azimuth it is about half of the gradient magnitude; the least-squares gradient estimates the full magnitude for any pair geometry.

The whole family is visualised with `viz.plot_gix_family`, which draws GIX, GIXS and GIXP95 in three stacked panels sharing one time axis.

```
viz.df_head(
    times[["time", "GIX_mtecu_km", "GIXS_mtecu_km", "GIXP95_mtecu_km", "n_pairs"]],
    nrows=3, ncols=7,
)
```

time	GIX_mtecu_km	GIXS_mtecu km	GIXP95_mtecu km	n_pairs
2024-02-04 12:00:00	2.564	4.789	15.095	1196
2024-02-04 12:00:30	2.559	4.792	15.008	1190
2024-02-04 12:01:00	2.555	4.804	15.052	1192

```
fig = viz.plot_gix_family(times, reg=reg, dmin=dmin, dmax=dmax)
```

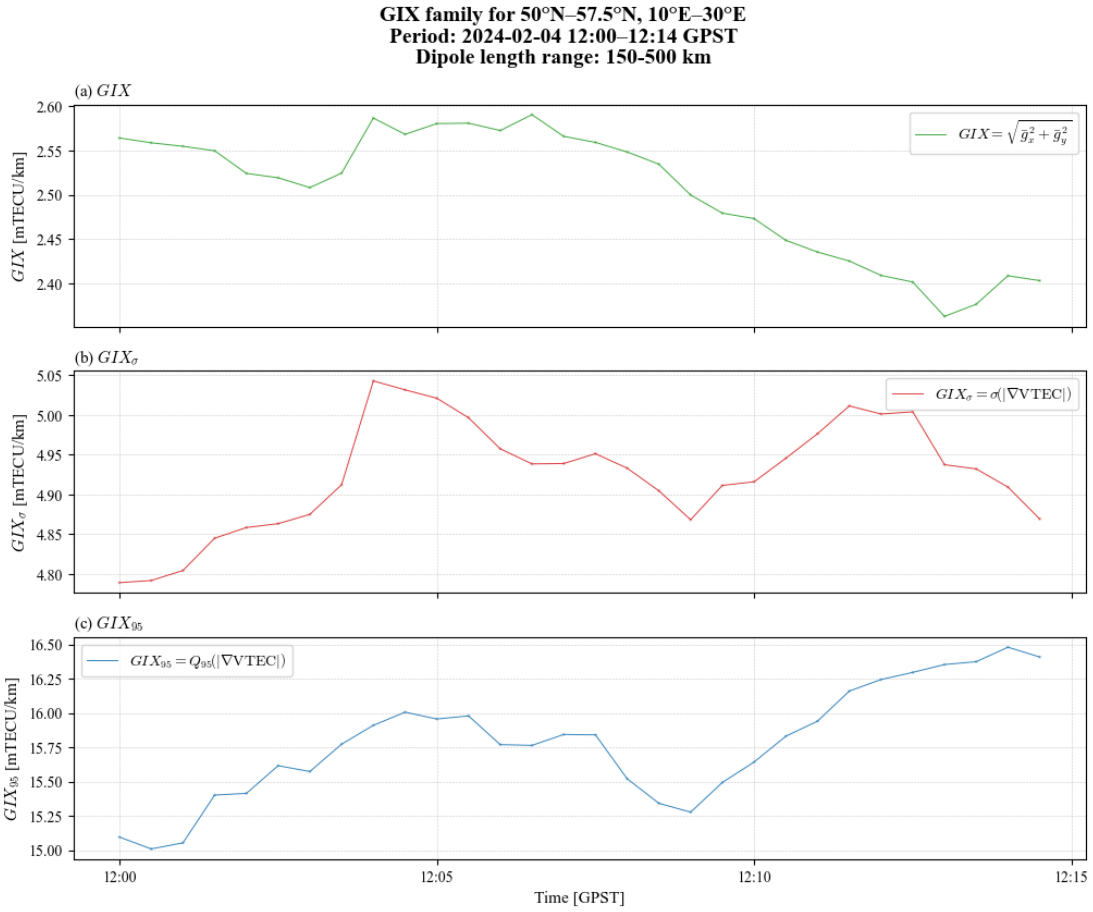


Figure 6.7: GIX family diagnostics for several spatial-gradient summaries.

In the `pairs` dataframe, you will find data on each pair that was created in each epoch. Each row contains the dipole centre coordinates, `CP_lat` and `CP_lon`, the absolute VTEC gradient calculated in the dipole direction, directional gradients, dipole length, mapping function values, station and satellite names, and VTEC differences between IPPs.

This data can be used for spatial visualisation of GIX. The `viz.plot_gix` function is used for this purpose. Keep in mind that we are operating on dipoles of different lengths, so when interpreting the image, do not be confused by the colours of individual points. What you are looking at are GIX values calculated for pairs of points separated by `dmin_km` and `dmax_km`. I encourage you to change the dipole length range and examine the plots and `pairs` dataframe after that. This will give you an intuitive insight into how GIX is calculated.

```
viz.df_head(
    pairs[["time", "CP_lat", "CP_lon", "grad_abs_mtecu_km", "dipole_km"]],
    nrows=3, ncols=7,
)
```

time	CP_lat	CP_lon	grad_abs mtecu_km	dipole_km
2024-02-04 12:00:00	52.207	18.852	4.577	459.082
2024-02-04 12:00:00	52.184	18.324	8.107	384.226
2024-02-04 12:00:00	52.100	17.053	6.983	208.216

The displayed value is the column `grad_abs_mtecu_km`, i.e. the absolute value of the gradient per kilometre in mTECU units. The image below can be interpreted as follows. When we see a large accumulation of bright spots in a given area, we can assume that there is a high VTEC gradient in that region, within a radius of `dmin_km`–`dmax_km` from the spot. Darker spots indicate weaker gradients for the same dipole selection rules.

As with ROTI, the figure is saved first and then displayed as an image. This keeps the layout stable when the notebook is converted to Markdown and then to PDF.

```
GIX_FIGURE_DIR.mkdir(parents=True, exist_ok=True)
if pairs.empty:
    raise RuntimeError("No GIX pairs were produced for the selected window.")

epochs = pd.DatetimeIndex(pd.to_datetime(pairs["time"].unique())).sort_values()
epoch = epochs[min(len(epochs) // 2, len(epochs) - 1)]
gix_figure_path = GIX_FIGURE_DIR / "gix_epochs.png"

if REGENERATE_FIGURES or not gix_figure_path.exists():
    # close=True: the saved PNG is displayed below, so the live figure is not shown twice.
    fig = viz.plot_gix(pairs, epoch, save_path=gix_figure_path, extent=extent, close=True)

display(Image(filename=str(gix_figure_path), width=760))
```

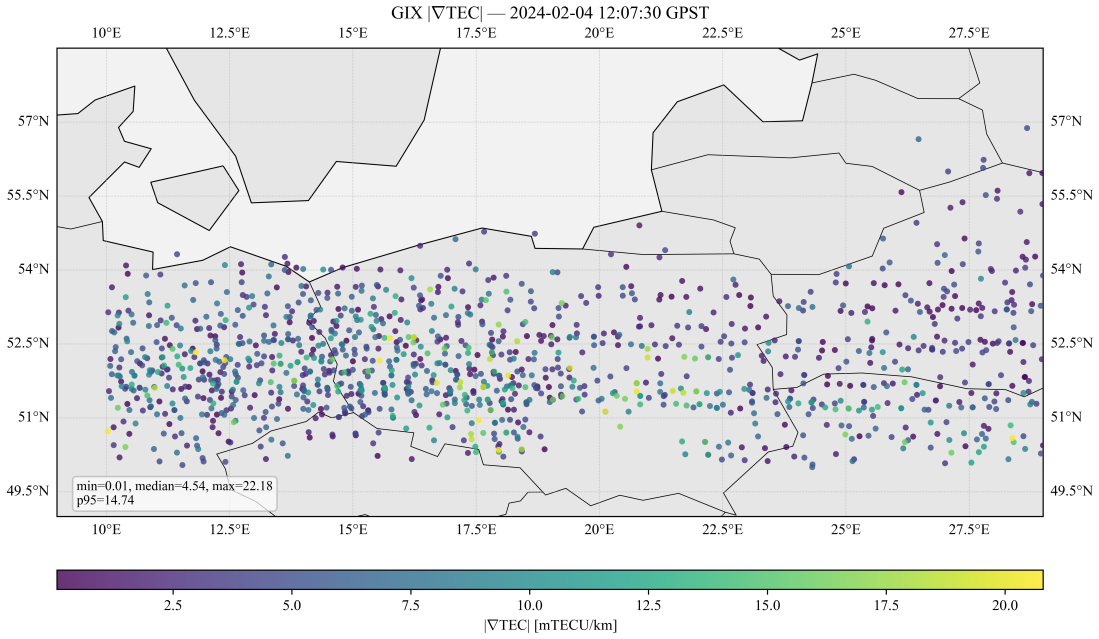


Figure 6.8: Spatial GIX example for one selected epoch.

6.1.6 Saving results

As you can see, all our data is created in pandas dataframe or series format. You can save data in any format supported by pandas. I recommend CSV or `parquet.gzip`, especially the latter in terms of space optimisation. For a tutorial repository, keep generated outputs in an ignored output directory and avoid committing large derived files unless they are intentionally curated figures for documentation.

```
## Example only - uncomment and choose an output directory when you want to persist results.
## out = TUTORIAL_DIR / "output" / "activity_indexes"
## out.mkdir(parents=True, exist_ok=True)
## sidx.to_frame().to_parquet(out / "sidx.parquet.gzip")
## times.to_parquet(out / "gix_times.parquet.gzip")
## pairs.to_parquet(out / "gix_pairs.parquet.gzip")
## roti.to_parquet(out / "roti.parquet.gzip")
```

In the next notebook, we will discuss the use of kriging to create VTEC models based on measurements from a network of GNSS stations. That workflow is more data-dependent and computationally heavier, so it should be validated separately from this activity-index introduction.

Chapter 7

Ionosphere Modelling with Kriging

In this guide, we will show how to use the kriging tools built into GNX-py to build a regional VTEC model from measured GNSS STEC. Kriging is a geostatistical interpolation method: it estimates a quantity at locations where we do not have direct measurements by combining neighbouring observations with weights derived from a spatial covariance/variogram model. In ionospheric modelling, the measured quantity is usually VTEC at ionospheric pierce points (IPPs), and the target is a regular grid of ionospheric grid points (IGPs).

The intuition is simple. If two IPPs are close to each other on the ionospheric shell, we expect their VTEC values to be more similar than values observed hundreds or thousands of kilometres apart. Kriging makes this intuition explicit through the variogram:

$$\gamma(h) = \frac{1}{2}\,\mathrm{Var}\left[Z(\mathbf{x}) - Z(\mathbf{x}+\mathbf{h})\right]$$

where $Z(\mathbf{x})$ is the VTEC field and $h = \|\mathbf{h}\|$ is the spatial separation. Equivalently, we can work with covariance:

$$C(h) = \mathrm{Cov}\left[Z(\mathbf{x}), Z(\mathbf{x}+\mathbf{h})\right]$$

For a target point $\mathbf{x}_0$, the kriging estimate is a weighted sum of nearby measurements:

$$\widehat{Z}(\mathbf{x}_0) = \sum_{i=1}^{N} w_i Z(\mathbf{x}_i)$$

The weights are not chosen only by inverse distance. They are solved from a linear system that uses the covariance/variogram between all selected IPPs and between each IPP and the target point. This is why kriging can produce both an estimate and a formal interpolation variance. The variance should not be interpreted as a full physical error budget, but it is a useful diagnostic: areas far from observations, or constrained by weak geometry, should have larger uncertainty.

GNX-py currently exposes three kriging modes through `IonoKrigingMonitor`:

Mode	Idea	Practical interpretation
`OK`	Ordinary Kriging	Assumes an unknown but locally constant mean. Good as a classical baseline.
`UK`	Universal Kriging	Adds a regional linear drift term, useful when the VTEC field has a large-scale slope across the region.

Mode	Idea	Practical interpretation
`WAAS`	Sparks/WAAS-style local kriging	Uses a local covariance model and neighbourhood selection around each grid point; this is the most ionosphere-oriented workflow in the current tutorial API.

In this notebook we use the same measured STEC network as in the activity-index tutorial. The satellite DCBs and receiver DCBs have already been handled during the STEC preparation: the files carry calibrated STEC, with the receiver DCB of every station calibrated against the CODE GIM (see the activity-index chapter and `network/README.md`), so we can convert levelled STEC to VTEC and interpolate it. This is crucial: unlike SIDX or ROTI, kriging compares different receiver-satellite links at the same epoch, so absolute levelling and bias consistency matter.

Kriging offers two major advantages in regional ionosphere modelling. First, it can generate a regional VTEC map at a finer grid spacing than products such as GIM. Second, it uses real measurements from the analysed epoch, so local structures can be visible more clearly than in empirical models such as Klobuchar, NeQuick, or NTCM. The cost is that kriging is not a time-propagation model: each map is estimated from measurements available at that epoch. It also depends strongly on the quality of calibration, station geometry, variogram choice, and neighbourhood selection.

The goal here is not to prove that one model is universally better. Instead, we will learn the GNX-py workflow: load measured STEC, prepare VTEC, choose a grid, configure `IonoKrigingMonitor`, generate a small set of maps, inspect the output dataset, compare the regional model against GIM and the broadcast models NTCM and BDGIM at common grid points, and check it station by station with a leave-one-out test.

```
from pathlib import Path

import numpy as np
import pandas as pd

import gnx_py as gnx
from gnx_py import viz

viz.apply_publication_style(font_scale=1.3, mathtext="cm")


def find_repo_root(start: Path | None = None) -> Path:
    start = (start or Path.cwd()).resolve()
    for candidate in (start, *start.parents):
        if (candidate / "gnx_py").is_dir() and (candidate / "tutorial").is_dir():
            return candidate
    raise RuntimeError("Could not find the GNX repository root. Start Jupyter from inside the cloned repository.")


REPO_ROOT = find_repo_root()
TUTORIAL_DIR = REPO_ROOT / "tutorial" / "ionosphere"
NETWORK_DIR = TUTORIAL_DIR / "network"
OUTPUT_DIR = TUTORIAL_DIR / "output" / "kriging"
SAMPLE_DATA_DIR = REPO_ROOT / "sample_data"

## Only repository-relative paths are printed, so no machine-specific path ends up in the output.
local_checkout = Path(gnx.__file__).resolve().parent == (REPO_ROOT / "gnx_py").resolve()
print(f"Network folder:  {NETWORK_DIR.relative_to(REPO_ROOT)}")
print(f"GNX-py module:   {'gnx_py/ of this repository' if local_checkout else 'an installed package'}")
```

```
Network folder:  tutorial/ionosphere/network
```

```
GNX-py module:   gnx_py/ of this repository
```

First, let us define a few parameters that control the code. `PLOT_EVERY_MIN` is the time interval at which we want to generate VTEC snapshots. For this tutorial we keep it at 60 minutes, so the run produces hourly maps. The analysis window is short on purpose: the supplied `network/` folder covers the full day, but kriging should be learned on a small run before scaling up.

`IONO_SHELL_HEIGHT_M` is the height used by the thin-shell mapping from STEC to VTEC. `ZENITH_SIGMA_TECU` is a simple measurement-noise proxy at zenith. We convert it to an elevation-dependent variance by scaling with $1/\sin(e)$:

$$\sigma_i(e)^2 = \left(\frac{\sigma_0}{\sin e_i}\right)^2$$

This is not a complete stochastic model of STEC calibration errors. It is a compact tutorial choice that tells the WAAS/Sparks solver that low-elevation observations should be trusted less.

```
PLOT_EVERY_MIN = 60
## GNX epochs are naive GPS time (GPST), the time scale of the TEC session that wrote the
    network files
ANALYSIS_START = pd.Timestamp("2024-02-04 12:00:00")
ANALYSIS_END = pd.Timestamp("2024-02-04 14:00:00")

IONO_SHELL_HEIGHT_M = 450e3
ZENITH_SIGMA_TECU = 1.5
MIN_ELEV_DEG = 30


def is_tick(ts: pd.Timestamp) -> bool:
    ts = pd.Timestamp(ts)
    if PLOT_EVERY_MIN > 60:
        plot_every_h = PLOT_EVERY_MIN // 60
        return (ts.second == 0) and (ts.hour % plot_every_h == 0) and (ts.minute == 0)
    return (ts.second == 0) and (ts.minute % PLOT_EVERY_MIN == 0)
```

Next, we set the paths where the network measurements and reference products are read from and where the generated model is written. The important part is that all paths are derived from the repository root. The notebook can therefore be executed from Jupyter, an IDE, or a converted notebook runner without assuming the current working directory is `tutorial/ionosphere`. Figures are not written to disk: every `gnx_py.viz` function returns a matplotlib `Figure`, which is shown inline and can be saved with `fig.savefig(...)` whenever you need a file.

```
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

GIM_PATH = SAMPLE_DATA_DIR / "COD0OPSFIN_20240350000_01D_01H_GIM.INX"
## RINEX 4 navigation file: every broadcast ionosphere set as a '> ION' record (NTCM and BDGIM
    below)
NAV4_PATH = SAMPLE_DATA_DIR / "BRD400DLR_S_20240350000_01D_MN.rnx"

required_paths = [NETWORK_DIR, GIM_PATH, NAV4_PATH]
missing = [str(path.relative_to(REPO_ROOT)) for path in required_paths if not path.exists()]
if missing:
    raise FileNotFoundError("Missing tutorial input data:\n" + "\n".join(missing))

print(f"Network STEC files: {len(list(NETWORK_DIR.glob('*STEC.parquet.gzip')))}")
print(f"Output directory:   {OUTPUT_DIR.relative_to(REPO_ROOT)}")
```

```
Network STEC files: 75
Output directory:   tutorial/ionosphere/output/kriging
```

Data loading and preparation is performed in the same spirit as for ionospheric activity indices. The difference is interpretation: for SIDX and ROTI we mostly cared about temporal changes along a link, whereas for kriging we need a consistent VTEC value at each IPP.

The loader returns levelled STEC in TECU. We then:

1. replace the `(sv, time)` index with a plain row index (the loader already keeps `sv` and `time` as columns too),
2. compute the mapping function $M(e; H)$,
3. create `vtec = stec / M`,
4. add receiver and satellite identifiers for optional covariance/bias terms,
5. add `meas_var`, the elevation-dependent measurement variance used by the WAAS/Sparks mode,
6. keep a short analysis window and a slightly extended spatial region around the target grid.

```
df_all, load_info = gnx.load_stec_folder(
    folder=NETWORK_DIR,
    min_elev_deg=MIN_ELEV_DEG,
    quiet=True,
    skip_negative=True,
)

## `sv` and `time` are both index levels and columms, so drop the index instead of moving it
    back.
df_all = df_all.reset_index(drop=True)
df_all["time"] = pd.to_datetime(df_all["time"])  # naive GPST, as written by TECSession
df_all["sv"] = df_all["sv"].astype(str).str[:3]
df_all["stec"] = df_all["leveled_tec"]
df_all["mf"] = gnx.mapping_M(df_all["ev"].to_numpy(), H_m=IONO_SHELL_HEIGHT_M)
df_all["vtec"] = df_all["stec"] / df_all["mf"]
df_all["rec_id"] = df_all["name"]
df_all["sat_id"] = df_all["sv"]

sigma_elev = ZENITH_SIGMA_TECU / np.sin(np.deg2rad(df_all["ev"].to_numpy()))
df_all["meas_var"] = sigma_elev**2

spatial_mask = df_all["lat_ipp"].between(45.0, 60.0) & df_all["lon_ipp"].between(5.0, 35.0)
time_mask = (df_all["time"] >= ANALYSIS_START) & (df_all["time"] < ANALYSIS_END)
df_work = df_all.loc[spatial_mask & time_mask].sort_values("time").copy()

print({k: load_info[k] for k in ["files_scanned", "files_accepted", "files_rejected"]})
print(f"Prepared rows: {len(df_work):,}")
print(f"Epochs in tutorial window: {df_work['time'].nunique()}")
```

```
{'files_scanned': 75, 'files_accepted': 60, 'files_rejected': 15}
Prepared rows: 55,809
Epochs in tutorial window: 240
```

Next, we define the grid on which we interpolate VTEC. A very dense grid, such as $0.1^\circ \times 0.1^\circ$, is useful for final figures, but it is unnecessarily heavy while learning the workflow. Here we use 0.5° spacing over the same regional area.

The important conceptual distinction is this:

- IPPs are the irregular measurement locations,
- IGPs are the regular output grid points,
- kriging estimates VTEC at IGPs from nearby IPPs.

You can reduce `GRID_STEP_DEG` later when you want publication-quality maps.

```
lat_min, lat_max = 50.0, 57.5
lon_min, lon_max = 10.0, 30.0
GRID_STEP_DEG = 0.5

lat_grid = np.arange(lat_min, lat_max + 0.5 * GRID_STEP_DEG, GRID_STEP_DEG)
```

```
lon_grid = np.arange(lon_min, lon_max + 0.5 * GRID_STEP_DEG, GRID_STEP_DEG)
region = (lon_min, lon_max, lat_min, lat_max)

print(f"Grid shape: {lat_grid.size} x {lon_grid.size} = {lat_grid.size * lon_grid.size:,} points")
```

```
Grid shape: 16 x 41 = 656 points
```

The heart of the kriging module in GNX-py is the `IonoKrigingMonitor` class. It separates the stable configuration of a grid and method from the epoch-by-epoch input measurements. This is useful because in an operational workflow we usually keep the same grid and stochastic assumptions, while the IPPs change at every epoch.

For the current tutorial we use `kriging_mode="WAAS"`, based on the Sparks/WAAS-style local approach. The core idea is local: for each IGP, the algorithm selects neighbouring IPPs and solves a covariance-based interpolation problem. The relevant parameters are:

- `Rmin_km`, `Rmax_km` — inner and outer search radii around each grid point,
- `Ntarget` — preferred number of IPPs used in the local solution,
- `Nmin` — minimum number of IPPs required to produce an estimate,
- `variogram` — covariance model that controls how quickly VTEC correlation decays with distance,
- `default_meas_var` and `meas_var` — measurement-noise variance if it is not provided or is provided per observation.

The Sparks variogram used here can be read as:

$$C(d) = (\sigma^2_{\text{total}} - \sigma^2_{\text{nominal}}) \exp\left(-\frac{d}{d_{\text{decorr}}}\right), \qquad d > 0$$

and

$$C(0) = \sigma^2_{\text{total}}$$

where d_{decorr} controls how quickly measurements become decorrelated with distance. Larger values spread information farther away; smaller values make the map more local and usually increase uncertainty away from dense IPP coverage.

`OK` and `UK` are also available. In GNX-py they use classical semivariogram parameters `sill`, `range`, and `nugget`; `UK` additionally includes a regional linear drift. They are useful for experiments and comparisons, but the WAAS/Sparks mode is the most directly tailored to this ionosphere workflow.

```
variogram = gnx.SparksVariogram(
    sigma_nominal=0.5,
    sigma_total=1.5,
    ddecorr_km=800.0,
)

mon = gnx.IonoKrigingMonitor(
    grid_lon=lon_grid,
    grid_lat=lat_grid,
    coord_frame="GEO",
    sm_transformer=None,
    sm_obstime=None,
    kriging_mode="WAAS",
    Rmin_km=600.0,
    Rmax_km=1800.0,
    Ntarget=25,
    Nmin=8,
    variogram=variogram,
    default_meas_var=ZENITH_SIGMA_TECU**2,
)
## a short summary: the full repr of the monitor also prints its grid arrays
```

```
print(f"kriging_mode={mon.kriging_mode}, grid {len(lat_grid)} x {len(lon_grid)} nodes, "
      f"Rmin/Rmax={mon.Rmin_km:.0f}/{mon.Rmax_km:.0f} km, Ntarget={mon.Ntarget}, Nmin={mon.Nmin}")
print(mon.variogram)
```

```
kriging_mode=WAAS, grid 16 x 41 nodes, Rmin/Rmax=600/1800 km, Ntarget=25, Nmin=8
SparksVariogram(sigma_nominal=0.5, sigma_total=1.5, ddecorr_km=800.0)
```

We prepare containers for storing epoch results and run kriging for the selected grid in a loop. Each epoch is handled independently. In this tutorial, `PLOT_EVERY_MIN=60`, so from a two-hour window we obtain two maps: 12:00 and 13:00 GPST.

The returned arrays are:

- `V` — gridded VTEC estimate in TECU,
- `SS` — formal kriging variance in TECU^2,
- `info` — diagnostic metadata such as mode, number of input points and neighbourhood settings.

We save the small result to NetCDF because this is the format used by the rest of the tutorial tooling and by xarray.

```
times_all = []
V_all = []
SS_all = []
info_all = []

for epoch_time, df_epoch in df_work.groupby("time"):
    if not is_tick(epoch_time):
        continue
    V, SS, info = mon.krige_epoch(df_epoch, epoch_time=epoch_time)
    times_all.append(epoch_time)
    V_all.append(V.astype(np.float32))
    SS_all.append(SS.astype(np.float32))
    info_all.append(info)

if not times_all:
    raise RuntimeError("No kriging epochs were selected. Check PLOT_EVERY_MIN and the analysis time window.")

ds = gnx.save_grids_to_netcdf(
    times=times_all,
    V_all=V_all,
    SS_All=SS_all,
    lat_grid=lat_grid,
    lon_grid=lon_grid,
    out_path=str(OUTPUT_DIR),
    out_name="model",
)

print(f"Generated epochs: {len(times_all)}")
print(info_all[0])
```

```
Generated epochs: 2
{'mode': 'WAAS', 'coord_frame': 'GEO', 'n_points': 232, 'n_estimated': 656, 'variogram': {'
    model': 'sparks', 'sigma_nominal': 0.5, 'sigma_total': 1.5, 'ddecorr_km': 800.0, 'sill_var
    ': 2.25}, 'sparks_vario': {'model': 'sparks', 'sigma_nominal': 0.5, 'sigma_total': 1.5, '
    ddecorr_km': 800.0, 'sill_var': 2.25}, 'Rmin_km': 600.0, 'Rmax_km': 1800.0, 'Ntarget': 25,
    'Nmin': 8}
```

The results are stored in `xarray.Dataset` format and saved in NetCDF format. The variables stored in the dataset are `V` (VTEC) and `SS` (kriging variance). The `time` coordinate holds naive GPST epochs and carries the attribute `time_scale="GPST"`. The data layout is `(time, lat, lon)`, which makes it convenient to select time slices, compute statistics, compare with GIM/NTCM, and plot maps.

To display the model we use `viz.plot_vtec_map` from `gnx_py.viz`. It draws one variable at one epoch on a Cartopy map with coastlines, borders, labelled gridlines and a colorbar, and returns the matplotlib `Figure`, so it can be adjusted further or saved with `fig.savefig(...)`.

```
print(ds.data_vars)
```

```
Data variables:
    V        (time, lat, lon) float32 5kB 32.26 32.07 31.94 ... 25.58 25.45
    SS       (time, lat, lon) float32 5kB 0.6863 0.6841 0.6858 ... 4.149 4.283
```

```
list(ds.coords)
```

```
['time', 'lat', 'lon']
```

```
int(len(ds.time))
```

```
2
```

```
t_index = len(ds.time) // 2
time = pd.Timestamp(ds["time"][t_index].values)

fig_vtec = viz.plot_vtec_map(
    ds,
    "V",
    t_index=t_index,
    region=region,
    cmap="viridis",
    cbar_label="VTEC [TECU]",
    title=f"VTEC map @ {time} GPST",
    figsize=(10, 10),
)

fig_var = viz.plot_vtec_map(
    ds,
    "SS",
    t_index=t_index,
    region=region,
    cmap="viridis",
    cbar_label=r"Kriging variance [TECU$^2$]",
    title=f"Kriging variance map @ {time} GPST",
    figsize=(10, 10),
)
```

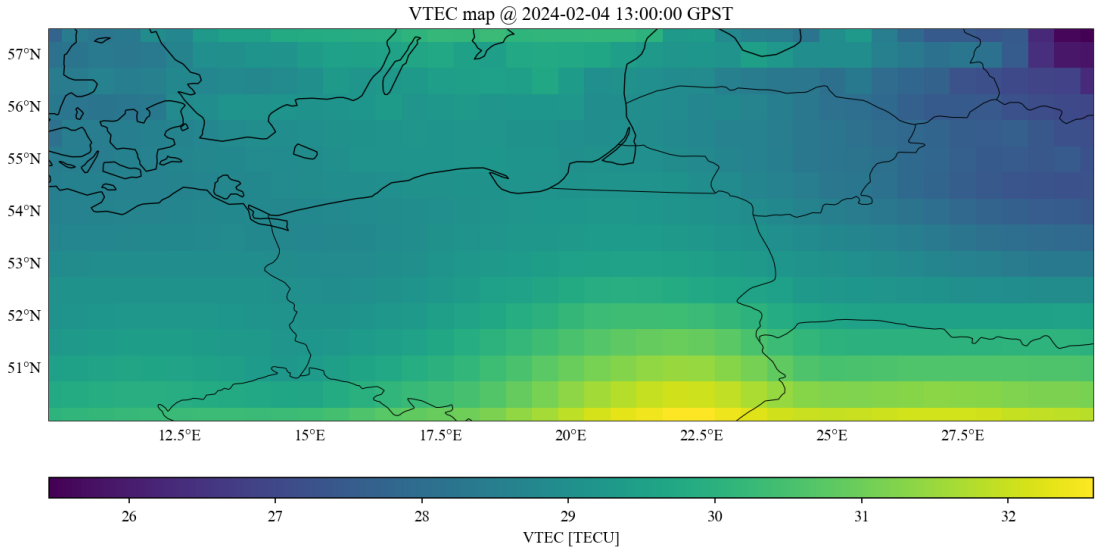


Figure 7.1: Regional VTEC map estimated with the GNX-py kriging workflow.

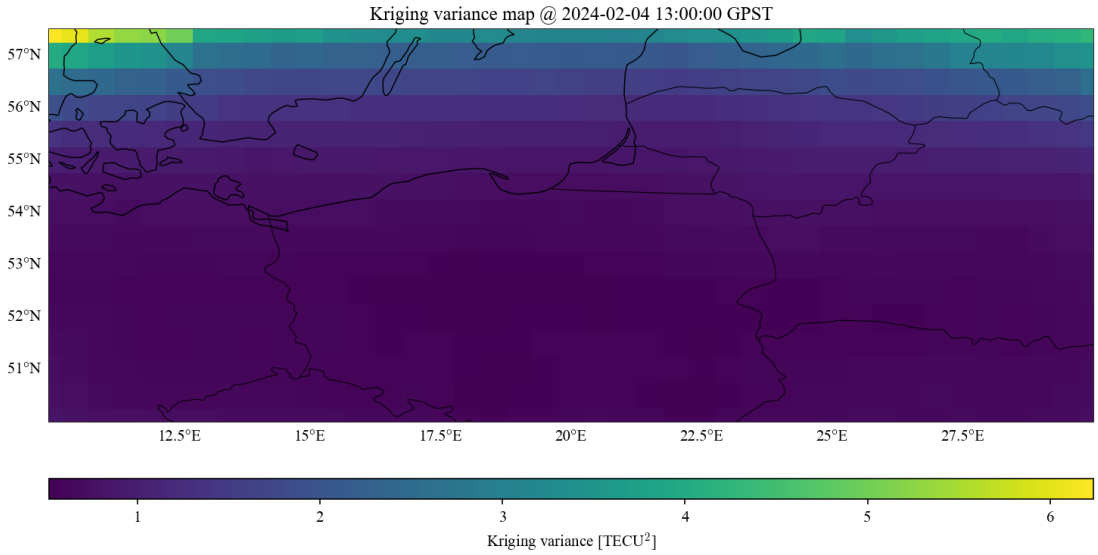


Figure 7.2: Formal kriging variance for the same VTEC map epoch.

Let us compare our model with GIM, and the broadcast models NTCM and BDGIM with GIM as well. It is worth recalling what we said at the beginning: GIM is a global assimilation product with a coarser native grid, while our kriging grid is regional and measurement-driven. Therefore this comparison should not be read as a definitive ranking of models. It is a diagnostic exercise:

- do the regional VTEC levels look plausible against GIM?
- how far are the broadcast models NTCM and BDGIM from the measurement-driven map?
- are differences spatially coherent or dominated by isolated artefacts?

One caveat applies to this particular network: its receiver DCBs were calibrated against the same CODE GIM that serves as the reference here, so the overall level of the kriging map is tied to that GIM by construction. The comparison mainly shows the spatial structure and the local differences; the leave-one-out test at the end of the chapter checks the interpolation independently of any GIM.

First, we load the IONEX file using GNX-py.

```
reader = gnx.GIMReader(tec_path=GIM_PATH)
```

```
ionex = reader.read()
inx_tec = ionex.tec
## a short summary: the dataset attributes also hold the full path of the file
print(f"TEC maps {dict(inx_tec['TEC'].sizes)}")
print(f"epochs {pd.Timestamp(inx_tec.time.values[0])} to {pd.Timestamp(inx_tec.time.values[-1])} "
      f"(UTC, IONEX map epochs), interval {inx_tec.attrs['interval_s']:.0f} s")
```

```
TEC maps {'time': 25, 'lat': 71, 'lon': 73}
epochs 2024-02-04 00:00:00 to 2024-02-05 00:00:00 (UTC, IONEX map epochs), interval 3600 s
```

To calculate TEC from the broadcast models we need their coefficients from the navigation message: the three Galileo coefficients of NTCM and the nine BeiDou-3 coefficients of BDGIM. A RINEX 4 navigation file keeps every broadcast set as a `> ION` record with its transmission time. `gnx_py.io.rinex_nav.read_iono_table` reads these records and checks each one for plausibility, and `rinex_nav.broadcast_iono_at` returns the set in force at each epoch (the last plausible set broadcast before it), the rule the SPP, PPP and TEC sessions apply to their broadcast ionosphere model. On this day the file holds two filler records of the BeiDou satellite C48, which the reader rejects with the warnings printed below, and the Galileo coefficients change at 14:11 GPST, after our map window. This gives two empirical-model baselines on the same grid and at the same epochs as the kriging model.

```
import os
import warnings

from gnx_py.io import rinex_nav

## The reader warns once for every rejected record; the messages name the file by its full path, so they are
## collected here and printed relative to the repository.
with warnings.catch_warnings(record=True) as captured:
    warnings.simplefilter("always")
    iono_table = rinex_nav.read_iono_table(NAV4_PATH, systems="EC")
for item in captured:
    print(f"warning: {str(item.message).replace(f'{REPO_ROOT}{os.sep}', '')}")

map_epochs = pd.DatetimeIndex(ds.time.values, name="time")  # naive GPST
gal_sets = rinex_nav.broadcast_iono_at(iono_table, "E", "NEQG", map_epochs)
bdgim_sets = rinex_nav.broadcast_iono_at(iono_table, "C", "BDGIM", map_epochs)
viz.df_head(pd.DataFrame(gal_sets, index=map_epochs, columns=["ai0", "ai1", "ai2"]), floatfmt=".4g")
```

```
warning: sample_data/BRD400DLR_S_20240350000_01D_MN.rnx: broadcast ionosphere record '> ION C48 D1D2' 2024-02-04 23:30:44 rejected: all eight raw Klobuchar counts equal 85 (filler record)
warning: sample_data/BRD400DLR_S_20240350000_01D_MN.rnx: broadcast ionosphere record '> ION C48 CNVX' 2024-02-04 04:24:14 rejected: all nine raw BDGIM words equal 0xAA modulo 256 (filler record)
```

time	ai0	ai1	ai2
2024-02-04 12:00:00	144	-0.2109	-0.002319
2024-02-04 13:00:00	144	-0.2109	-0.002319

We now generate NTCM and BDGIM grids corresponding to the kriging grid. `compute_ntcm_grid` takes the latitude vector, longitude vector, the epochs (GPST) and one Galileo coefficient set, so we call it once per epoch with the set in force. `compute_bdgim_grid` (module `gnx_py.ionosphere.bdgim`) takes one coefficient set per epoch and needs the time scale of the epochs explicitly. Both evaluate the model at the grid nodes, taken as ionospheric points, without a line of sight or a mapping function; the BDGIM grid is not clamped at 0.

```
import xarray as xr
from gnx_py.ionosphere.bdgim import compute_bdgim_grid
```

```
3
4  lat_grid_cmp = ds["V"].lat.values
5  lon_grid_cmp = ds["V"].lon.values
6  times_cmp = ds.time.values
7
8  ds_ntcm = xr.concat(
9      [gnx.compute_ntcm_grid(lat_grid_cmp, lon_grid_cmp, [epoch], gal_alpha=list(gal_set))
10      for epoch, gal_set in zip(times_cmp, gal_sets)],
11     dim="time",
12 )
13 ds_bdgim = compute_bdgim_grid(lat_grid_cmp, lon_grid_cmp, times_cmp, bdgim_sets, time_scale="
       GPST")
14 ds_bdgim = ds_bdgim.rename({"vtec": "V"})
15 ds_ntcm
```

section	summary
dataset	xarray.Dataset Size: 11kB
dimensions	(time: 2, lat: 16, lon: 41)
coord time	(time) datetime64[ns] 16B 2024-02…
coord lat	(lat) float64 128B 50.0 50.5 51.0…
coord lon	(lon) float64 328B 10.0 10.5 11.0…
variable	V (time, lat, lon) float64…

Datasets are convenient because comparison can be performed at common coordinates. `viz.compare_vtec_maps` maps the absolute TEC difference between two datasets at their common grid nodes and returns the figure together with the mean and median absolute differences (in the map titles they appear as `MAE` and `MedAE`; both are computed from $|\Delta\text{TEC}|$, so neither is a signed mean error). We compare every generated epoch to collect these statistics, but keep only the middle epoch on screen: the other figures are closed right away with `close=True`. BDGIM is compared in the same way, but only its statistics are kept. The epochs are matched by their labels: the IONEX maps are labelled in UTC and the kriging maps in GPST, so 12:00 GPST is compared with the 12:00 UTC map, 18 s apart, which is negligible for hourly maps. The time in the map titles is this common label.

Keep in mind that when comparing kriging and GIM, the goal is not to show which model is objectively "better". They are produced using different assumptions, resolutions and input data. Here we use GIM as a reference surface for a quick sanity check of regional VTEC levels.

```
1  mns = []
2  mns_ntcm = []
3  mns_bdgim = []
4  comparison_time_index = len(ds.time) // 2
5
6  for num, epoch_time in enumerate(ds.time.values):
7      show_this = num == comparison_time_index
8
9      fig_kriging, mean_abs, median_abs = viz.compare_vtec_maps(
10         ds,
11         inx_tec,
12         epoch_time,
13         var_model="V",
14         var_ref="TEC",
15         region=region,
16         cmap="Blues",
17         figsize=(5, 5),
18         dpi=300,
19         title_prefix=r"Δ|TEC| = |TEC$_\mathrm{GIM}$ - TEC$_\mathrm{KRIGING}$|",
20         close=not show_this,
21     )
22     mns.append(mean_abs)
23
24     fig_ntcm, mean_abs, median_abs = viz.compare_vtec_maps(
25         ds_ntcm,
26         inx_tec,
27         epoch_time,
```

```
        var_model="V",
        var_ref="TEC",
        region=region,
        cmap="Blues",
        figsize=(5, 5),
        dpi=300,
        title_prefix=r"Δ|TEC| = |TEC$_\mathrm{GIM}$ - TEC$_\mathrm{NTCM}$|",
        close=not show_this,
    )
    mns_ntcm.append(mean_abs)

    _, mean_abs, _ = viz.compare_vtec_maps(
        ds_bdgim,
        inx_tec,
        epoch_time,
        var_model="V",
        var_ref="TEC",
        region=region,
        title_prefix=r"Δ|TEC| = |TEC$_\mathrm{GIM}$ - TEC$_\mathrm{BDGIM}$|",
        close=True,
    )
    mns_bdgim.append(mean_abs)
```

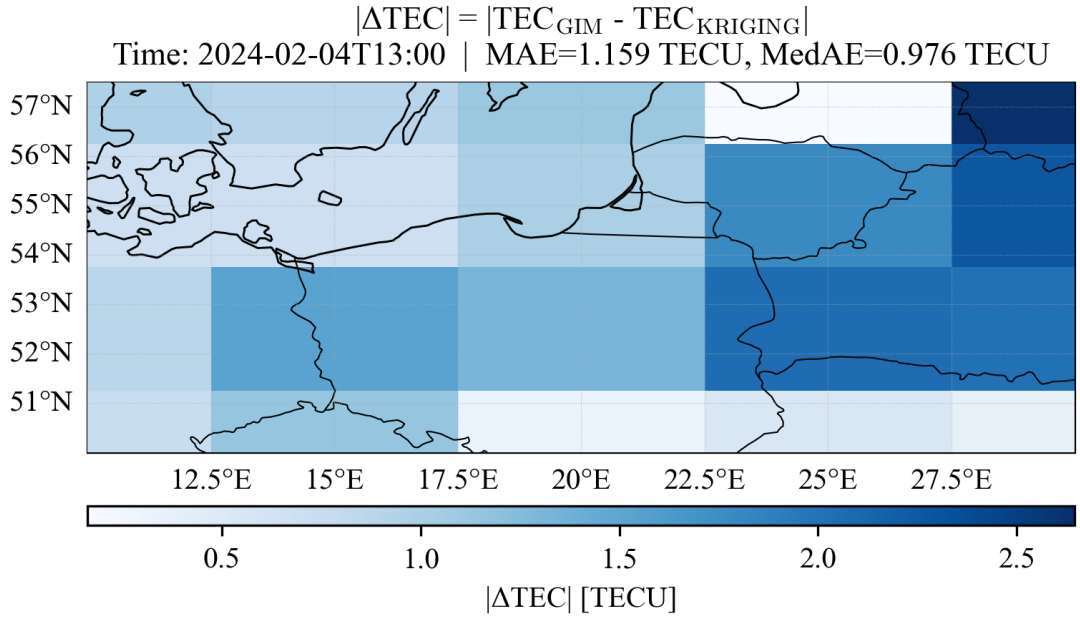


Figure 7.3: Absolute VTEC difference between the kriging map and GIM at a common epoch.

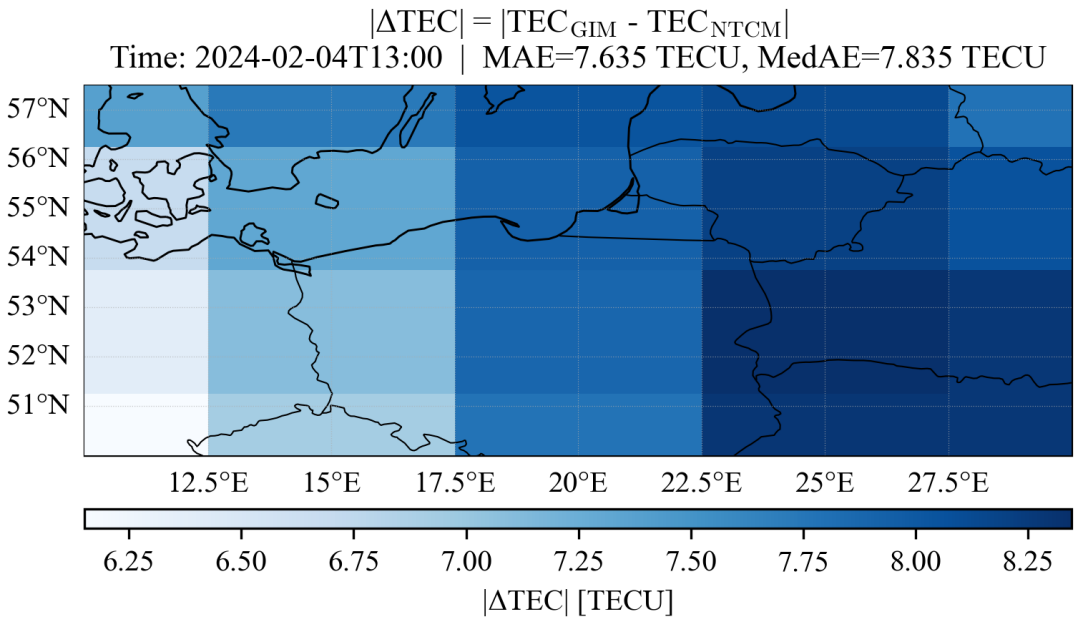


Figure 7.4: Absolute VTEC difference between the NTCM grid and GIM at a common epoch.

As you can see, the TEC differences between NTCM and GIM are not the same type of signal as the differences between kriging and GIM. NTCM and BDGIM are empirical broadcast models; kriging is a measurement-driven regional interpolation. Finally, let's plot the mean absolute error at each generated epoch on a common graph.

```
errors = pd.DataFrame(
    {"Kriging": mns, "NTCM": mns_ntcm, "BDGIM": mns_bdgim},
    index=pd.RangeIndex(len(mns), name="Map number"),
)
fig = viz.plot_vtec_error_series(errors, reference="GIM", metric="MAE", units="TECU")
```

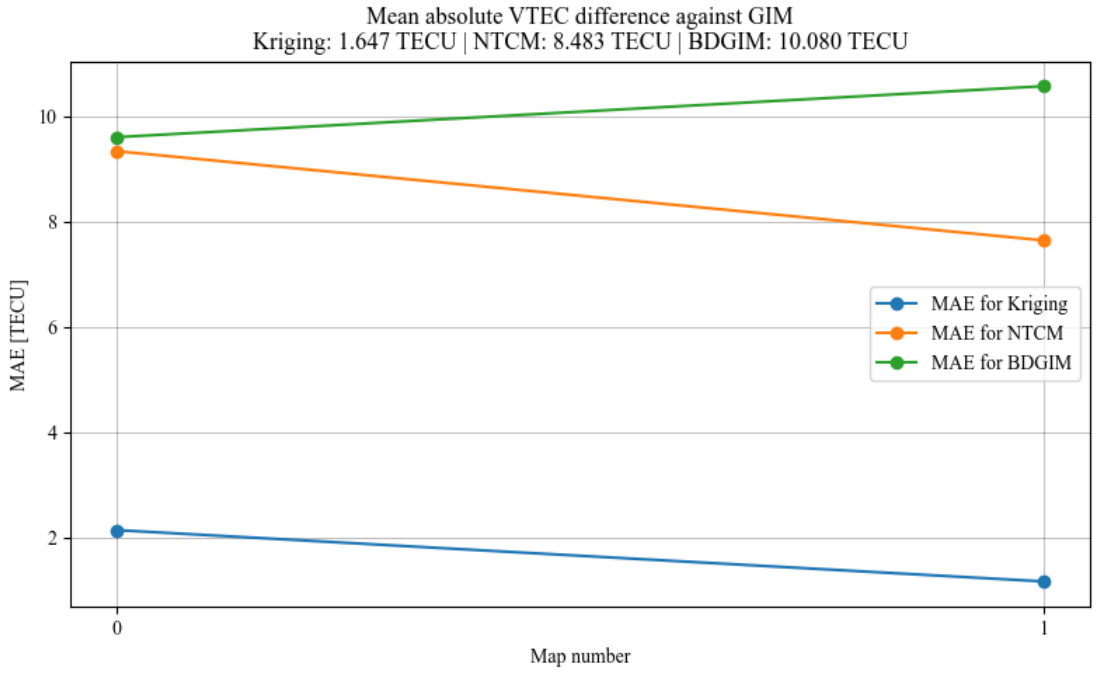


Figure 7.5: Mean absolute VTEC difference against GIM for the kriging, NTCM and BDGIM maps.

7.1 Leave-one-station-out check

A comparison with GIM says little about the places between the stations, where the interpolation matters most. A simple independent check is leave-one-out: remove one station (both of its GPS and Galileo links) from an epoch, krige VTEC at that station's IPPs from the remaining stations, and compare the result with the VTEC the station measured. Repeating this for every station gives an RMS per station. We use the variogram and neighbourhood settings from above, at the epochs of the maps.

```
loo_errors = {}
for epoch_time in ds.time.values:
    df_epoch = df_work[df_work["time"] == pd.Timestamp(epoch_time)]
    station = df_epoch["name"].str[:4]  # file names start with the 4-character station id
    for station_id, df_station in df_epoch.groupby(station):
        loo_mon = gnx.IonoKrigingMonitor(
            grid_lon=df_station["lon_ipp"].to_numpy()[None, :],
            grid_lat=df_station["lat_ipp"].to_numpy()[None, :],
            coord_frame="GEO",
            kriging_mode="WAAS",
            Rmin_km=600.0,
            Rmax_km=1800.0,
            Ntarget=25,
            Nmin=8,
            variogram=variogram,
            default_meas_var=ZENITH_SIGMA_TECU**2,
        )
        V_loo, _, _ = loo_mon.krige_epoch(df_epoch[station != station_id], epoch_time=pd.
    Timestamp(epoch_time))
        loo_errors.setdefault(station_id, []).extend(V_loo[0] - df_station["vtec"].to_numpy())

loo = pd.DataFrame({
    "n_ipp": {s: int(np.isfinite(e).sum()) for s, e in loo_errors.items()},
    "rms_tecu": {s: float(np.sqrt(np.nanmean(np.square(e)))) for s, e in loo_errors.items()},
}).rename_axis("station").sort_values("rms_tecu", ascending=False)
print(f"stations: {len(loo)}, median RMS {loo['rms_tecu'].median():.2f} TECU, "
      f"worst {loo.index[0]} {loo['rms_tecu'].iloc[0]:.2f} TECU")
viz.df_head(loo, nrows=5, ncols=3, floatfmt=".2f")
```

```
stations: 36, median RMS 1.19 TECU, worst PINS 2.10 TECU
```

station	n_ipp	rms_tecu
PINS	19	2.10
KLNK	14	2.04
CLIB	15	2.04
BRTS	19	2.04
CPAR	15	1.92

The table lists the stations with the largest leave-one-out RMS. Stations at the edge of the network are usually among them, because their IPPs must be extrapolated from one side; a single station far above the others would rather point to a problem in its STEC, for example a wrong receiver DCB. The same loop with another variogram or another neighbourhood (`Rmin_km`, `Rmax_km`, `Ntarget`) is a fair way to compare settings. It is written for `WAAS`: `OK` and `UK` need a rectilinear grid, so they cannot take the scattered pierce points of the left-out station as one grid.

You now know the practical GNX-py kriging workflow. Together with the functions that calculate ionospheric activity indices, kriging can be used to monitor and visualise regional electron content changes during quiet and disturbed ionospheric conditions.

A few practical rules are worth remembering:

- kriging needs calibrated or consistently levelled STEC/VTEC,
- the result quality depends on station geometry and IPP coverage,
- variogram parameters are not universal constants and should be validated for the region and time period,

- `SS` is a formal interpolation variance, not a complete physical error estimate,
- a leave-one-out test is a simple check of the interpolation that needs no external product,
- `OK`, `UK`, and `WAAS` answer slightly different modelling questions, so compare them before adopting one as default.

You already know the SPP and PPP modules. A useful follow-up experiment is to connect a kriging-derived VTEC model to positioning and compare positioning residuals against GIM, Klobuchar/NTCM, or unmodelled ionosphere cases.

The VTEC model generated using kriging is stored in xarray/NetCDF format. The file can be loaded with `xarray.load_dataset()` or `xarray.open_dataset()`:

```
import xarray as xr
loaded = xr.load_dataset(TUTORIAL_DIR / "output" / "kriging" / "model.nc")
```

The xarray library is convenient for grid data because it keeps dimensions, coordinates and variables together. That makes it natural to select epochs, interpolate, compare with GIM-like products, and export figures for reports.

Chapter 8

References

Arikan, F., H. Nayir, U. Sezen, and O. Arikan. 2008. "Estimation of Single-Station Inter-Frequency Receiver Bias Using GPS-TEC." *Radio Science* 43 (4): RS4004. https://doi.org/10.1029/2007RS003785.

Carmo, C. S. do, C. M. Denardini, Â. Figueiredo, et al. 2021. "Evaluation of Different Methods for Calculating the ROTI Index over the Brazilian Sector." *Radio Science* 56 (4): e2020RS007140. https://doi.org/10.1029/2020RS007140.

Cherniak, Iurii, Andrzej Krankowski, and Irina Zakharenkova. 2018. "ROTI Maps: A New IGS Ionospheric Product Characterizing the Ionospheric Irregularities Occurrence." *GPS Solutions* 22: 69. https://doi.org/10.1007/s10291-018-0730-1.

European Union Agency for the Space Programme. 2021. *European GNSS (Galileo) Open Service Signal-in-Space Interface Control Document*. Issue 2.0. European Union. https://www.gsc-europa.eu/sites/default/files/sites/all/files/Galileo-OS-SIS-ICD.pdf.

Glaner, M. F. 2022. *Towards Instantaneous PPP Convergence Using Multiple GNSS Signals*. TU Wien Thesis / Online PDF. https://repositum.tuwien.at/bitstream/20.500.12708/95695/1/Glaner%20Marcus%20Franz%20-%202022%20-%20Towards%20instantaneous%20PPP%20convergence%20using...pdf.

HofmannWellenhof, Bernhard, Herbert Lichtenegger, and Elmar Wasle. 2008. *GNSS – Global Navigation Satellite Systems: GPS, GLONASS, Galileo and More*. Springer Vienna. https://doi.org/10.1007/978-3-211-73017-1.

Jakowski, N., and M. M. Hoque. 2019. "Estimation of Spatial Gradients and Temporal Variations of the Total Electron Content Using Ground-Based GNSS Measurements." *Space Weather* 17: 339–56. https://doi.org/10.1029/2018SW002119.

Kouba, Jan, and Pierre Héroux. 2001. "Precise Point Positioning Using IGS Orbit and Clock Products." *GPS Solutions* 5 (2): 12–28. https://doi.org/10.1007/PL00012883.

Labbe, Roger R. Jr. 2020. *Kalman and Bayesian Filters in Python*. GitHub repository. https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python.

Mannucci, A. J., B. D. Wilson, D. N. Yuan, C. M. Ho, U. J. Lindqwister, and B. A. Iijima. 1998. "A Global Mapping Technique for GPS-Derived Ionospheric Total Electron Content Measurements." *Radio Science* 33 (3): 565–82. https://doi.org/10.1029/97RS02707.

Mervart, Leos. 1995. "Ambiguity Resolution Techniques in Geodetic and Geodynamic Applications of the Global Positioning System." PhD thesis, University of Bern, Astronomical Institute. https://zhw-b.s3.cloud.switch.ch/aiub/papers/lmdiss.ps.

Montenbruck, Oliver, Peter Steigenberger, and André Hauschild. 2015. "Broadcast Versus Precise Ephemerides: A Multi-GNSS Perspective." *GPS Solutions* 19 (2): 321–33. https://doi.org/10.1007/s10291-014-0390-8.

Montenbruck, Oliver, Peter Steigenberger, and André Hauschild. 2018. "Multi-GNSS Signal-in-Space Range Error Assessment – Methodology and Results." *Advances in Space Research* 61 (12): 3020–38. https://doi.org/10.1016/j.asr.2018.03.041.

National Executive Committee for Space-Based Positioning, Navigation, and Timing. 2023. *Global Positioning Systems Directorate: Navstar GPS Space Segment/Navigation User Interfaces: IS-GPS-200N*. U.S. Government. https://www.gps.gov/technical/icwg/IS-GPS-200N.pdf.

Sardón, E., A. Rius, and N. Zarraoa. 1994. "Estimation of the Transmitter and Receiver Differential Biases and the Ionospheric Total Electron Content from Global Positioning System Observations." *Radio Science* 29 (3): 577–86. https://doi.org/10.1029/94RS00449.

Schaer, Stefan. 1999. "Mapping and Predicting the Earth's Ionosphere Using the Global Positioning System." PhD thesis, Astronomical Institute, University of Bern. https://mediatum.ub.tum.de/doc/1365835/647845.pdf.

Schaer, Stefan. 2018. *SINEX BIAS — Solution (Software/Technique) INdependent EXchange Format for GNSS Biases, Version 1.00*. Swisstopo / AIUB. https://files.igs.org/pub/data/format/sinex_bias_100.pdf.

Schönemann, Erik. 2014. "Analysis of GNSS Raw Observations in PPP Solutions." PhD thesis, Technische Universität Darmstadt. https://tuprints.ulb.tu-darmstadt.de/server/api/core/bitstreams/b434977f-79d8-410c-833a-00a5d316f723/content.

Teunissen, Peter J. G., and Oliver Montenbruck, eds. 2017. *Springer Handbook of Global Navigation Satellite Systems*. Springer International Publishing. https://link.springer.com/book/10.1007/978-3-319-42928-1.